

## Índice

|                                       |    |
|---------------------------------------|----|
| Que es una Arquitectura.....          | 5  |
| Generaciones de la Arquitectura ..... | 6  |
| Generación 0.....                     | 6  |
| Generación 1.....                     | 6  |
| Generación 2.....                     | 6  |
| Generación 3.....                     | 7  |
| Generación 4.....                     | 7  |
| Generación 5.....                     | 7  |
| Codificación de la Información .....  | 7  |
| Código BCD.....                       | 7  |
| Empaquetado y Desempaquetado .....    | 8  |
| Códigos Redundantes .....             | 8  |
| Codificación.....                     | 9  |
| Código Hamming.....                   | 10 |
| Circuitos Combinacionales.....        | 13 |
| Circuitos Secuenciales.....           | 13 |
| Biestables .....                      | 14 |
| Tipos de Biestables.....              | 14 |
| RS .....                              | 14 |
| D .....                               | 15 |
| JK .....                              | 16 |
| T.....                                | 16 |
| Buses y Registros.....                | 17 |
| Registros.....                        | 17 |
| Tipos de Registros .....              | 17 |
| Buses .....                           | 18 |
| Registro de Desplazamiento .....      | 19 |
| Registro de Contador .....            | 19 |
| ABACUS .....                          | 20 |
| ALU .....                             | 21 |
| SemiSumador .....                     | 22 |
| .....                                 | 22 |
| Sumador.....                          | 22 |
| Sumador en Serie.....                 | 23 |

|                                                                   |    |
|-------------------------------------------------------------------|----|
| SemiSustractor .....                                              | 23 |
| Sumador Paralelo.....                                             | 24 |
| Gobierno Secuencial de Desplazamiento .....                       | 24 |
| NUMEROS BINARIOS CON SIGNO .....                                  | 25 |
| Desbordamiento .....                                              | 27 |
| COMA FLOTANTE .....                                               | 30 |
| Ajuste de Signos .....                                            | 32 |
| Esquema de Control del sumador/sustractor con coma flotante ..... | 33 |
| MULTIPLICACION EN ABACUS.....                                     | 36 |
| DIVISION EN ABACUS .....                                          | 37 |
| DIVISION CON RESTAURACION .....                                   | 38 |
| Memoria.....                                                      | 39 |
| Tipos de Memoria .....                                            | 40 |
| Jerarquía de memorias .....                                       | 41 |
| NIVEL 0: .....                                                    | 42 |
| NIVEL 1: .....                                                    | 42 |
| NIVEL 2 .....                                                     | 44 |
| NIVEL 4: .....                                                    | 44 |
| NIVEL 5 .....                                                     | 45 |
| NIVEL 6 .....                                                     | 45 |
| Ciclo de Histéresis .....                                         | 46 |
| Anillos de Ferrita .....                                          | 47 |
| Implementación 2D.....                                            | 47 |
| Implementación 3D.....                                            | 49 |
| Ventajas y Desventajas de ambas.....                              | 49 |
| RAM ESTÁTICA .....                                                | 50 |
| RAM DINÁMICA .....                                                | 50 |
| PUNTO MEMORIA 2D.....                                             | 51 |
| PUNTO DE MEMORIA 3D .....                                         | 52 |
| ORGANIZACIÓN DE MEMORIA DE 2 ½ D.....                             | 52 |
| METODOS DE DIRECCIONAMIENTO.....                                  | 53 |
| Direccionamiento Directo .....                                    | 54 |
| Direccionamiento Inmediato .....                                  | 54 |
| Direccionamiento Indirecto .....                                  | 55 |
| Direccionamiento por Base.....                                    | 56 |
| Direccionamiento por Referencia al Programa.....                  | 57 |

|                                                           |    |
|-----------------------------------------------------------|----|
| Direccionamiento por Página.....                          | 58 |
| Direccionamiento Indexado.....                            | 59 |
| RUTA DE DATOS .....                                       | 59 |
| Cronograma de la Suma .....                               | 62 |
| DEFINICIÓN .....                                          | 63 |
| Organización de los Registros de la Ruta de Datos.....    | 67 |
| Plano Funcional.....                                      | 67 |
| Plano Estructural.....                                    | 68 |
| SECUANCIADOR .....                                        | 69 |
| Modelo Cableado y Microprogramado.....                    | 70 |
| Secuenciador Cableado Sincrónico .....                    | 71 |
| Secuenciador Cableado Asincrónico .....                   | 72 |
| Distribuidor de Fase .....                                | 73 |
| Cronogramas.....                                          | 75 |
| Operaciones Lógicas .....                                 | 78 |
| Secuenciador Microprogramado: WILKES .....                | 80 |
| Codificación por campo .....                              | 81 |
| Codificación por Tipo de Instrucción .....                | 82 |
| Métodos de direccionamiento.....                          | 83 |
| Acompasamiento .....                                      | 84 |
| CISC y RISC.....                                          | 86 |
| CISC .....                                                | 86 |
| RISC .....                                                | 87 |
| Comparación .....                                         | 89 |
| LENGUAJE MAQUINA.....                                     | 90 |
| Concepto .....                                            | 90 |
| Clasificación de Arquitecturas.....                       | 91 |
| CANALES.....                                              | 94 |
| Métodos de Transferencia .....                            | 94 |
| Modo Bloqueado .....                                      | 94 |
| Modo por Prueba de Estado .....                           | 94 |
| Modo por Interrupción de Programa .....                   | 95 |
| Modo Canal.....                                           | 95 |
| Modo Encadenamiento Automático de las Transferencias..... | 96 |
| Interfaz.....                                             | 96 |
| Esquemas de conexión de Canales con Controladores .....   | 97 |

|                                                           |     |
|-----------------------------------------------------------|-----|
| Concentración por Multiplaje .....                        | 97  |
| Concentración por Línea ómnibus .....                     | 97  |
| Técnicas de Transferencias .....                          | 98  |
| Transferencia Programada.....                             | 98  |
| Transferencia por Instrucción Forzada .....               | 98  |
| Transferencia por Robo de Ciclo .....                     | 99  |
| Transferencia por Acceso Directo a Memoria .....          | 99  |
| Enlace Programado .....                                   | 100 |
| Canal Automático.....                                     | 100 |
| Programa Canal .....                                      | 101 |
| Arquitectura 8086.....                                    | 104 |
| Arquitectura .....                                        | 104 |
| Registros de Direcciones de Segmentos .....               | 104 |
| Registros de Propósito General .....                      | 105 |
| Los Registros de Direcciones de Datos (Apuntadores) ..... | 105 |
| Registros de Banderas.....                                | 106 |
| Modos de Direccionamiento.....                            | 106 |
| Datos Extras .....                                        | 106 |

## Que es una Arquitectura

Máquina digital electrónica programable para el tratamiento automático de información/datos. Realiza operaciones. (Computadora más que arquitectura).

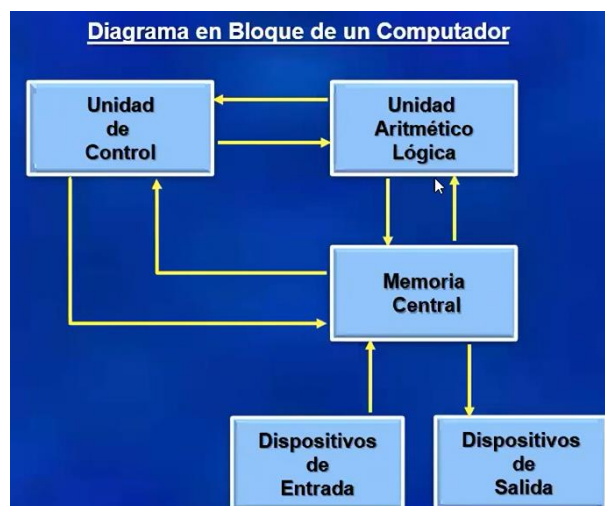
Modelo de sistema en un determinado contexto, que representa los componentes necesarios para desarrollar dicho sistema.

**Ing. Gustavo Maurokefalidis**

**Una arquitectura de Computadora determina el criterio lógico necesario de diseño para definir y relacionar los componentes de un Ordenador sin perder de vista el campo de aplicación del mismo.**

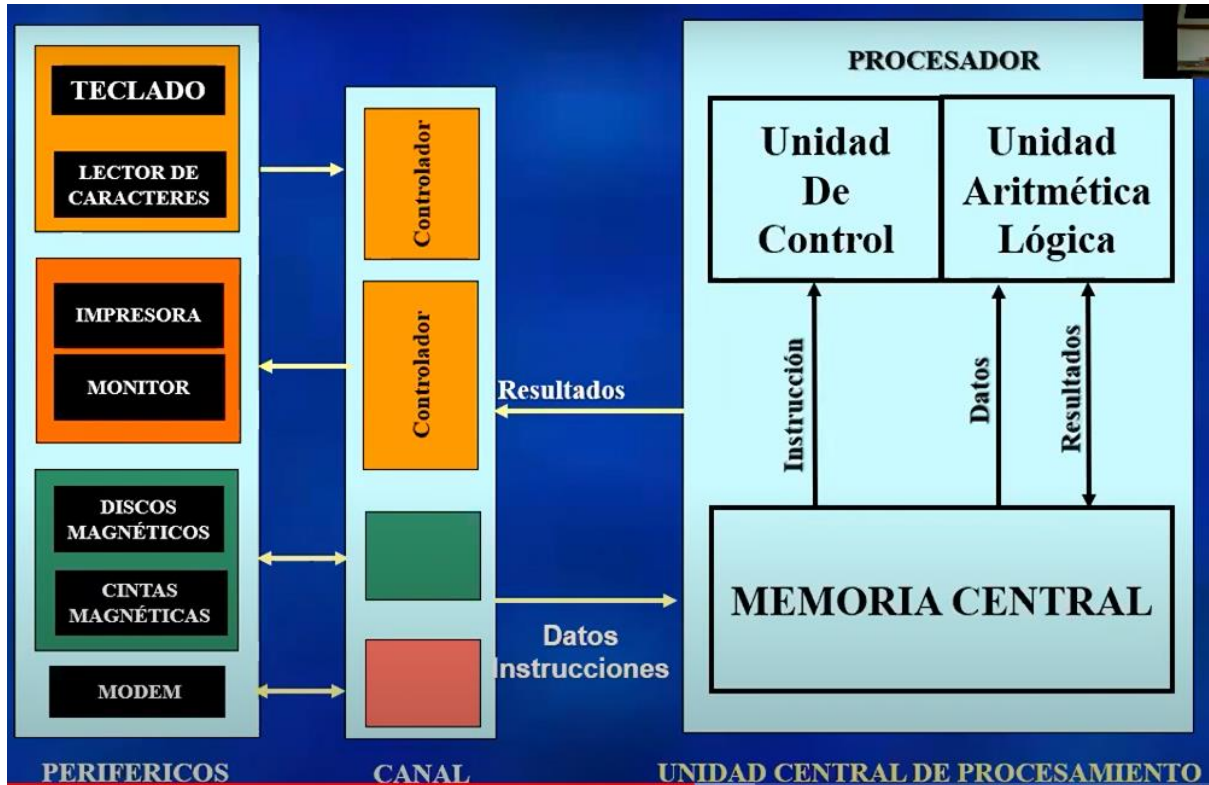


Esquema general de una arquitectura:



Nada se procesa si no pasa por la memoria central, ya que es la encargada de administrar los datos que ingresan y salen. (Memoria RAM)

Modelo planteado por Meinadier:



## Generaciones de la Arquitectura

### Generación 0

- Hasta 1945.
- Sistemas mecánicos y electromecánicos
- Ábacos, sistemas basados en relés (Maquina Diferencial, Harvard Mark I), calculadoras mecánicas (Pascalina).

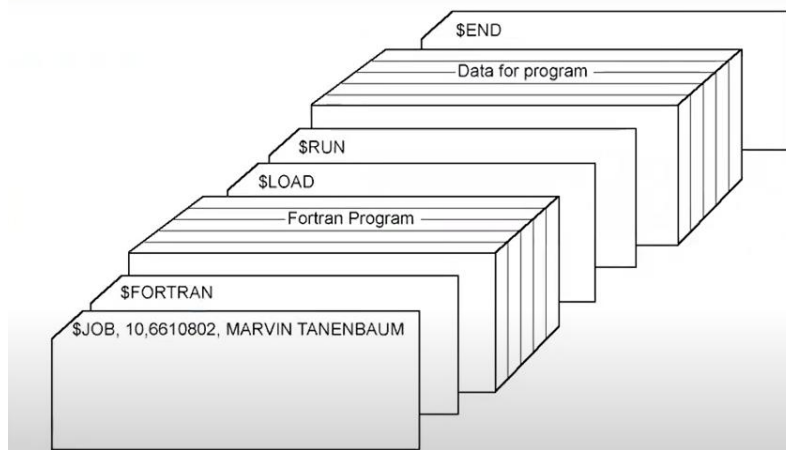
### Generación 1

- 1945-1955
- Tubos al vacío, tableros.
- Se utilizan las válvulas como compuertas lógicas.
- Sistemas muy grandes y caros, necesitaban mucho mantenimiento.

### Generación 2

- 1955-1965.
- Transistores. Reemplaza las válvulas
- Aparecen los primeros programas escritos a papel, luego se pasaban a tarjetas perforadas.
- Se crea el proceso por lotes.
- Primer Compilador (FORTRAN)

### Fortran Monitor System



### Generación 3

- 1965-1980.
- Circuitos integrados, ventaja en precio y desempeño.
- Multiprogramación.
- Discos duros.
- Se introduce el “tiempo compartido” para permitir que varios programadores trabajen simultáneamente.
- IBM 360.

### Generación 4

- Desde 1980.
- Usan LSI (Large Scale Integration).
- GUI.
- Arquitectura 8080.
- Basadas en RISC.

### Generación 5

- Computación Cuántica
- Basada en principios como el de superposición
- Aparecen los CUBIT, con múltiples valores.
- Requieren sistemas de refrigeración más elaborados, debido a que generan mucho calor.

## **Codificación de la Información**

### Código BCD

El código BCD permite representar números decimales en bloques binarios de 4 bits.

## Empaquetado y Desempaquetado

Los códigos BCD tienen dos aplicaciones en el campo de la computación

- **DESEMPAQUETADO:** Los números se codifican utilizando un Byte por dígito (zona y dígito) y es utilizado para transmisión desde hacia periféricos del computador
- **EMPAQUETADO:** Para el tratamiento de cálculos numéricos la ALU no necesita la ZONA, por lo que la elimina, reduciendo la codificación a solo 4 Bits por dígito rellenando con unos los de orden superior. Se los utiliza para cálculos en el entorno de la ALU. El signo se lo incluye por lo general en la parte sobrante del octeto (cabecera o cola del número). Se agrega un cero adelante para mantener la paridad de la cadena en caso de ser necesaria.

### BCD (Empaquetado y Desempaquetado)

La misma UAL que opera en binario puro puede operar en BCD. Lo que cambia es la forma en que se producen las distintas etapas para llevar a cabo la operación. Estas etapas están definidas por instrucciones en el programa, que permiten la gestión de la operación, considerando como base del sistema la decimal y no la binaria.

|       |                                |             |
|-------|--------------------------------|-------------|
| +3456 | desempaquetado: 2B 33 34 35 36 |             |
|       | empaquetado: 03456B            | Signo + (B) |
| +279  | desempaquetado: 2B 32 37 39    |             |
|       | empaquetado: 279B              |             |
| -3456 | desempaquetado: 2D 33 34 35 36 |             |
|       | empaquetado: 03456D            | Signo - (D) |
| -279  | desempaquetado: 2D F2 F7 F9    |             |
|       | empaquetado: 279D              |             |

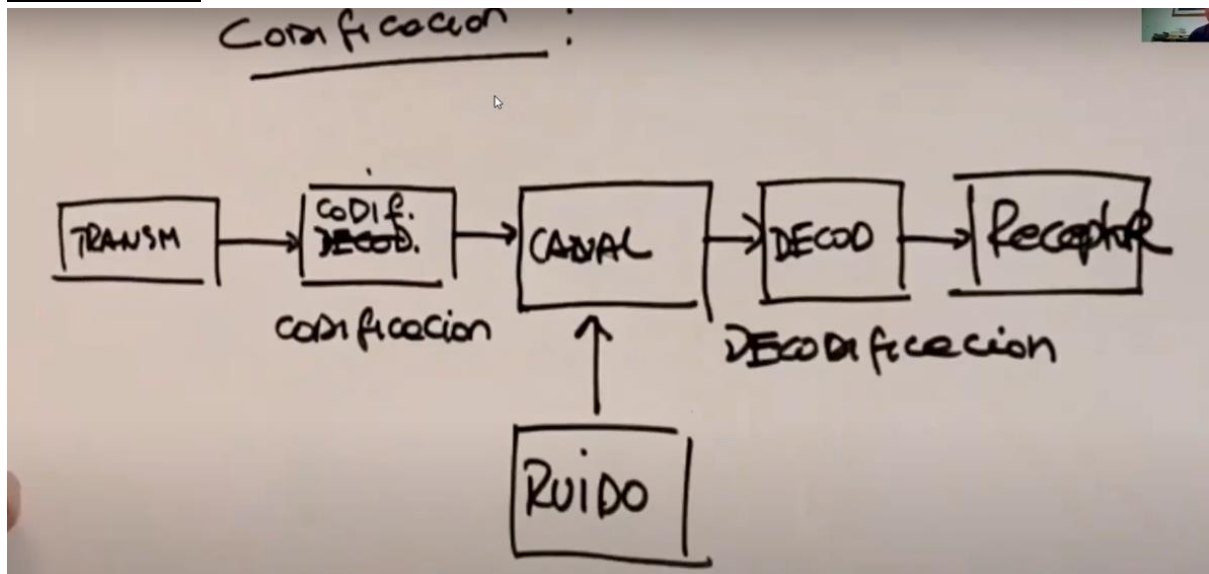
## Códigos Redundantes

Paridad simple a nivel de carácter.

Longitud de cadenas de caracteres fijas: Cada dato ocupa un número determinado de bytes.

Longitud de cadenas de caracteres variables: Cada dato tiene en su inicio un campo que indica la longitud del string (en bytes).

## Codificación



Un proceso de comunicación puede tener lugar en diversas formas, e involucra el movimiento de un flujo de información a través de un CANAL del sistema.

El RUIDO son todos los factores que afectan a la transmisión de datos, generando que se pierda información.

Es por este RUIDO que se agregan el codificador y el decodificador, con el fin de minimizar lo más posible estas interferencias.

### Código 2 en 3:

- Si las 3 transmisiones son iguales, la información es correcta.
- Si 2/3 son iguales, se toma una de las 2 como correctas.
- Si las 3 son distintas, se pide retransmisión.

Es código redundante, ya que se manda 3 veces el mismo mensaje (como mínimo).

Por ejemplo, puede ser útil cuando se tiene que transmitir información de los buses a la UMC, la cual trabaja en el orden de los nanosegundos, donde los errores no son muy repetitivos, y la velocidad no es un problema, a priori.

### Paridad:

|       |   |        |
|-------|---|--------|
| PAR   | 0 | 110110 |
|       | 1 | 110110 |
|       |   |        |
| IMPAR | 0 | 110111 |
|       | 1 | 010110 |

Controla la cantidad de bits y ve si cumplen la condición de paridad seleccionada. Pero bien puede presentar errores múltiples que den por válido un mensaje incorrecto.

Se colocan bits redundantes

\*Paridad Par de 1 → Representada con una compuerta XOR

#### CÓDIGO ENTRELAZADO:

##### Paridad horizontal y vertical a nivel de bloque o CÓDIGO ENTRELAZADO

- Los caracteres a transmitir se agrupan en bloques de m filas y n columnas
- Se calcula el bit de paridad de cada fila y se añade al principio (o al final, según convenio) de la fila
- Se calcula el bit de paridad de cada columna y se añade al principio (o al final, según convenio) de la columna
- El bloque final a transmitir tendrá por tanto una fila y una columna más que el original.
- La nueva columna estará formada por los bits de paridad horizontal de todas las filas
- La nueva fila estará formada por los bits de paridad vertical de todas las columnas.
- Adicionalmente se emplea un bit de paridad cruzada que se calcula a partir de los bits de paridad de filas y columnas.
- Los chequeos de paridad horizontal y vertical se usan para detectar y corregir los posibles errores que se puedan producir durante la transmisión de datos.

Sirve para detectar y corregir el error. Puede detectar si hay más de un error, pero solo sirve para corregir uno.

#### Código Hamming

Permite detectar y corregir errores. Es útil cuando la retransmisión de datos representa un alto costo. Ej burdo: Una película de Netflix.

Distancia entre 2 combinaciones binarias: cantidad de dígitos que hay que cambiar de una combinación a otra.

## Distancia de Hamming

El código Hamming se genera a partir de una distancia mínima de 3, ya que una distancia de 1 o 2 es inconveniente para poder generar un código para detectar un error.

La distancia mínima de Hamming está dada por la siguiente ecuación:

$$D_m = 2X + 1$$

Donde  $D_m$  es la distancia mínima de un código para permitir la corrección de datos y  $X$  las líneas de datos.

Si se tuviese una distancia de 1 no tendríamos valores para saber si hay o no un error.

Si se tienen una distancia de 2 se podría lograr un sistema de corrección de error con un bit de paridad pero el problema que presenta es que un sistema con distancia 2 puede tener datos erróneos que pueden pasar como datos correctos.

Con la distancia de 3 se tiene que los códigos no van a tener datos o resultados similares por ende se puede detectar el error y se puede desarrollar un sistema para su corrección como el Hamming

Se puede calcular la distancia Hamming entre 2 códigos como la suma SIN ACARREO entre los números.

## Distancia de Hamming

| Distancia | Longitud Total (n) | Longitud Información (k) | Notación (n,k) | Eficiencia sobre Canal | Detección | Corrección $\text{int}((m-1)/2)$ |
|-----------|--------------------|--------------------------|----------------|------------------------|-----------|----------------------------------|
| m         | $(2^m)-1$          | $(2^m)-1-m$              | (n,k)          | k/n                    | (m-1)     | (m-1)/2                          |
| 1         | 1                  | 0                        | (1,0)          | 0,0%                   | 0         | 0                                |
| 2         | 3                  | 1                        | (3,1)          | 33,3%                  | 1         | 0                                |
| 3         | 7                  | 4                        | (7,4)          | 57,1%                  | 2         | 1                                |
| 4         | 15                 | 11                       | (15,11)        | 73,3%                  | 3         | 1                                |
| 5         | 31                 | 26                       | (31,26)        | 83,9%                  | 4         | 2                                |
| 6         | 63                 | 57                       | (63,57)        | 90,5%                  | 5         | 2                                |
| 7         | 127                | 120                      | (127,120)      | 94,5%                  | 6         | 3                                |
| 8         | 255                | 247                      | (255,247)      | 96,9%                  | 7         | 3                                |
| 9         | 511                | 502                      | (511,502)      | 98,2%                  | 8         | 4                                |
| 10        | 1023               | 1013                     | (1023,1013)    | 99,0%                  | 9         | 4                                |
| 11        | 2047               | 2036                     | (2047,2036)    | 99,5%                  | 10        | 5                                |
| 12        | 4095               | 4083                     | (4095,4083)    | 99,7%                  | 11        | 5                                |

Con distancia 1 no se detecta ni se corrigen errores

Con distancia 2 se detecta un error sin posibilidad de corrección (paridad simple)

La eficiencia de este código comienza a partir de distancias igual a 3 o superior

# Código de Hamming

Este código permite detectar y corregir un solo errores producido en una transmisión con sólo agregar  $p$  bits de paridad, de forma que se cumpla la relación siguiente:  $2^p \geq i + p + 1$  donde:

- $i$  = Cantidad de bits de Información a Transmitir
- $P$  = Cantidad de bits de paridad

Para satisfacer la relación enunciada antes:

$i = 4$  para que la relación  $2^p \geq i + p + 1$  se cumpla debe ser  
 $p = ?$   $p = 3$ , luego  $2^3 \geq 4 + 3 + 1$ .

La tabla siguiente es la guía para determinar la distribución y el cálculo de los bits de paridad y su chequeo luego de la transmisión de hasta 11 dígitos binarios.

## Control y corrección

Cuando el mensaje es enviado, el receptor calcula el valor de los bits de control de la siguiente forma:

$$\begin{aligned}c_0 &= 1 \oplus 3 \oplus 5 \oplus 7 \\c_1 &= 2 \oplus 3 \oplus 6 \oplus 7 \\c_2 &= 4 \oplus 5 \oplus 6 \oplus 7\end{aligned}$$

Si el bit de control es igual a 0 es correcto, si es 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados en forma decreciente, es decir  $c_2$ ,  $c_1$  y  $c_0$ . Si todos los bits de control son igual a cero no se detecta error en el mensaje.

Se hace con XOR's porque permiten detectar la paridad.

## Circuitos Combinacionales

Son circuitos que poseen entradas y cuyas salidas dependen de dichas entradas en el mismo instante de tiempo.

Son predecibles, entregando la misma salida ante una misma entrada, debido a que no tienen en cuenta la variable tiempo.

Están basados en el Álgebra de Boole.



Para los combinacionales se usa:

**Forma Normal Disyuntiva:** si hay pocos 1's.  $[AB + \sim AD]$ .

**Forma Normal Conjuntiva:** si hay pocos 0's.  $[(A+B+D) * (\sim A + \sim D + B)]$ .

## Circuitos Secuenciales

Estos circuitos, a diferencia de los combinacionales, sus salidas dependen del tiempo, no solo de sus entradas, y también de su estado actual.

Se utilizan autómatas de estados finitos, siendo estos un sistema de estados donde su salida se asocia a una función que depende de los estímulos externos y de sus estados.

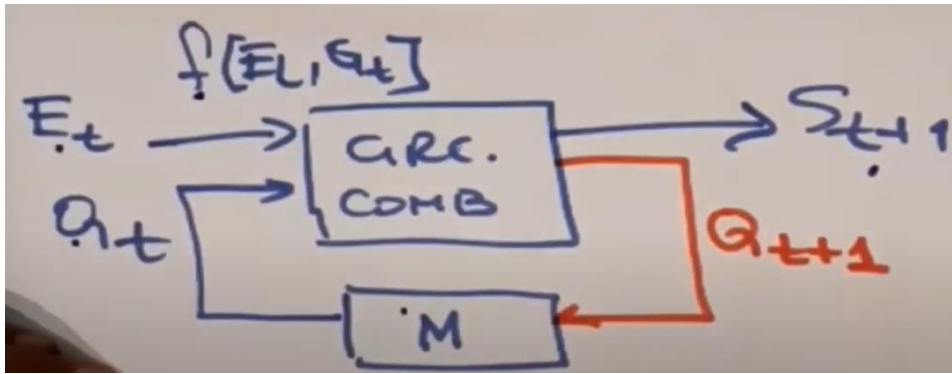
Existen distintas funciones:

### Funciones de transición

$S(t+1) = F(H(t), E(t))$  expresión genérica

$S(t+1) = F(Q(t), E(t))$

$Q(t+1) = G(Q(t), E(t))$



Este circuito secuencial se puede representar a través de un autómata de estados finitos, particularmente el modelo de Meeli, donde las salidas en un instante  $t+1$  dependen del estado y la entrada anterior.

## Biestables

Circuito lógico con dos estados.

Un biestable almacena solo 1 bit.

Es un circuito **re-alimentado** ya que la salida se inyecta en la entrada.

Como indica su nombre, va alternando sus estados.

### **Tabla de Excitación**

#### Tabla de Excitación

Para lograr determinar la función combinacional  $G$  del Circuito Secuencial, encargada de actualizar el estado del autómata, es necesario deducir los valores de entrada de los Flip Flops necesarios para producir cada transición de  $t$  a  $t+1$ , definido en el diagrama de transición.

Por tal motivo la Tabla de Transición es un resumen de la tabla de comportamiento donde, las entradas están planteadas como los estados en  $t$  y  $t+1$ , mientras que las salidas por las entradas de cada Flip Flop encargado de registrar dichos estados.

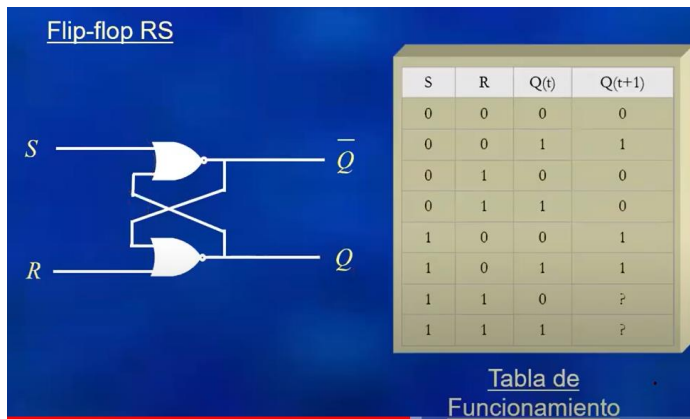
## Tipos de Biestables

RS

Consiste en un circuito de dos compuertas NAND con sus salidas retroalimentadas. La entrada **Set** almacena un uno mientras que **Reset** almacena un cero.

En ausencia de señal a la entrada, el flip-flop mantiene el valor almacenado, lo que lo convierte en un componente ideal para almacenar un valor lógico de manera temporal.

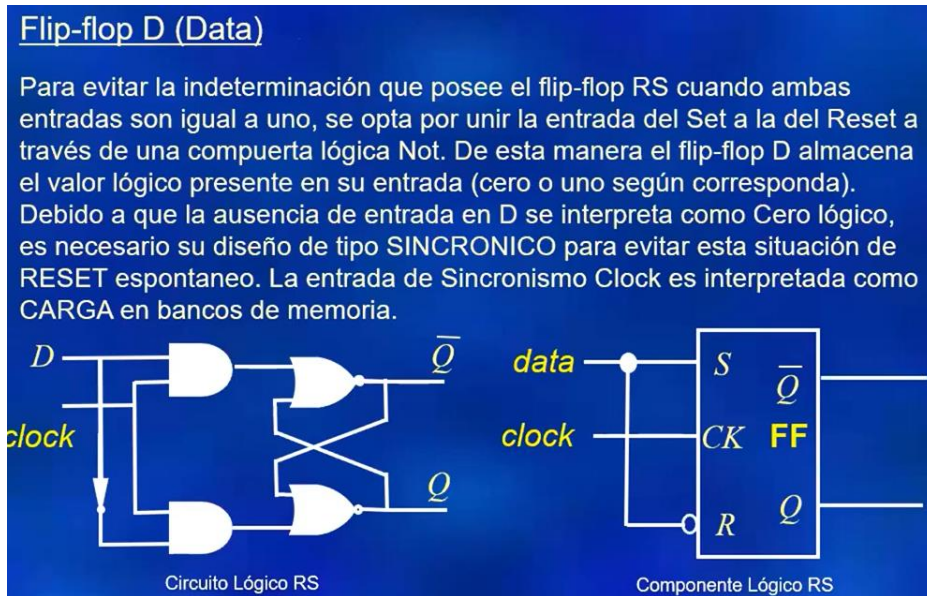
La activación de Set y Reset simultáneamente produce indeterminación, es decir no se puede predecir su comportamiento, pudiendo ponerlo en cero o uno de manera azarosa.



### Tabla de Excitación

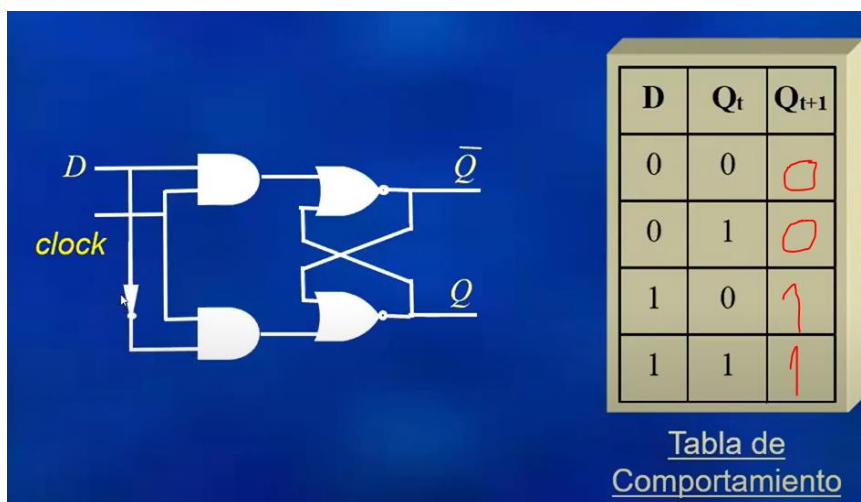
| $Q(t)$ | $Q(t+1)$ | S | R |
|--------|----------|---|---|
| 0      | 0        | 0 | X |
| 0      | 1        | 1 | 0 |
| 1      | 0        | 0 | 1 |
| 1      | 1        | X | 0 |

D



Este circuito requiere ser síncrono ya que en otro caso estaría reseteando el valor constantemente.

Es lo más parecido a un punto de memoria.



| $Q_t$ | $Q_{t+1}$ | D |
|-------|-----------|---|
| 0     | 0         | 0 |
| 0     | 1         | 1 |
| 1     | 0         | 0 |
| 1     | 1         | 1 |

Tabla de Excitación

JK

Es una modificación del flip-flop RS donde  $J=\text{Set}$  y  $K=\text{Reset}$ .  
 Agrega una nueva funcionalidad al circuito para el caso de indeterminación del RS. Si  $J=K=1$  procede a complementar el contenido lógico almacenado. La retroalimentación entre  $\bar{Q}$  con J y Q con K se activan solo en el caso en que  $J = K = 1$ , produciendo que las salidas actúen sobre las entradas complementando el valor almacenado.

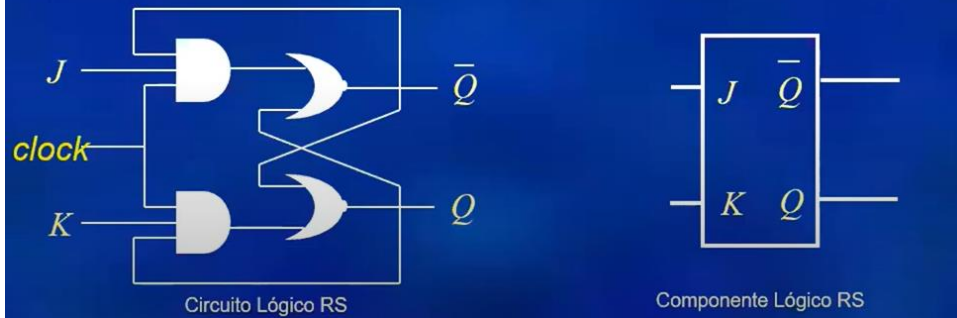


Tabla de Comportamiento

| J | K | $Q_t$ | $Q_{t+1}$ |
|---|---|-------|-----------|
| 0 | 0 | 0     | 0         |
| 0 | 0 | 1     | 1         |
| 0 | 1 | 0     | 0         |
| 0 | 1 | 1     | 0         |
| 1 | 0 | 0     | 1         |
| 1 | 0 | 1     | 1         |
| 1 | 1 | 0     | 1         |
| 1 | 1 | 1     | 0         |

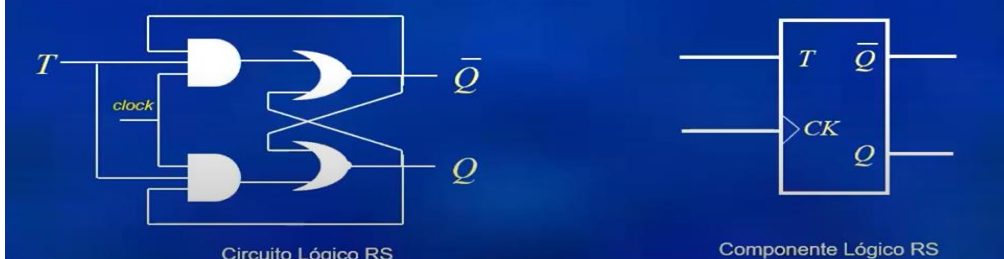
Tabla de Excitación

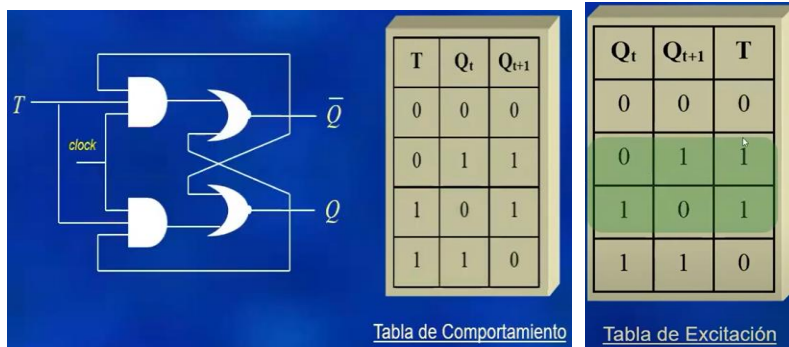
| $Q_t$ | $Q_{t+1}$ | J | K |
|-------|-----------|---|---|
| 0     | 0         | 0 | x |
| 0     | 1         | 1 | x |
| 1     | 0         | x | 1 |
| 1     | 1         | x | 0 |

T

### Flip-flop T (Toggle o Conmutación)

Es una modificación del flip-flop JK .  
 Este Biestable cambia de estado alternando entre Cero y Uno, siempre que la entrada  $T=1$  y este habilitada la señal de clock.  
 Cuando  $T=0$  mantiene el último estado almacenado.





## Buses y Registros

### Registros

Arreglo de FF organizados mediante una lógica combinacional/secuencial, de manera que permita realizar operaciones sobre información. (Ej: Lectura, grabación)

Un arreglo de Registros puede verse como una memoria.

### Tipos de Registros

#### TIPOS DE REGISTROS

- **Registros de datos:** Usados para guardar números enteros. Antiguamente existía un único registro donde se guardaba toda la información, llamado acumulador.
- **Registros de memoria:** Son usados para guardar exclusivamente direcciones de memoria. Muy usados en Arquitectura Harvard ya que muchas veces las direcciones tienen un tamaño de palabra distinto que la de datos.
- **Registros de propósito general** (en Inglés GPRs o General Purpose Registers): Pueden guardar tanto datos como direcciones. Son fundamentales en la arquitectura Von Neumann. La mayor parte de las computadoras modernas usa GPR.
- **Registros de coma flotante:** Usados para guardar datos en formato de coma flotante.
- **Registros de constantes:** Tienen valores creados por hardware de sólo lectura.
- **Registros de propósito específico:** Guardan información específica del estado del sistema, como el puntero de pila o el registro de estado.

Estructura básica:

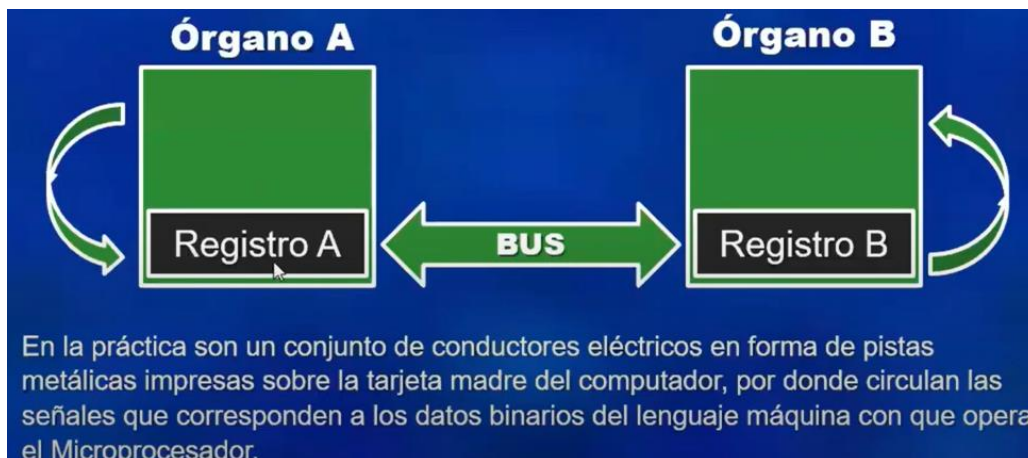


Estructura de Registro

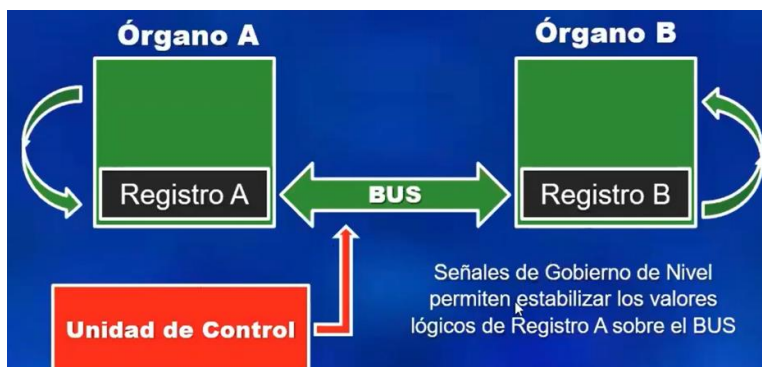
Lógica combinacional/secuencial

## Buses

Arreglo de líneas lógicas encargadas de interconectar los órganos del computador entre sí.

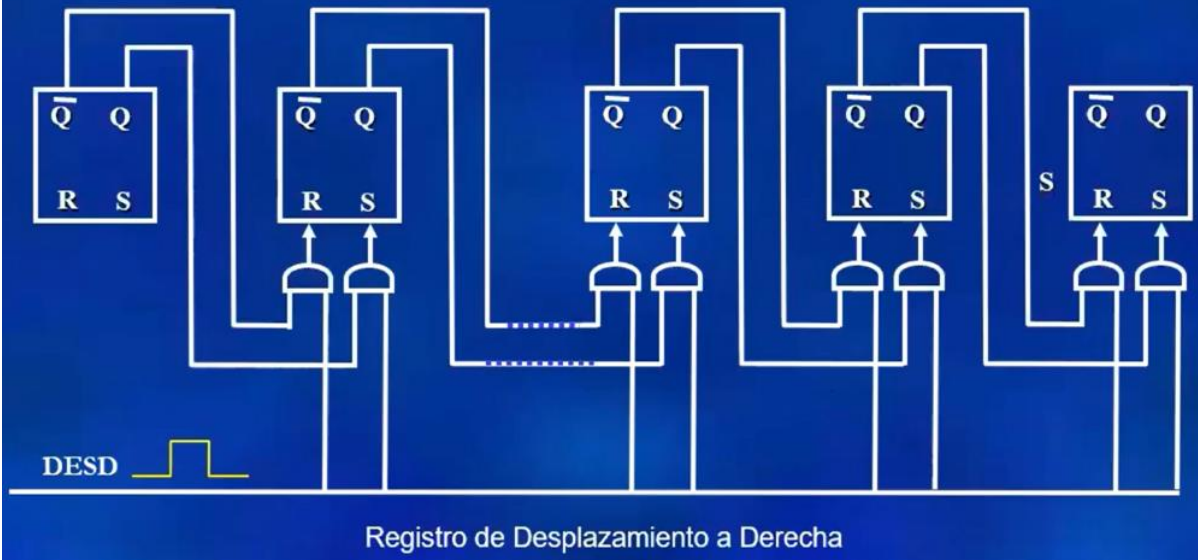


Por ejemplo, la comunicación de la ALU con la MC, a través del Bus de Datos (Bus M).



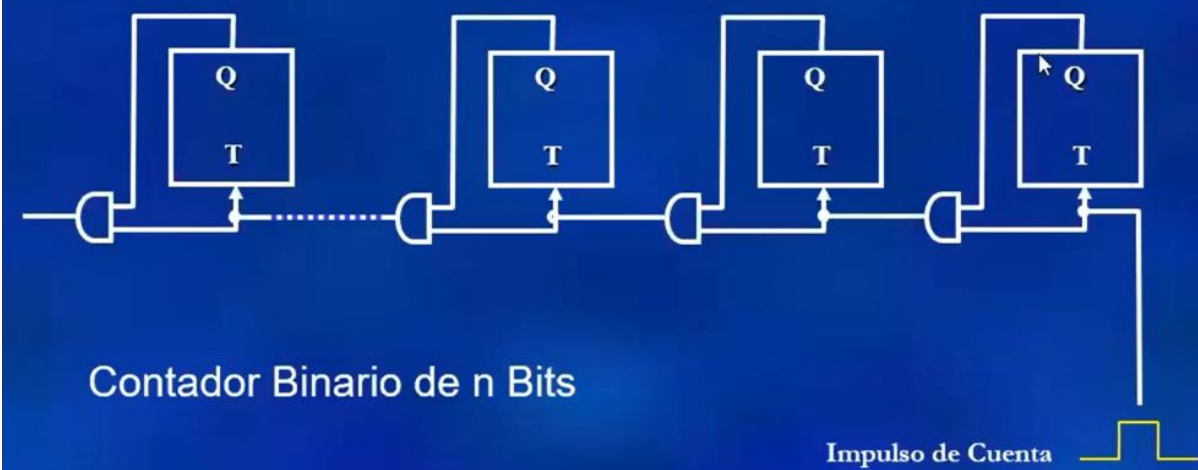
## Registro de Desplazamiento

**Registros e Desplazamiento:** Permiten desplazar la información a Derecha, Izquierda, o en ambos sentidos, según la información de control y la lógica combinacional que lo gobierne



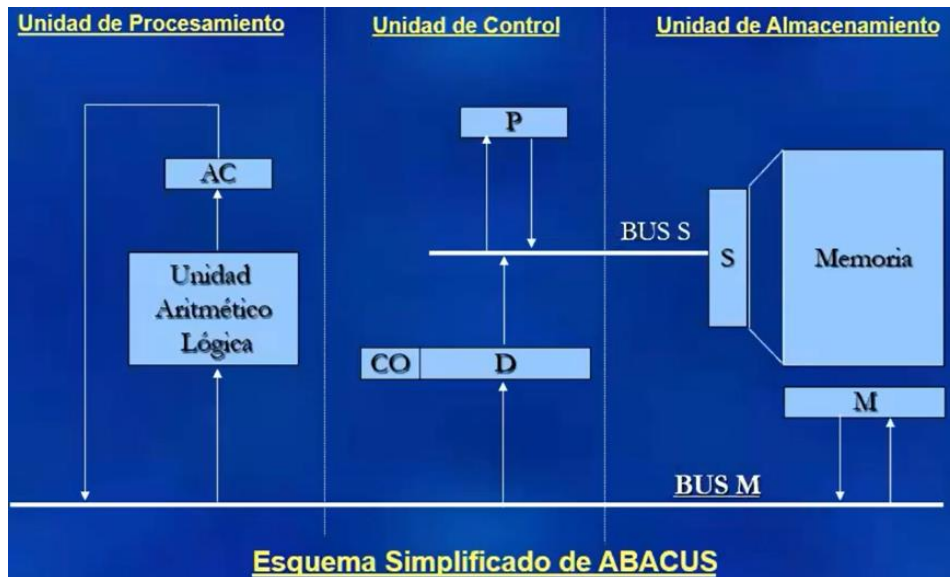
## Registro de Contador

**Registros Contadores:** Permiten incrementar su contenido binario en una unidad por cada señal de cuenta.

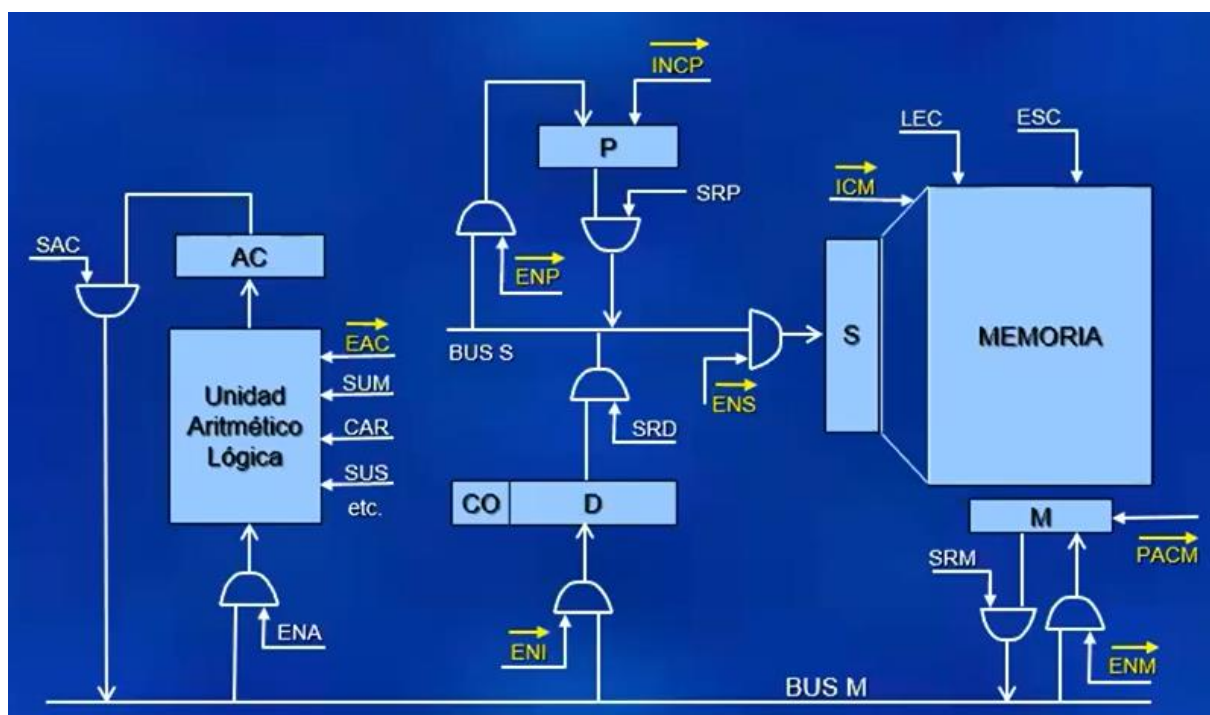


## ABACUS

Máquina de sincronismo, posee un registro de instrucción de una sola dirección (código de operación y dirección del operando). Posee un Bus M para instrucciones y operandos, otro Bus S de direcciones de operandos.



Abacus con varias instrucciones



## ALU

Encargada de ejecutar instrucciones de cálculo.

Posee Operadores o Unidades Funcionales.

Es un órgano compuesto por circuitos lógicos combinacionales y secuenciales dedicados a resolver operaciones de cálculo matemático y lógicas.



Todo operador aritmético debe tener como mínimo un conjunto de indicadores que informen del éxito o no de una operación, o de alguna anomalía.

### Indicadores del Operador Aritmético:

- Última operación Realizada ▶
- Error de Desbordamiento
- Error de desborde de capacidad
- Error de División por Cero
- Estado Acumulador Positivo
- Estado Acumulador Negativo
- Estado Acumulador Igual a cero

**Ultima operación:** Informa si se realizó o no la última operación, es necesario para que la arquitectura sepa cuando se encuentra disponible nuevamente el operador.

**Error de desborde:** Se excede en un bit. Se desplaza a derecha.

**Error de desborde de capacidad:** El desborde es de más de un bit, por lo que el ordenador deberá realizar algún tipo de acción.

Los últimos 3 sirven para tomar decisiones de saltos.

## SemiSumador

### EL SEMISUMADOR

| Entradas |   | Salidas |   |
|----------|---|---------|---|
| A        | B | C       | S |
| 0        | 0 | 0       | 0 |
| 0        | 1 | 0       | 1 |
| 1        | 0 | 0       | 1 |
| 1        | 1 | 1       | 0 |

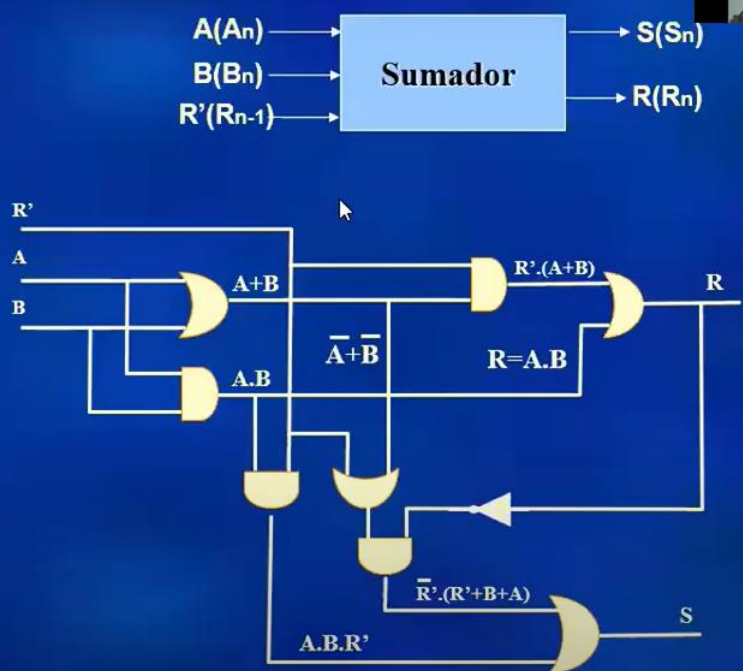


C = carry

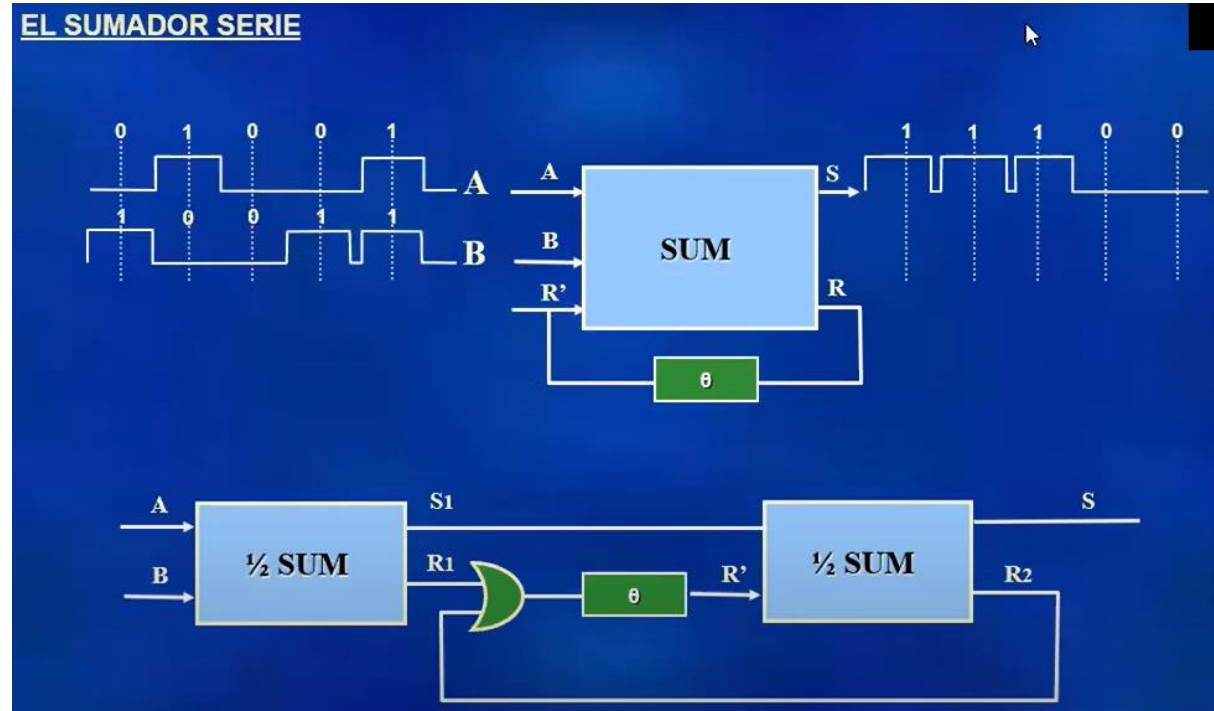
## Sumador

### EL SUMADOR

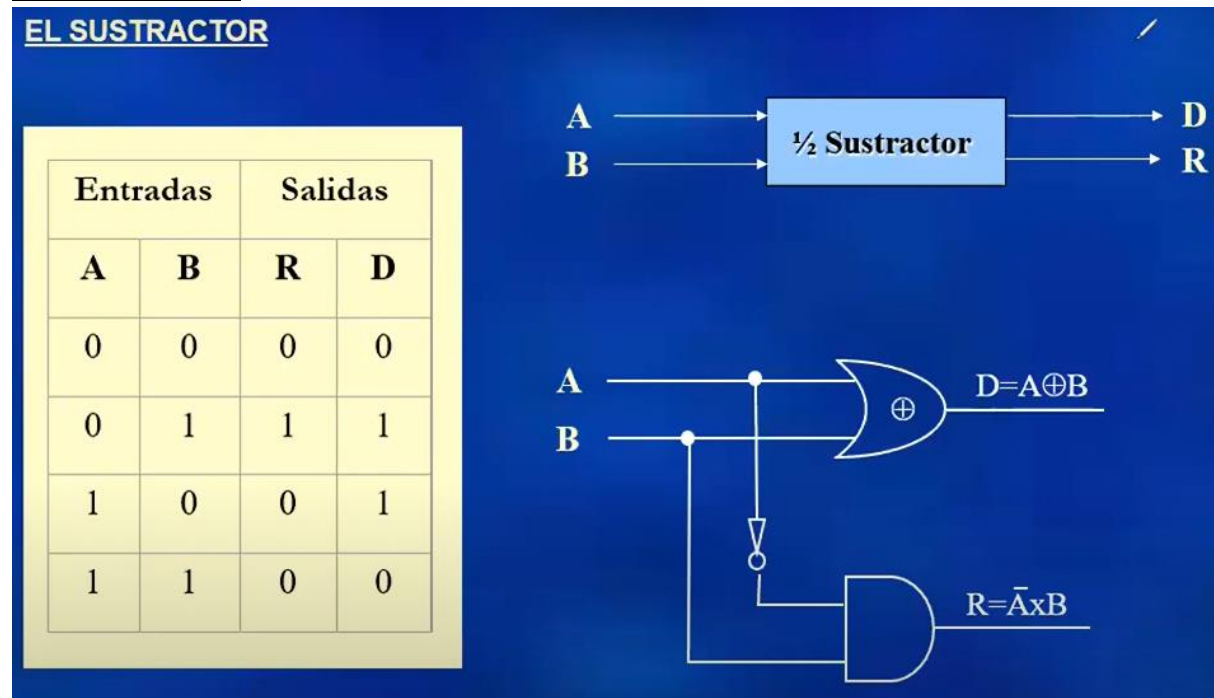
| Entradas |   |    | Salidas |   |
|----------|---|----|---------|---|
| A        | B | R' | R       | S |
| 0        | 0 | 0  | 0       | 0 |
| 0        | 1 | 0  | 0       | 1 |
| 1        | 0 | 0  | 0       | 1 |
| 1        | 1 | 0  | 1       | 0 |
| 0        | 0 | 1  | 0       | 1 |
| 0        | 1 | 1  | 1       | 0 |
| 1        | 0 | 1  | 1       | 0 |
| 1        | 1 | 1  | 1       | 1 |



## Sumador en Serie

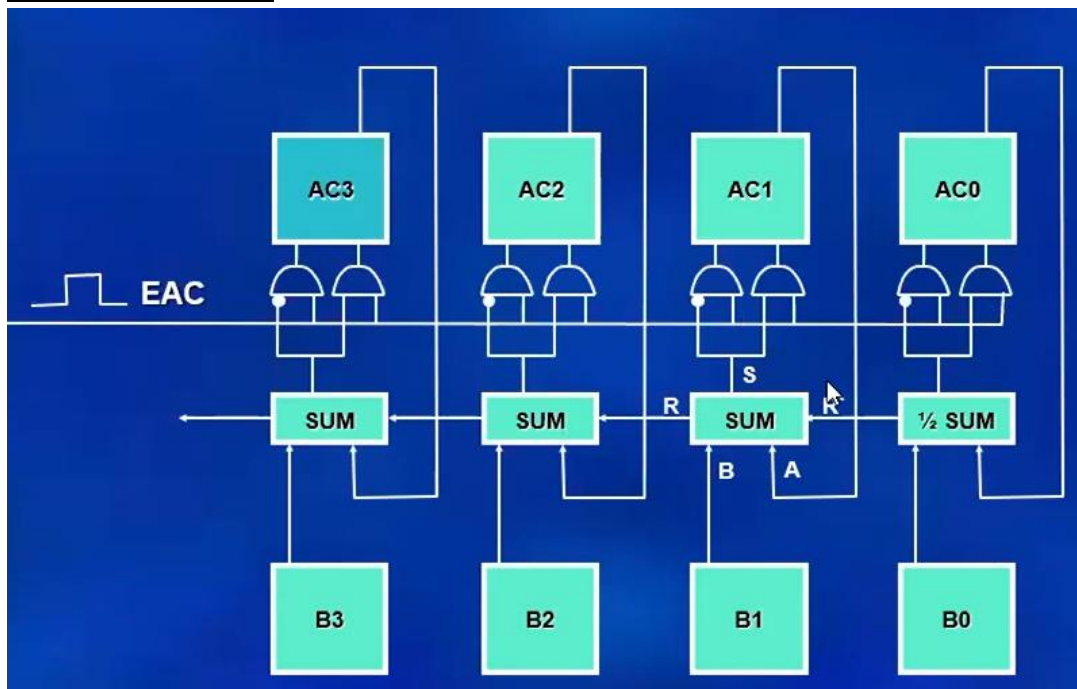


## SemiSustractor



\*Es netamente demostrativo, ya que las restas se realizan con complemento a la base\*

## Sumador Paralelo



Ventajas de este modelo en comparación con el sumador en serie:

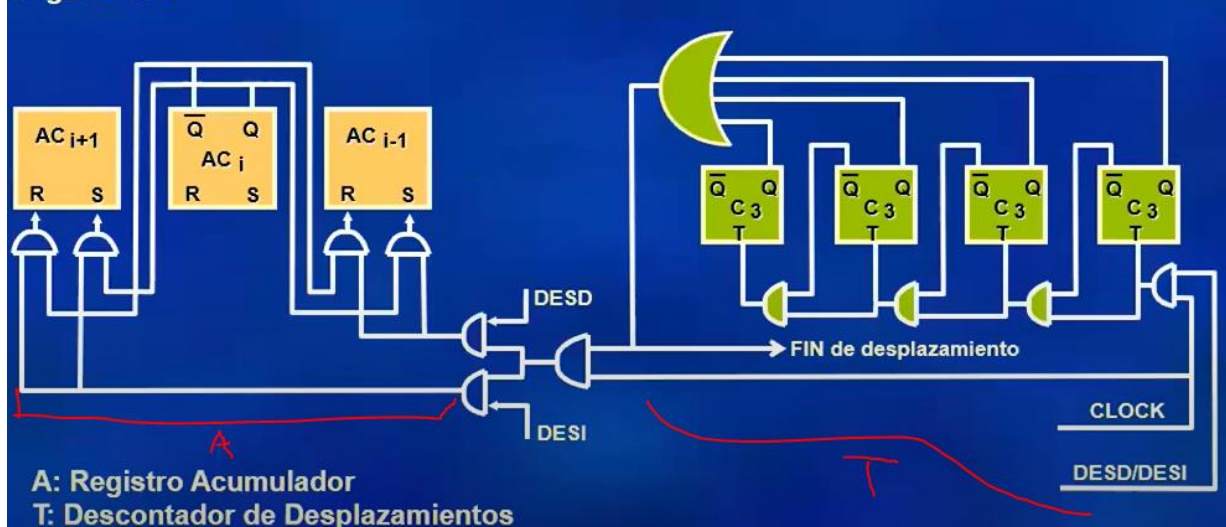
- Puede resolver una operación de suma en el lapso de un batido y otro de reloj.
- Es más veloz.
- Tiene una mejor performance.

Desventajas:

- Es más costoso por la cantidad de componentes.

## Gobierno Secuencial de Desplazamiento

**Acumulador montado como un registro de desplazamiento. El contador se ve restado por cada señal de desplazamiento del acumulador, y detiene su operación cuando este llega a cero.**





Estos son números enteros

1—Signo negativo

0—Signo Positivo

Se utiliza el complemento a la base para las operaciones ya que es más útil a la hora de tratar un desbordamiento.

\*\*En la practica se complementa a partir del 1er 1 que se encuentra.

EJEMPLO

[ UAL ] ABACU

Números Binarios con Signo

Ejemplos:

|   |     |          |   |     |          |
|---|-----|----------|---|-----|----------|
| + | +6  | 0 000110 | + | -6  | 1 111010 |
|   | +9  | 0 001001 |   | +9  | 0 001001 |
|   | +15 | 0 001111 |   | +3  | 0 000011 |
| + | +6  | 0 000110 | + | -9  | 1 110111 |
|   | -9  | 1 110111 |   | -9  | 1 110111 |
|   | -3  | 1 111101 |   | -18 | 1 101110 |

Los dos números de los cuatro ejemplos se suman, con sus bits de signo incluidos. Cualquier arrastre del bit de signo se descarta y los resultados negativos se producen automáticamente en la forma de complemento de 2

Ing. Gustavo Maurokefalidis 28 Arquitectura de Computadores

En el 2d0 (-6+9) se complementa a 6 y se hace la suma

En todos los casos se suma, ya que, si los signos son diferentes, se complementan los negativos y luego se suma normalmente.

[ UAL ]
ABACU


### Números Binarios con Signo

**SOBRECAPACIDAD o DESBORDAMIENTO:**

Debido a la longitud fija del tamaño de los registros del computador, estos deben verificar situaciones de sobre capacidad e indicar esta situación en un biestable diseñado a tal efecto.

No puede haber Sobrecapacidad o Desbordamiento después de una suma si: Un número Positivo y otro Negativo

Puede haber Sobrecapacidad o Desbordamiento después de una suma si: Ambos números son positivos o negativos.

La condición de Sobrecapacidad o Desbordamiento se detecta observando el arrastre producido por el bit mas significativo de la Magnitud y el del Bit del Signo. Si son distintos se produce una condición de Sobrecapacidad debiendo indicarlo en Bit Desbordamiento.

Ing. Gustavo Maurokefalidis
29
Arquitectura de Computadores

[ UAL ]
ABACU


### Sumador/Sustractor de Numeros Algebraicos Binarios en Punto Fijo

**Para poder desarrollar un circuito sumador/ sustractor es necesario:**

- Expresar las magnitudes positivas en valor absoluto e indicar en el bit de mayor peso su signo a través de un Cero
- Expresar las Magnitudes Negativas por complemento Absoluto, e indicar en el bit de mayor peso su signo a través de un Uno
- Verificar situaciones de desborde a través de analizar el arrastre del dígito de mayor peso de la magnitud junto al arrastre de la suma de los signos, detectando situaciones donde alguno de ellos sea igual a UNO

De esta manera es posible plantear un circuito que permita la suma o resta a través de sumadores, ajuste el signo de la operación, y detecte e indique en un biestable la condición de arrastre de la operación.

Ing. Gustavo Maurokefalidis
30
Arquitectura de Computadores

EJEMPLO

sum sus algebraico.xlsx - Excel

Gustavo Maurokefalidis

Archivo Inicio Insertar Disposición de página Fórmulas Datos Revisar Vista Ayuda ¿Qué desea hacer?

Calibri 11 Fuente Alineación Número Estilos Formato Dar formato Estilos de Insertar Eliminar Formato Celdas Edición

E35 0

|    | Carry | Signo | A <sub>5</sub> | A <sub>4</sub> | A <sub>3</sub> | A <sub>2</sub> | A <sub>1</sub> | A <sub>0</sub> |
|----|-------|-------|----------------|----------------|----------------|----------------|----------------|----------------|
| 28 |       |       |                |                |                |                |                |                |
| 29 |       |       |                |                |                |                |                |                |
| 30 |       |       |                |                |                |                |                |                |
| 31 |       |       | 0              | 1              | 1              | 1              | 1              | 1              |
| 32 |       |       |                |                |                |                |                |                |
| 33 |       |       | 0              | 1              | 1              | 1              | 1              | 1              |
| 34 |       |       |                |                |                |                |                |                |
| 35 | 1     |       | 0              | 1              | 1              | 1              | 1              | 0              |
| 36 |       |       |                |                |                |                |                |                |
| 37 |       | XOR   |                |                |                |                |                |                |
| 38 |       |       |                |                |                |                |                |                |
| 39 |       |       |                |                |                |                |                |                |

Hoja1 Hoja2 Hoja3

(En donde esta seleccionada iba un 1, pero como se produce acarreo, se cambia directamente de numero)

Arquitectura de Computadoras - Unidad Aritmética Lógica de ABACUS Unidad de Punto Flotan...

Gustavo Maurokefalidis

Archivo Inicio Insertar Disposición de página Fórmulas Datos Revisar Vista Ayuda ¿Qué desea hacer?

Calibri 11 Fuente Alineación Número Estilos Formato Dar formato Estilos de Insertar Eliminar Formato Celdas Edición

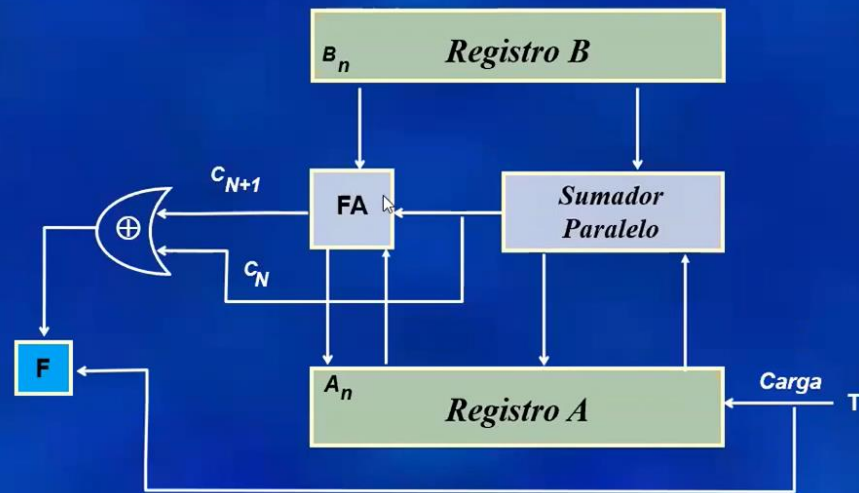
E47 1

|    | Carry | Signo | A <sub>5</sub> | A <sub>4</sub> | A <sub>3</sub> | A <sub>2</sub> | A <sub>1</sub> | A <sub>0</sub> |
|----|-------|-------|----------------|----------------|----------------|----------------|----------------|----------------|
| 40 |       |       |                |                |                |                |                |                |
| 41 |       |       |                |                |                |                |                |                |
| 42 |       |       |                |                |                |                |                |                |
| 43 |       |       | 1              | 1              | 0              | 0              | 0              | 1              |
| 44 |       |       |                |                |                |                |                |                |
| 45 |       |       | 1              | 0              | 0              | 0              | 0              | 1              |
| 46 |       |       |                |                |                |                |                |                |
| 47 | 1     |       | 1              | 0              | 0              | 0              | 1              | 0              |
| 48 |       |       |                |                |                |                |                |                |
| 49 |       | XOR   |                |                |                |                |                |                |
| 50 |       |       |                |                |                |                |                |                |
| 51 |       |       |                |                |                |                |                |                |

Hoja1 Hoja2 Hoja3

Pasa lo mismo que en el caso anterior, en A5  $1+1 = 10 \rightarrow$  Se invierte el valor del signo, quedando en 1

REPRESENTACION CON FF's

Adición y Sustracción de Números Algebraicos Binarios

(Morris Mano Figura 8-10 Pág. 340)

El **REGISTRO A** almacena un sumando con su bit de signo en  $A_n$

El **REGISTRO B** almacena un sumando con su bit de signo en  $B_n$

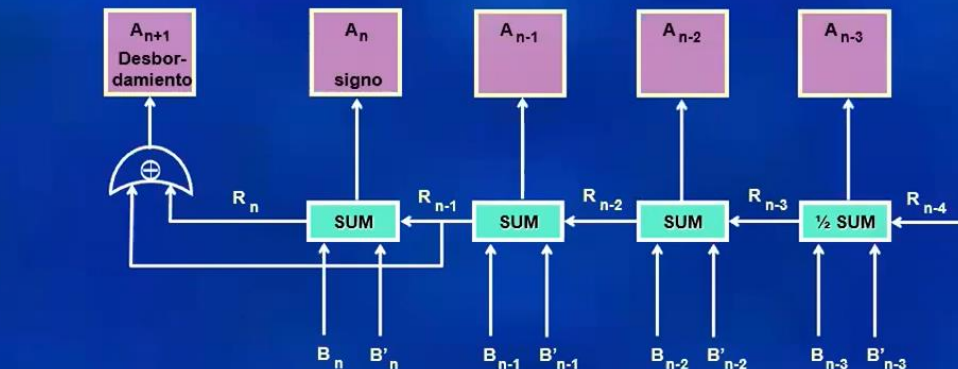
Los dos números se suman a través de un **SUMADOR PARALELO** de  $n$  bits

Un sumador completo **FA** (SUM) suma todos los bits de mayor peso correspondiente al signo

Un **XOR** analiza desbordamiento y lo indica con un 1 en el FF **F**

En MEINADIER:

Se plantea un sumador de una dirección.

**Adición y Sustracción de Números Algebraicos Binarios****Condiciones de Signo:**

2 operandos > 0  
el resultado debe ser positivo

1 operando > 0  
1 operando < 0  
No puede haber desbordamiento

2 operandos < 0  
El resultado debe ser negativo

(J. Pierre Meinadier Figura 9 Pág. 113)

Ing. Gustavo Maurokefalidis

32

Arquitectura de Computadores

**COMA FLOTANTE**

Para empezar, recordemos que los números en coma flotante se representan en la forma  $S.M.a^E$ , donde S es el signo del número, M la mantisa y E el exponente y a generalmente vale 2 o 16 en aritmética binaria. Las operaciones se llevan a cabo sobre números normalizados, esto es, sobre números cuyo valor de exponente está ajustado para que la mantisa tenga el mayor número posible de dígitos significativos. Por consiguiente, los operadores en aritmética flotante deben, no solamente ejecutar la operación correspondiente, sino también normalizar el resultado obtenido.



Se considera a un número en coma flotante como normalizado, cuando el valor binario de su signo y el del bit más significativo de su mantisa son distintos

Se busco para poder realizar operaciones con números con coma.

**Dato Color:** Hasta los 90 las placas madre venían con un sócalo para incluir el punto flotante para darle mayor potencia. No lo incluían de fábrica. Los computadores debían incluir un 2do procesador para realizar estos cálculos ya que eran de gran importancia.

Nos encontramos con un exponente (E): Se utiliza notación científica ya que si se incluyese una coma (carácter) implicaría mucho más espacio y se trataría de un alfanumérico.



En primer lugar, tomemos conciencia de las dificultades planteadas por estas delicadas operaciones sobre unos ejemplos significativos, descritos, por simplificar, en aritmética decimal.

La primera fase consiste en *alinear las mantisas* con objeto de enfrentar los bits de igual peso. Supongamos a sumar los números  $A = 1200 \times 10^{-0}$  y  $B = 80000 \times 10^{-2}$ , que se escriben, respectivamente:

$$\begin{array}{lcl}
 \text{A} & \boxed{+} \boxed{0} \boxed{0} \boxed{1} \boxed{2} \boxed{0} \boxed{0} & = 1200 \times 10^0 \\
 \text{B} & \boxed{+} \boxed{2} \boxed{8} \boxed{0} \boxed{0} \boxed{0} \boxed{0} & = 80000 \times 10^2
 \end{array}$$

Estos números no están alineados para operarse ya que tienen exponentes de orden distintos

No puede operarse directamente la suma de las mantisas; se comparan los exponentes y se detecta que el primero de los números es más pequeño; sobre este número se ejecuta una doble operación: **incremento del exponente en una unidad y desplazamiento de la mantisa una posición a la derecha**. De nuevo se comparan y se recommienza hasta obtener identidad de exponentes. En ese momento puede efectuarse la suma de las mantisas:

Se debe llevar el menor exponente al mayor, sino se pierde precisión en el número mayor.

## Ajuste de Signos

[ UAL ] Suma /Sustracción en Coma Flotante en ABACU

La segunda fase consiste en **ajustar los signos**. Consideremos la sustracción  $A - B$ . En la representación en valor absoluto y signo, se deberán primeramente comparar las mantisas de A y B, comprobar que la de B es superior a la de A y deducir de ello que la operación pertinente es sustraer la mantisa de A de la mantisa de B y al resultado darle el signo menos:

|    |   |   |   |   |   |   |   |
|----|---|---|---|---|---|---|---|
| A  | + | 2 | 0 | 0 | 0 | 1 | 2 |
| -B | + | 2 | 8 | 0 | 0 | 0 | 0 |

|    |   |   |   |   |   |
|----|---|---|---|---|---|
| MB | 8 | 0 | 0 | 0 | 0 |
| MA | 0 | 0 | 0 | 1 | 2 |

---

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| - | 2 | 7 | 9 | 9 | 8 | 8 |
|---|---|---|---|---|---|---|

Debe tomarse así en cuenta en esta fase la clase de operación (suma o sustracción), el signo de los operandos y el resultado de la comparación de mantisas. Nótese que el problema se simplifica si el conjunto signo-mantisa fuera representado en complementación cuando el número sea negativo.

Ing. Gustavo Maurokefalidis 36 Arquitectura de Computadores

Si la mantisa del negativo es mayor, se invierte la operación y luego se expresa con signo negativo

Al realizarse la suma y resta se pueden dar 2 situaciones:

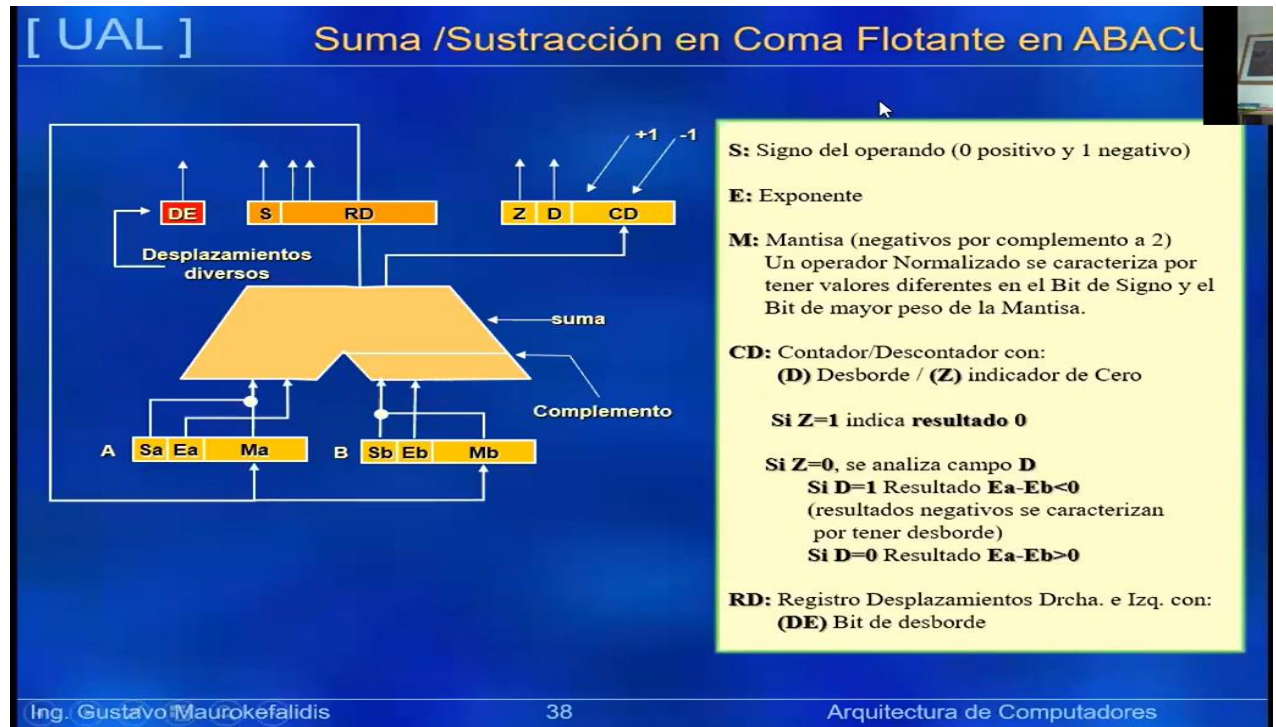
[ UAL ] Suma /Sustracción en Coma Flotante en ABACU

La tercera fase es la **normalización del resultado**. Puede que el resultado bruto de la operación no esté normalizado, por dos razones: (1) **hay desbordamiento de capacidad en la suma**, en cuyo caso es necesario **desplazar a mantisa una posición a la derecha**, aumentando al tiempo en una unidad el exponente; (2) **la diferencia de la mantisas implica la formación de un número  $n$  de ceros a la izquierda del resultado**, en cuyo caso es necesario **desplazar la mantisa  $n$  posiciones a la izquierda del resultado** simultáneamente. Veamos un ejemplo de suma en ambos casos:

|               |   |   |   |   |   |   |   |
|---------------|---|---|---|---|---|---|---|
| 1º Operando   | + | 2 | 8 | 0 | 0 | 0 | 0 |
| 2º Operando   | + | 2 | 8 | 0 | 0 | 0 | 0 |
| Operación     |   | 1 | 6 | 0 | 0 | 0 | 0 |
| Normalización | + | 3 | 1 | 6 | 0 | 0 | 0 |

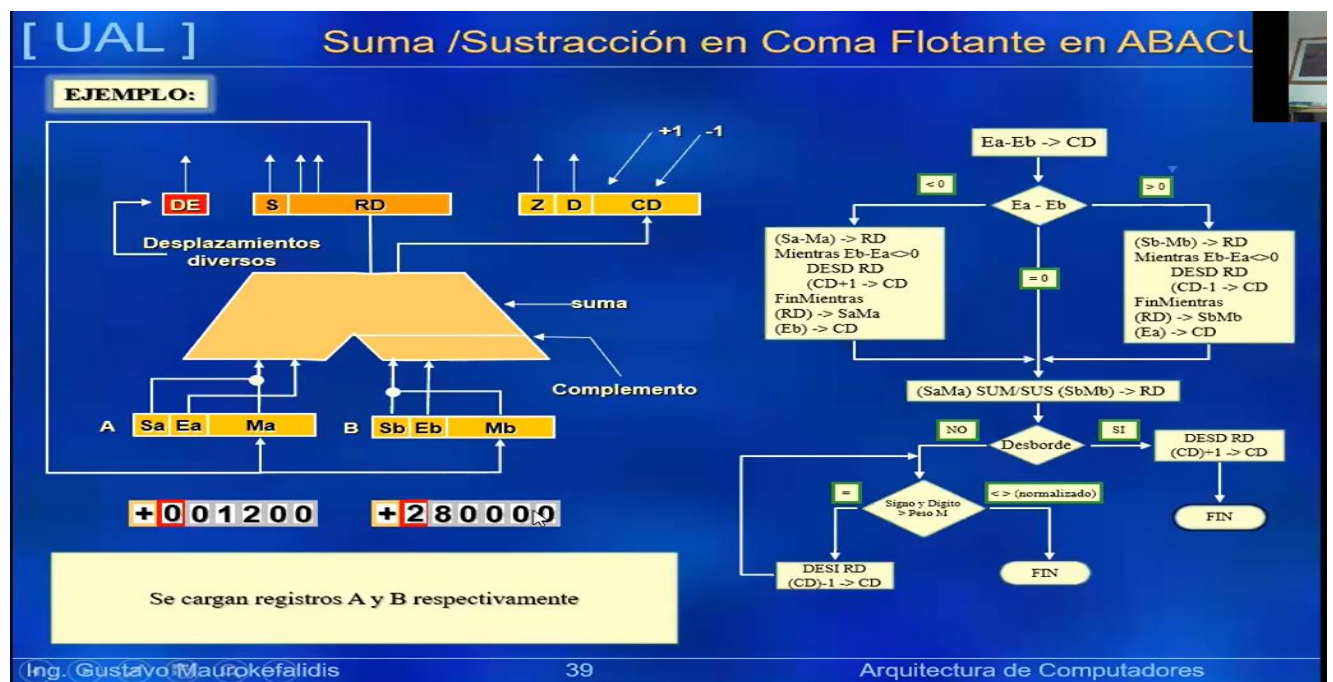
|               |   |    |   |   |   |   |   |
|---------------|---|----|---|---|---|---|---|
| 1º Operando   | + | 2  | 8 | 0 | 0 | 0 | 0 |
| 2º Operando   | + | 2  | 7 | 9 | 9 | 9 | 9 |
| Operación     |   |    | 0 | 0 | 0 | 0 | 1 |
| Normalización | + | -2 | 1 | 0 | 0 | 0 | 0 |

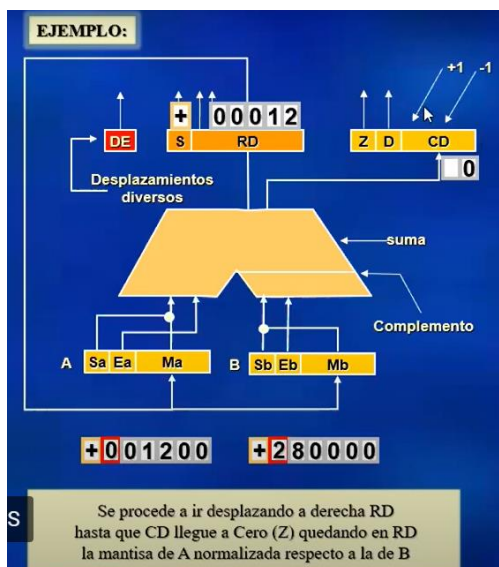
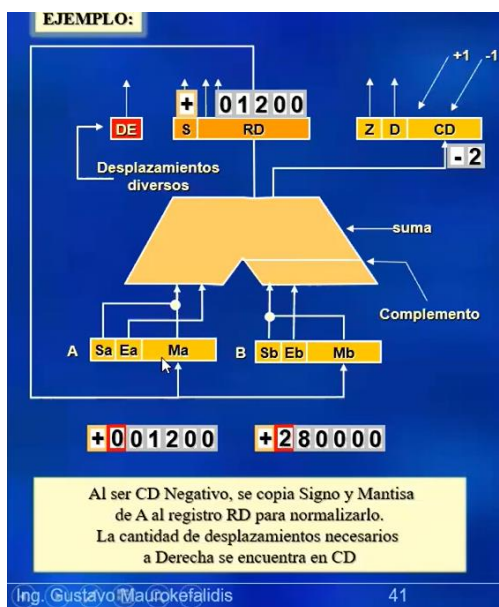
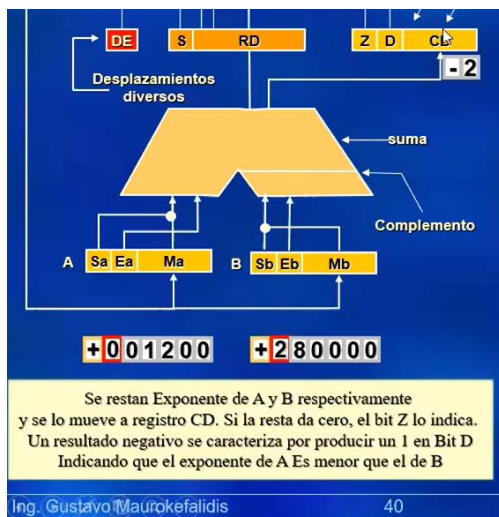
Ing. Gustavo Maurokefalidis 37 Arquitectura de Computadores

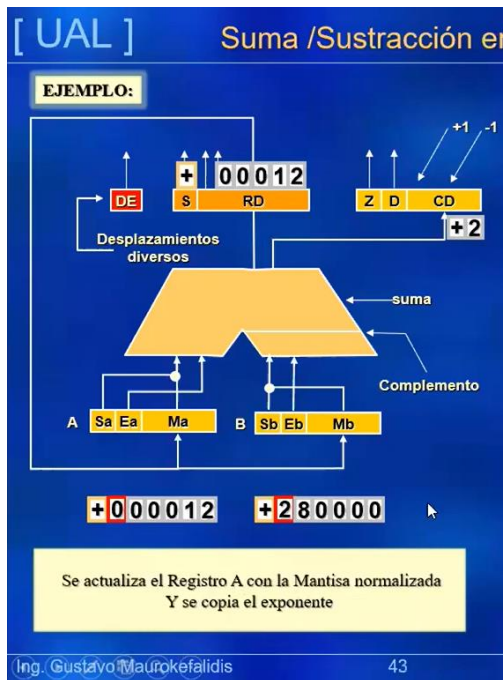


\*\*DE es DESBORDE

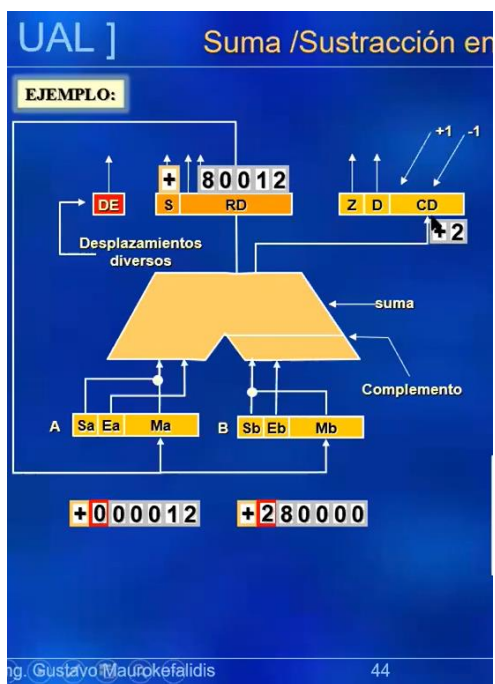
EJEMPLO

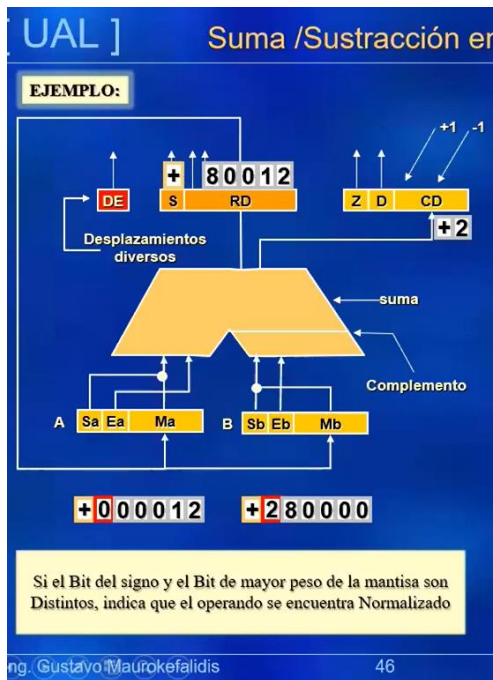






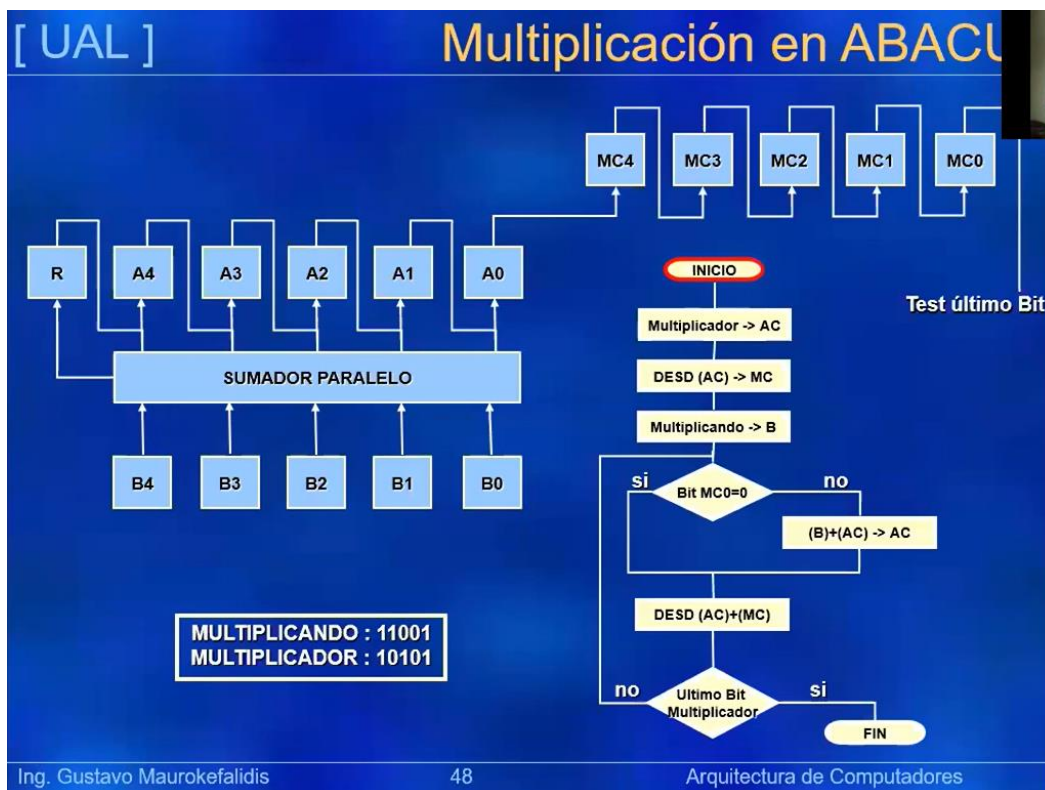
Ahora se realiza la suma (abajo)



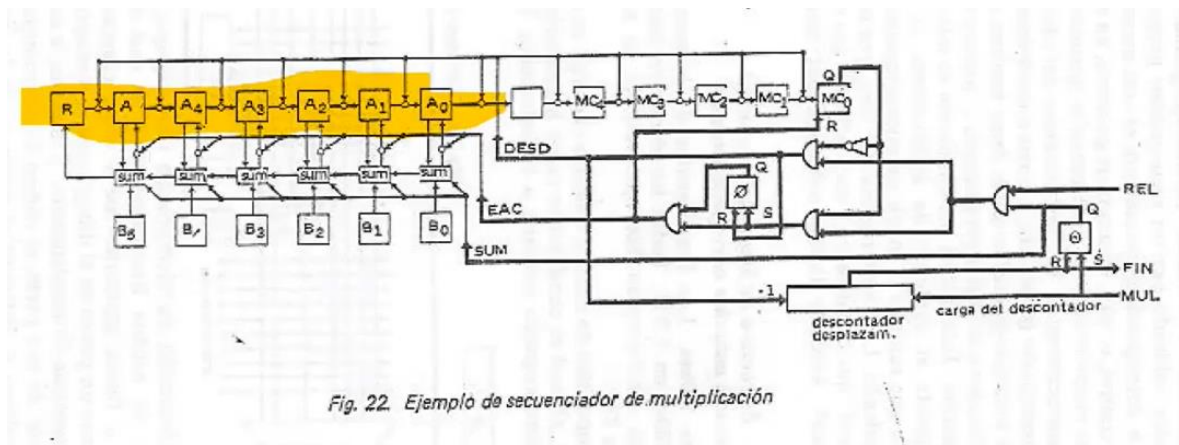


## MULTIPLICACION EN ABACUS

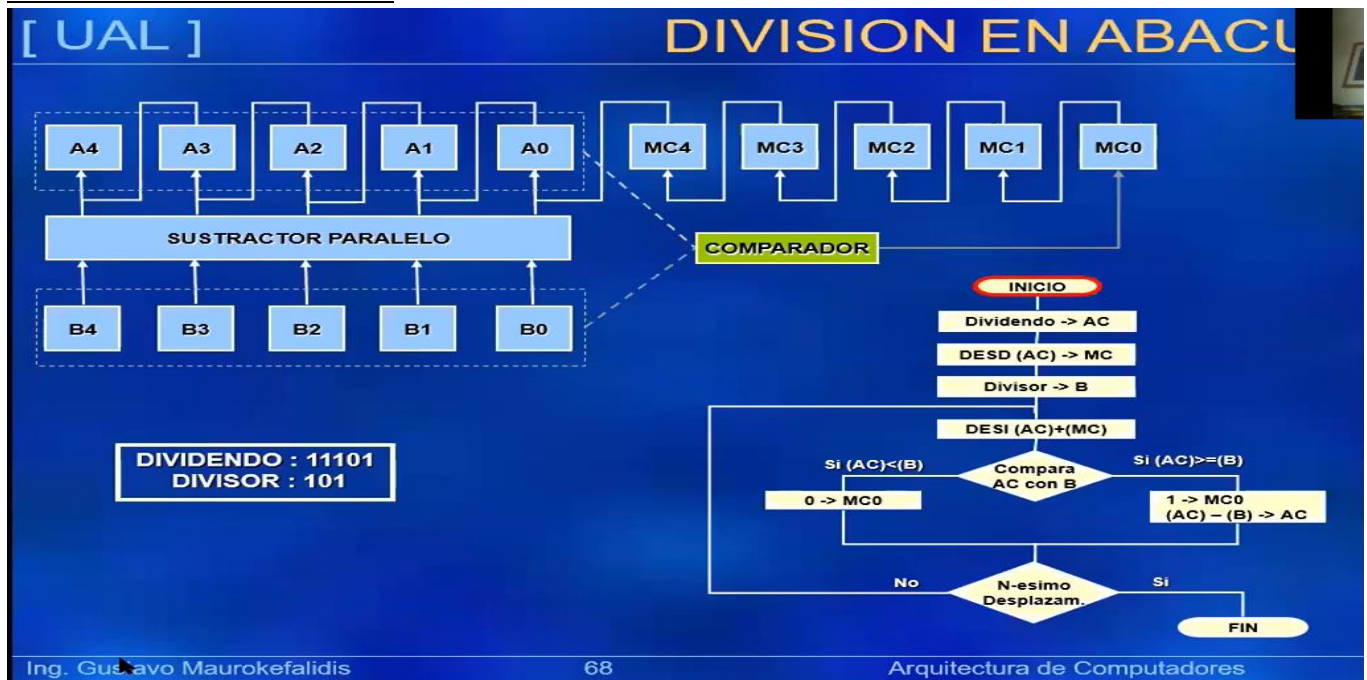
Cuando viene un 1, se desplaza a izquierda y se suma el contenido, cuando hay un 0 solo se desplaza a izquierda



Como está en Meinadier:



## DIVISION EN ABACUS

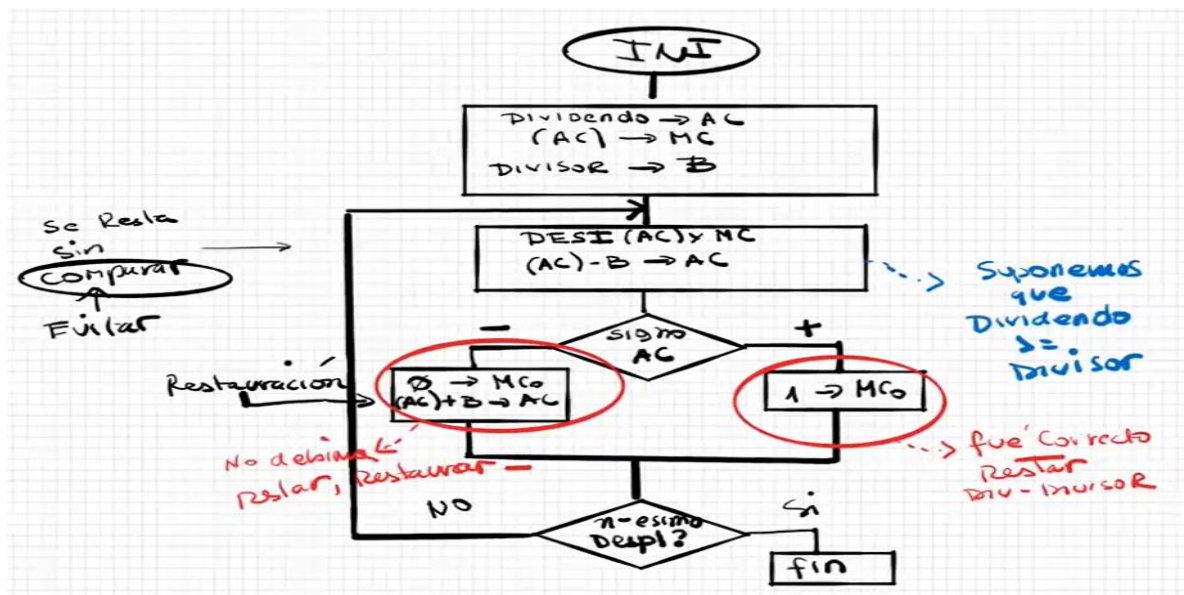


Este modelo es económico y útil ya que se reutiliza el mismo modelo tanto para multiplicar como para dividir, solo cambia su lógica de operación.

Sin embargo, es inviable debido a su complejidad, ya que su costo y tamaño crecen abruptamente para sistemas más extensos de mayores bits.

## DIVISION CON RESTAURACION

Alternativa para el modelo anterior, es mejor ya que se evita el comparador.



Se presupone que se puede realizar la resta. Se hace el desplazamiento para realizar la resta, se resta (sin comparar). Ahora se ve el signo, 0 o 1 (no hace falta un comparador para ello), si es 0 esta correcto, si es negativo, se debe deshacer la operación (restauración).

## CONSIDERACIONES

[ UAL ]
Consideraciones Finales

- En General, el diseño de la ALU juega un papel muy importante en los ciclos de cálculo de computador, incidiendo en gran porcentaje sobre la performance de la arquitectura del computador.
- El tipo y cantidad de operadores cableados o microprogramados será preponderante en la relación costo/performance.
- Observamos que ciertas funciones se han optado por cablear con el tiempo y en función de los resultados obtenidos, tal el caso de las operaciones en coma flotante, que hasta los 90 estaban microprogramadas y era una opción la incorporación de un Co Procesador adicional para estas operaciones que mejoraba notablemente la performance global
- En lo particular, la ALU de ABACUS consiste en un dispositivo que cablea operaciones básicas tales como suma/sus y algunas operaciones lógicas básicas. A su vez desarrolla esquemas que puedan implementar operaciones mas complejas como el manejo de números con coma, y la multiplicación y división algebraica binaria.
- A su vez prescinde de un registro fuente de entrada (Bi obligando al registro palabra de memoria a sostener todo el tiempo los valores lógicos del Bus M en pos del calculo solicitado por la instrucción en curso.

## Memoria

### MEMORIA

Dispositivo electrónico capaz de almacenar de manera temporal una información binaria de tal forma que todo órgano que requiera servirse de él, pueda acceder a la información solicitada en cualquier momento

### PUNTO DE MEMORIA

Es el soporte físico que permite almacenar la mínima unidad de información binaria. Deberá contar con la lógica necesaria para poder leer o escribir información binaria en él, tal el caso de los antiguos núcleos de ferrita o el Biestable Electrónico

Como la manera de conectar órganos es a través de buses, es por esto que cada órgano tiene un registro para permitir la comunicación o transmisión de información.

Dato Color: Punto de Memoria de memoria dinámica son Transistores

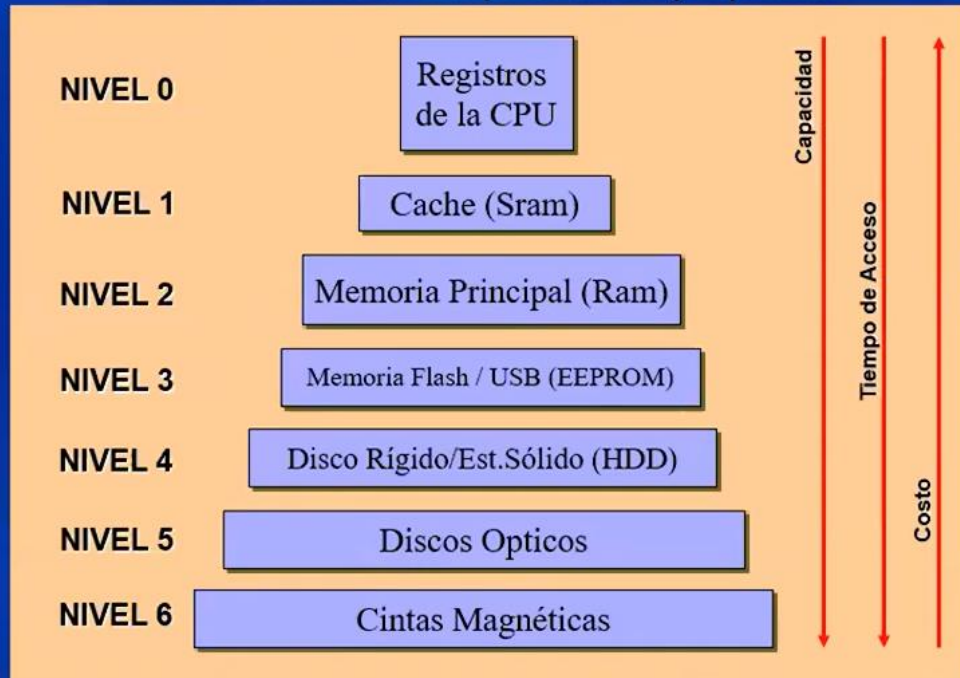
## Tipos de Memoria

| Tipo de Memoria                       | Clase                           | Borrado                  | Mecanismo de Escritura | Volatilidad |
|---------------------------------------|---------------------------------|--------------------------|------------------------|-------------|
| Memoria de Acceso aleatorio (RAM)     | Memoria de lectura / escritura  | Eléctricamente por Bytes | Eléctricamente         | Volátil     |
| Memoria de solo Lectura (ROM)         | Memoria de solo Lectura         | No Disponible            | Mediante Máscaras      | No Volátil  |
| ROM Programable (PROM)                |                                 |                          | Eléctricamente         |             |
| ROM Borrable (EPROM)                  | Luz Ultravioleta, chip completo |                          |                        |             |
| Memoria FLASH                         | Eléctricamente por bloques      |                          |                        |             |
| PROM Borrable eléctricamente (EEPROM) | Eléctricamente por bytes        |                          |                        |             |

Memoria Volátil: Es la RAM; puede perder el contenido.

## Jerarquía de memorias

En un ordenador hay una jerarquía de memorias atendiendo al tiempo de acceso y a la capacidad que normalmente son factores contrapuestos por razones económicas y en muchos casos también físicas. Comenzando desde el procesador al exterior, es decir en orden creciente de tiempo de acceso y capacidad



Los niveles representan campos de aplicación. Que más de un elemento esté en el mismo nivel representa que tienen características de acceso y capacidad similares.

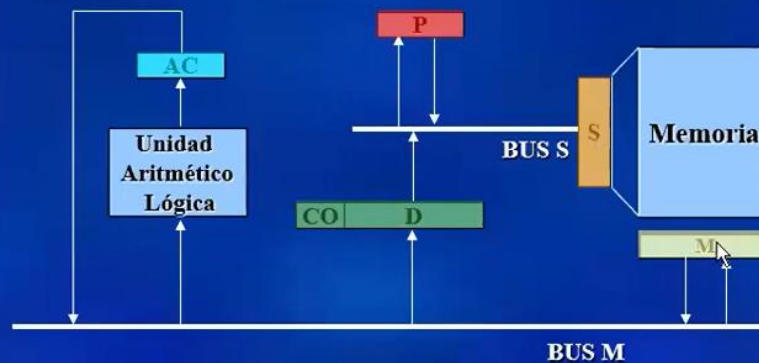
Dato Color: En el nivel 4, Los SSD (discos rígido) son mucho más veloces que los discos mecánicos, pero menos veloces que los del nivel siguiente.

NIVEL 0:

### NIVEL 0 - Registros Del Procesador :

Estos registros interaccionan continuamente con la CPU (porque forman parte de ella). Los registros tienen un tiempo de acceso muy pequeño y una capacidad mínima, normalmente igual a la palabra del procesador (1 a 8 bytes).

Los Registros intermedios constituyen un paso intermedio entre el procesador y la memoria, tienen un tiempo de acceso muy breve y muy poca capacidad.



NIVEL 1:

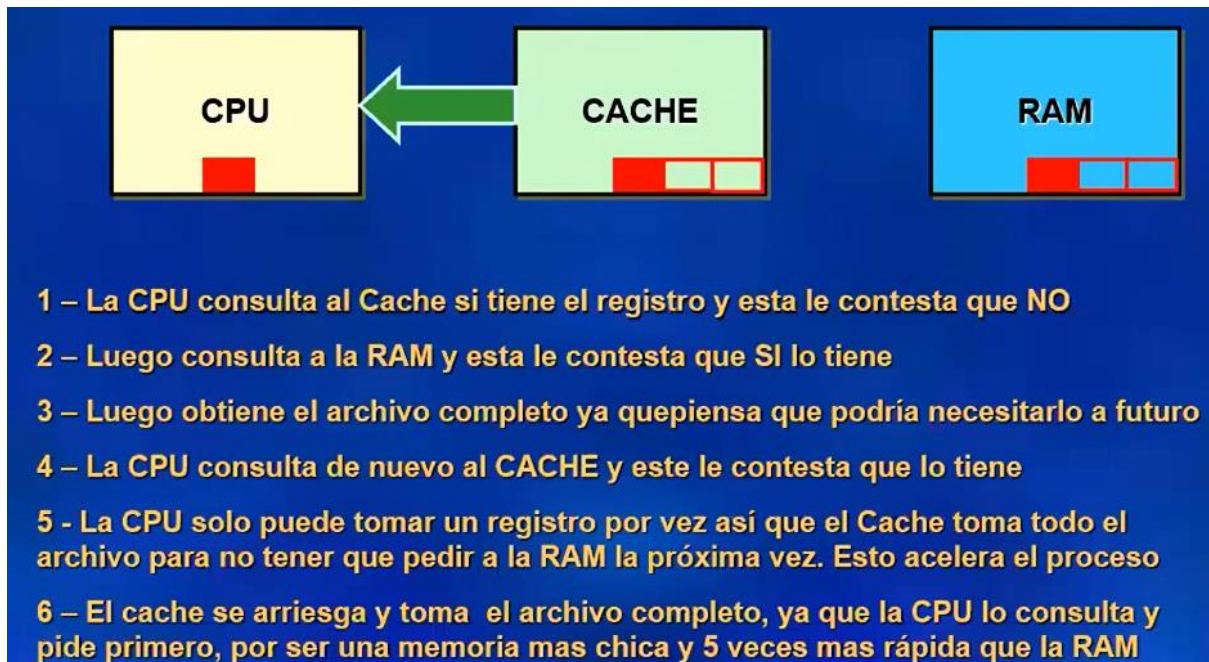
### NIVEL 1 - Memorias Caché:

Son memorias de pequeña capacidad.

Normalmente una pequeña fracción de la memoria principal. y pequeño tiempo de acceso. Este nivel de memoria se coloca entre la CPU y la memoria central. Hace algunos años este nivel era exclusivo de los ordenadores grandes pero actualmente todos los ordenadores lo incorporan. Dentro de la memoria caché puede haber, a su vez, dos niveles denominados caché on chip, memoria caché dentro del circuito integrado, y caché on board, memoria caché en la placa de circuito impreso pero fuera del circuito integrado, evidentemente, por razones físicas, la primera es mucho más rápida que la segunda. Existe también una técnica, denominada Arquitectura Harvard, en cierto modo contrapuesta a la idea de Von Newmann, que utiliza memorias caché separadas para código y datos. Esto tiene algunas ventajas como se verá posteriormente

Son rápidas pero de poca capacidad.

Ejemplo de funcionamiento:

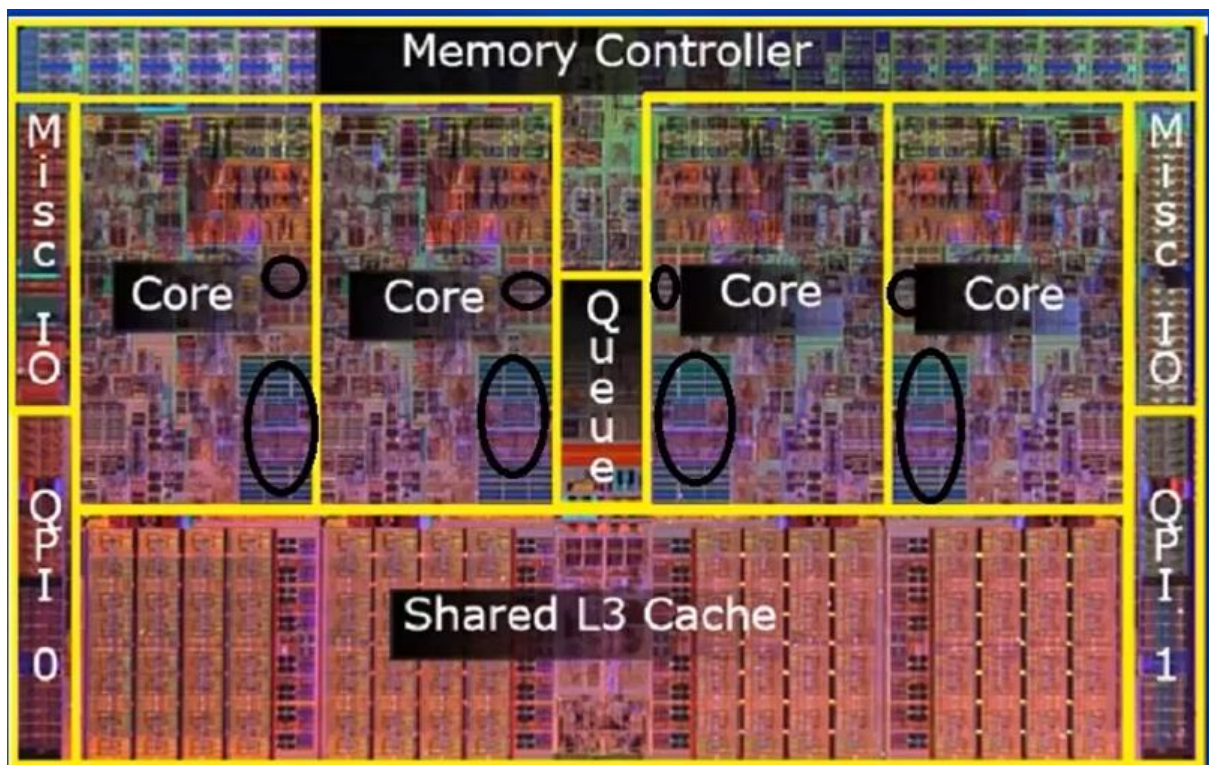


Por niveles:

Nivel 1: memoria básica, pequeña (menos capacidad que las demás), la más cercana al procesador y la más rápida y cara.

Nivel 2: Sigue estando cerca del núcleo, pero ya posee mayor capacidad. Sus tiempos de acceso son más lentos.

Nivel 3: Son mas lentos, mas alejados de los núcleos pero poseen una gran capacidad de almacenamiento.



## NIVEL 2

### NIVEL 2 – Memorias RAM

En este nivel residen los programas y los datos. La CPU lee y escribe datos en él aunque con menos frecuencia que en los niveles anteriores. Tiene un tiempo de acceso relativamente rápido y gran capacidad.

Las Extensiones de memoria central son memorias de la misma naturaleza que la memoria central que amplían su capacidad de forma modular.

El tiempo de similar, a lo sumo un poco mayor, al de la memoria central y su capacidad puede ser algunas veces mayor.



## NIVEL 4:

### NIVEL 4 – Discos Magnéticos:

Son memorias que residen en dispositivos externos al ordenador, en ellas se archivan programas y datos para su uso posterior. También se usan estas memorias para apoyo de la memoria central en caso de que ésta sea insuficiente (memoria virtual).

Estas memorias suelen tener gran capacidad pero pueden llegar a tener un tiempo de acceso muy lento. Dentro de ellas también se pueden establecer varios niveles de jerarquía.



#### NIVEL 4 – Discos De Estado Sólido

La unidad de estado sólido o SSD (acrónimo inglés de solid-state drive) es un tipo de dispositivo de almacenamiento de datos que utiliza memoria no volátil, como la memoria flash, para almacenar datos, en lugar de los platos o discos magnéticos de las unidades de discos duros (HDD) convencionales.

la mayoría de las SSD utilizan memoria flash basada en puertas NAND, que retiene los datos sin alimentación eléctrica. Para aplicaciones que requieren acceso rápido, pero no necesariamente la persistencia de datos después de la pérdida de alimentación, las SSD pueden ser contruidos a partir de memoria de acceso aleatorio (RAM). Estos dispositivos pueden emplear fuentes de alimentación independientes, como baterías, para mantener los datos después de la desconexión de la corriente eléctrica.

#### NIVEL 5

##### NIVEL 5 – Discos Opticos:

**Un disco óptico es básicamente un pedazo de plástico lleno de marcas o huecos que un rayo láser puede leer. Cuando mas marcas podamos hacer (y leer) en su superficie, mayor será la capacidad de datos que podremos almacenar en él.**

Tiene más capacidad ya que se pueden grabar “n” discos, se entiende capacidad a eso.

#### NIVEL 6

##### NIVEL 6 – Cintas Magnéticas:

**data cartridge / backup tape / cinta magnética de datos / cartucho de datos: tecnología de almacenamiento magnético secuencial, utilizada por primera vez en el ámbito informático en 1949, la cuál está conformada por un contenedor plástico que tiene en su interior un par de carretes móviles y una larga cinta magnética, sobre la cuál se leen, regraban y almacenan bits permanentemente por medio de inducción electromagnética.-Extraído de InformaticaModerna.com.**

## Ciclo de Histéresis

A través de un núcleo ferromagnético, se hace pasar una corriente, la cual genera un campo magnético.

Se llega al punto de saturación cuando se deja de aplicar corriente, quedando en una magnetización remanente

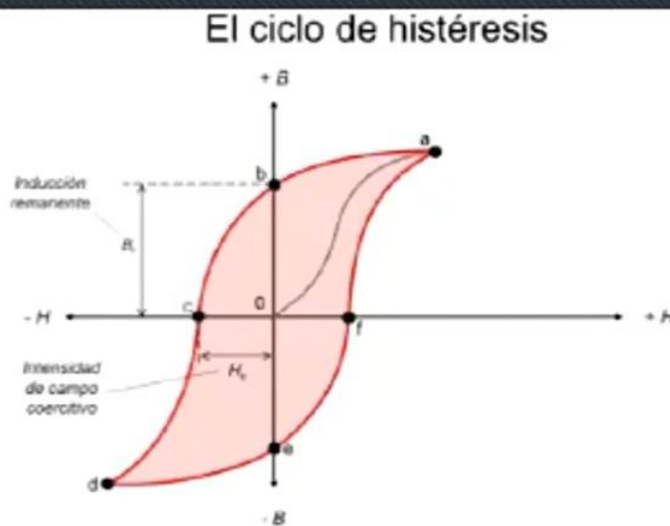
Si se baja la corriente, el campo llega a su punto de campo remanente.

La magnitud del campo remanente depende enteramente del material.

Al quedar magnetizado se representa el 1, en otro caso, un 0.

Si se aplica una fuerza coercitiva, se llega al punto nulo. Si se sigue aplicando corriente en otro sentido, se invierten los valores.

Esto llega al punto de remanencia, y luego a un nuevo punto de remanencia (opuesta a la anterior)



**Ciclo de histéresis de mat. ferromagnéticos**

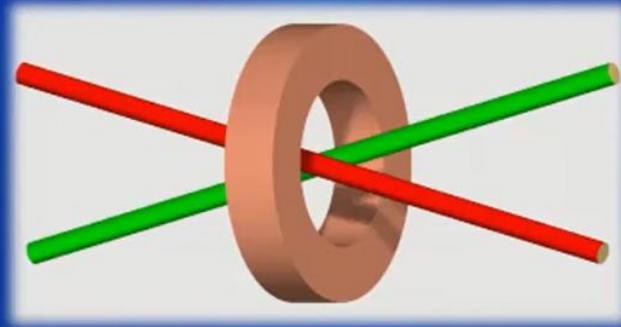
Para saber el valor del anillo, lo que se hace es agregar otro cable (transversal al de la corriente para que no se confundan) que desmagnetiza el anillo. Si estaba magnetizado (1) el cambio será brusco, lo que provocará que el cable transversal se induzca, y con esto se puede deducir que había un 1 en ese momento. Si había un 0, el cambio es muy leve.

**DATO DE COLOR:** las maquinas con este sistema, mantenían su estado de RAM aunque se apague la máquina (eran RAM dinámicas). Y utilizaban un circuito de acero para que se desmagnetice toda la memoria.

Estas memorias necesitaban 2 ciclos de memoria para acercarse y luego leerse.

## Anillos de Ferrita

### Anillo de Ferrita: Implementación

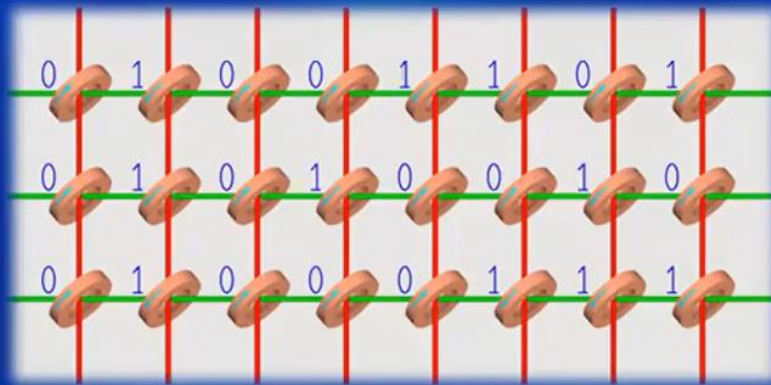


Esta tecnica planteó un gran problema: Una memoria de 1 KB con palabras de 8 bits tendría 8.192 anillos, y con dos hilos por cada anillo necesitaríamos 16.384 hilos con sus 16.384 circuitos para controlarlos.

Este sistema es muy costoso.

## Implementación 2D

### Anillo de Ferrita: Implementación 2D



El hilo que pasa por todos los anillos de una misma posición (hilo horizontal) se llama hilo de palabra, y el hilo que pasa por los anillos homólogos (hilo vertical) se llama hilo de bit. Para una memoria de 1 KB se necesitarían 1.024 hilos de palabra y 8 hilos de bit, mucho menos que los 16.384

**HILO VERDE: Hilo de palabra:** Governa la selección de la palabra

**HILO ROJO: Hilo de bit:** Governa el bit de la palabra (el valor individual)

Esta implementación hace que el sistema sea más económico

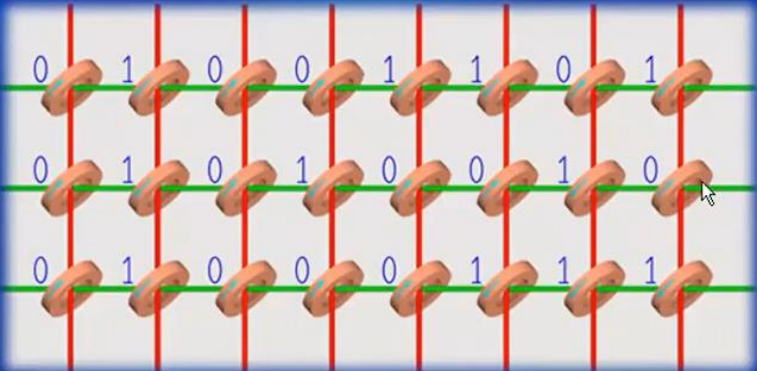
¿COMO SE GRABAN?

Esto se logra con una corriente coincidente; consiste en mandar media tensión de la requerida para cambiar a un 1 por el hilo de palabra. Donde se quiere el 1, se manda otro medio volt por el hilo de bit.

Esto se une a un Decodificador, unido al bus S (de dirección)

Escritura

### Anillo de Ferrita: Implementación 2D



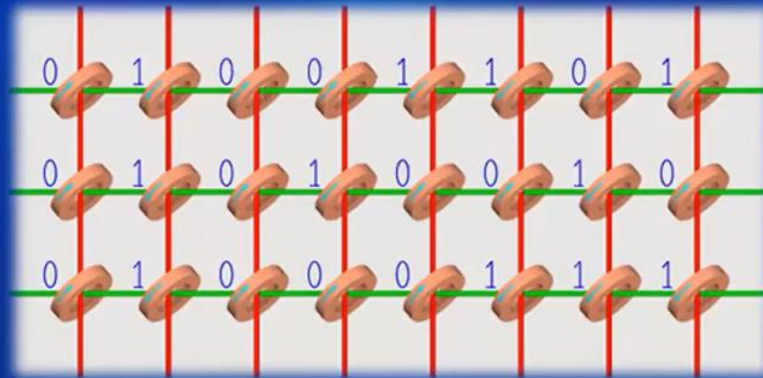
**Escritura:** Primero se pone la palabra en cero. Luego se hace pasar una corriente eléctrica positiva por el hilo de palabra y, simultáneamente, otras iguales por los hilos de bits cuyos anillos hay que poner a "1".

La intensidad de esas corrientes está establecida de forma que una, por sí sola, no es capaz de hacer que los anillos que atraviesa se pongan a "1"; pero si dos juntas. Los anillos que reciben el efecto combinado de la corriente en el hilo de palabra con la del hilo de bit se pondrán a "1".

Como consume dos ciclos (pone a cero, escribe la palabra), es lenta

Lectura:

## Anillo de Ferrita: Implementación 2D

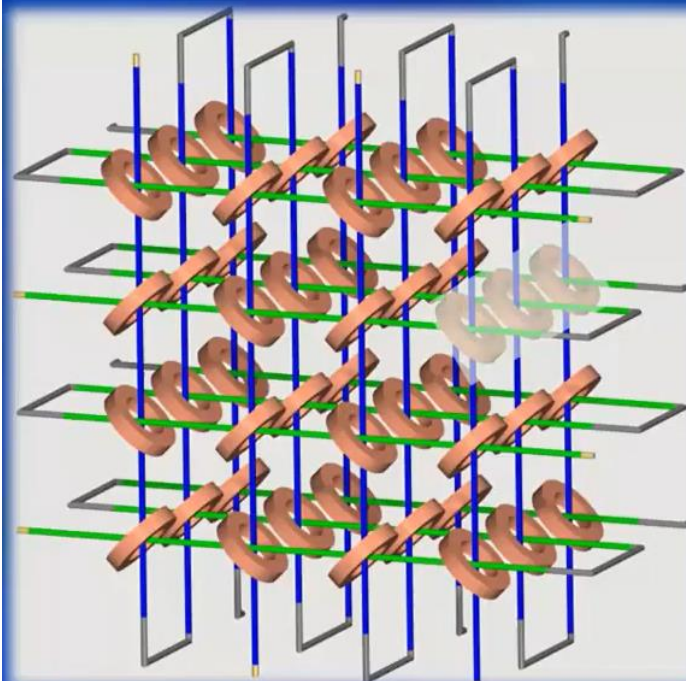


**Lectura:** Se hace pasar una corriente negativa por el hilo de la palabra elegida, que lleve a todos los anillos al estado "0" (lectura destructiva).

Los que tienen un "0" siguen igual, y los que tienen un "1" pasan a "0" induciendo una corriente apreciable en su hilo de bit. Pero como esta corriente es menor que la necesaria para forzar una conmutación, no afecta al estado de los demás anillos atravesados por el mismo hilo de bit.

### Implementación 3D

## Anillo de Ferrita: Implementación 3D



Con el fin de disminuir aun mas la cantidad de hilos la Organización 3D plantea superponer planos colocándolos alternados, uno al derecho y otro al revés. Finalmente, se conectan en serie los hilos de selección homólogos uniéndolos en lados alternos, de modo que se forme una especie de zig-zag con cada hilo. Un decodificador de direcciones X (Horizontal) e Y (Vertical) logran a través de corrientes coincidentes, seleccionar una palabra espacial (Ej: tres primeros planos en memoria de 4x4 bits)

Ventajas y Desventajas de ambas

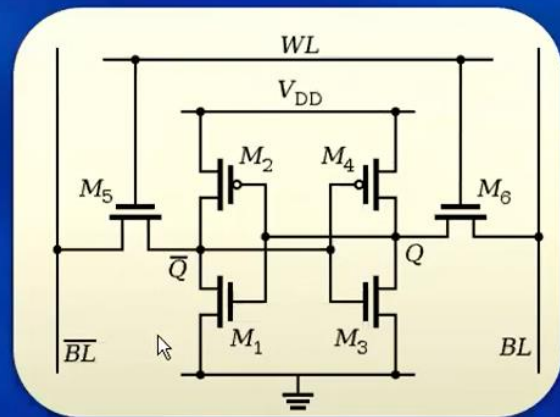
Ventaja de 2D: Son simples de construir

Desventajas de **2D**: Crecen de manera lineal

Ventajas de **3D**: Ocupan menos lugar, tienen menos conexiones, cables, etc. Trata de mejor manera las palabras.

## RAM ESTÁTICA

- Estas memorias son de acceso aleatorio, las posiciones en la memoria pueden ser escritas o leídas en cualquier orden, independientemente de cual fuera la última posición de memoria accedida.
- Cada bit en una SRAM se almacena en cuatro transistores, que forman un biestable.
- Este circuito biestable tiene dos estados estables, utilizados para almacenar (representar) un 0 o un 1.
- Se utilizan otros dos transistores adicionales para controlar el acceso al biestable durante las operaciones de lectura y escritura.
- Una SRAM típica utilizará seis MOSFET para almacenar cada bit.



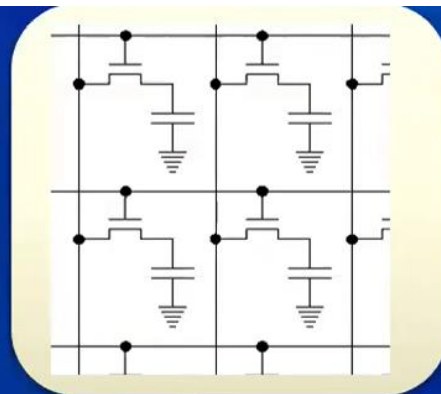
- El acceso a la celda es controlado por un bus de control (WL en la figura), que controla los dos transistores de acceso M5 y M6, quienes controlan si la celda debe ser conectada a los buses BL y BL. Ambos son utilizados para transmitir datos tanto para las operaciones de lectura como las de escritura, y aunque no es estrictamente necesario disponer de ambos buses, se suelen implementar para mejorar los márgenes de ruido.

Ventajas: alta velocidad para guardar 1's o 0's. Los tiempos de acceso son muy bajos. Dedicadas a memorias de hardware. (Memorias de hardware y de la unidad de control, se encuentran en la cima de la pirámide de jerarquías de memoria)

Pero son mucho más caras y consumen mayor corriente.

## RAM DINÁMICA

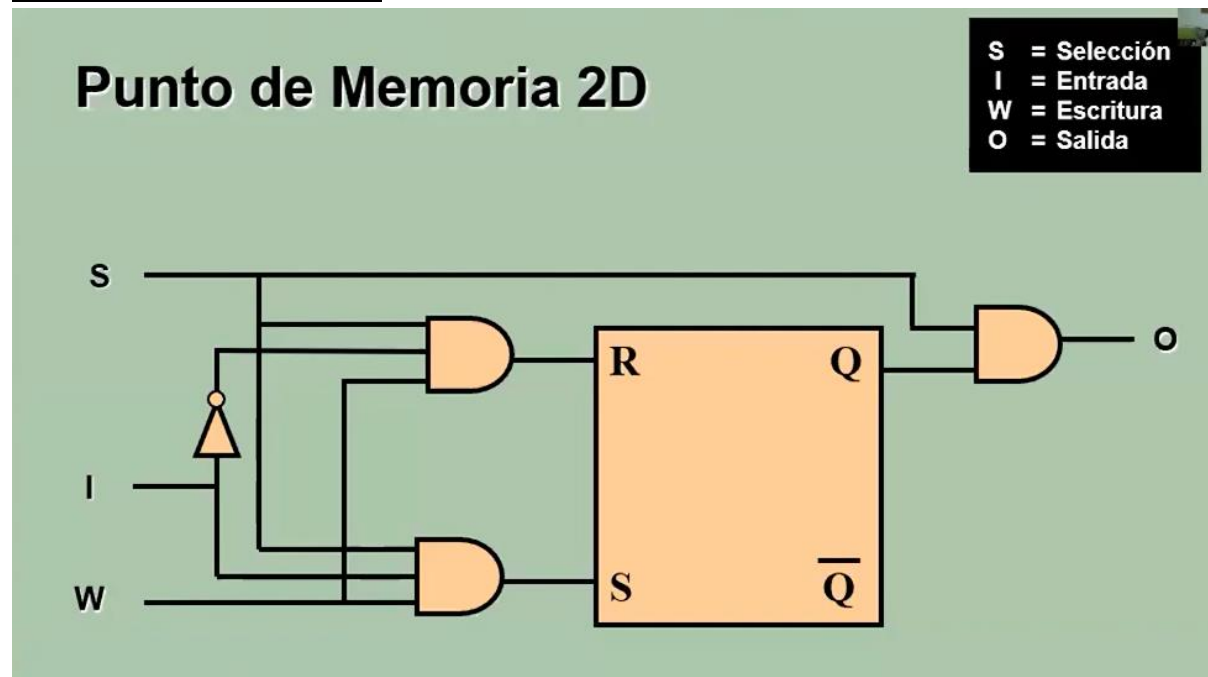
- La memoria dinámica fue desarrollada en los laboratorios de IBM pasando por un proceso evolutivo que la llevó de usar 6 transistores a sólo un condensador y un transistor. La invención de esta última la hizo Robert Dennard quien obtuvo una patente norteamericana en 1968 por una memoria fabricada con un solo transistor de efecto de campo y un condensador.
- Las columnas son precargadas a un voltaje igual a la mitad del voltaje de 1 lógico. Esto es posible ya que las líneas se comportan como grandes condensadores, dada su longitud tienen un valor más alto que la de los condensadores en las celdas.
- Una fila es energizada por medio del decodificador de filas que recibe la dirección de la palabra. Esto hace que los transistores conectados a una fila conduzcan y permitiendo la conexión eléctrica entre las líneas de columna y una fila de condensadores.
- Aquella fila cuya media carga indique la posición de la palabra, cargará solo los capacitores cuya columna tengan la media carga restante necesaria para hacerlo.



Ventajas: requiere menos elementos, menos costosa.

Desventaja: Es más lenta, se tienen que cargar los capacitores.

## PUNTO MEMORIA 2D



Organización de memoria central cuyos puntos de memoria están representados lógicamente por FF.

Permiten seleccionar el punto de memoria el cual se quiere escribir, leer o leer y escribir.

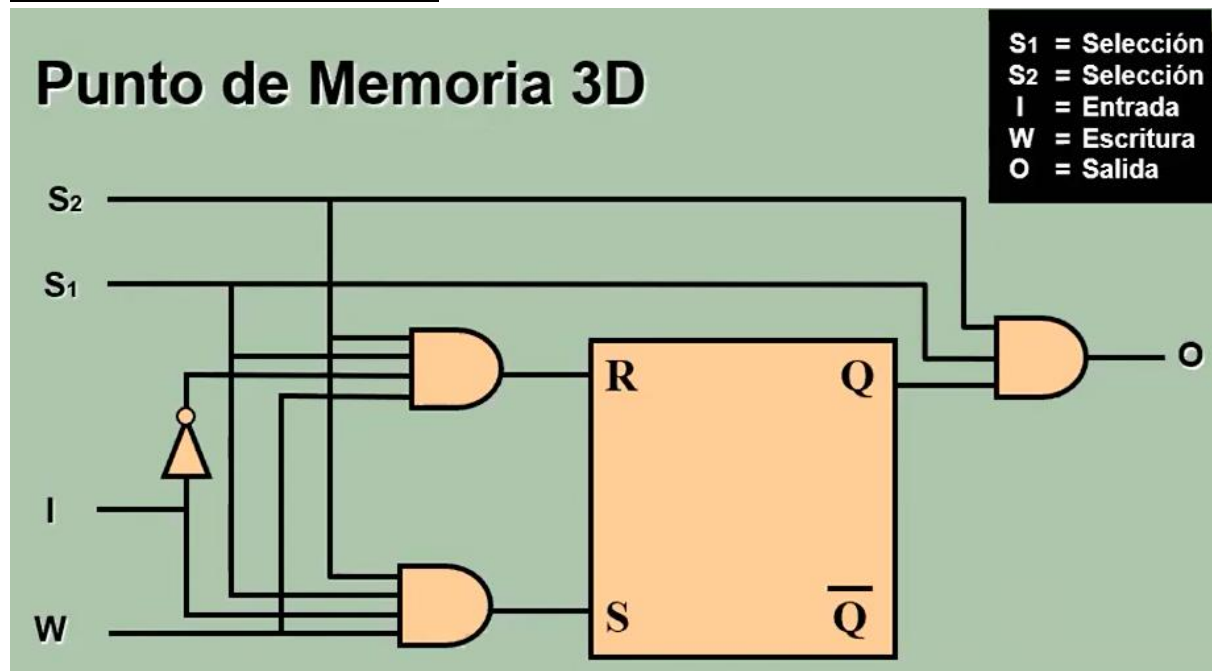
El hecho de seleccionar produce la lectura automática de la información.

Para evitar la lectura automática, se agregan compuertas AND para ver si se quiere hacer esa acción o no.

Ventaja de Organización 2D: Tiene un punto de memoria simple, una arquitectura sencilla, económica y de fácil implementación

Desventaja de Organización 2D: El decodificador se vuelve costoso. Requiere mucho espacio físico.

## PUNTO DE MEMORIA 3D



Posee una selección más.

Funciona similar a la 2D, pero para la lectura, se deben seleccionar a la vez S1 y S2. La escritura igual también lo requiere.

Este sistema es más costoso.

Al ser en 3D, se utiliza S2 para poder relacionar los bits con la palabra, es decir, se relacionan filas y columnas.

La longitud está dada por la cantidad de planos.

S1: Decodificador de filas

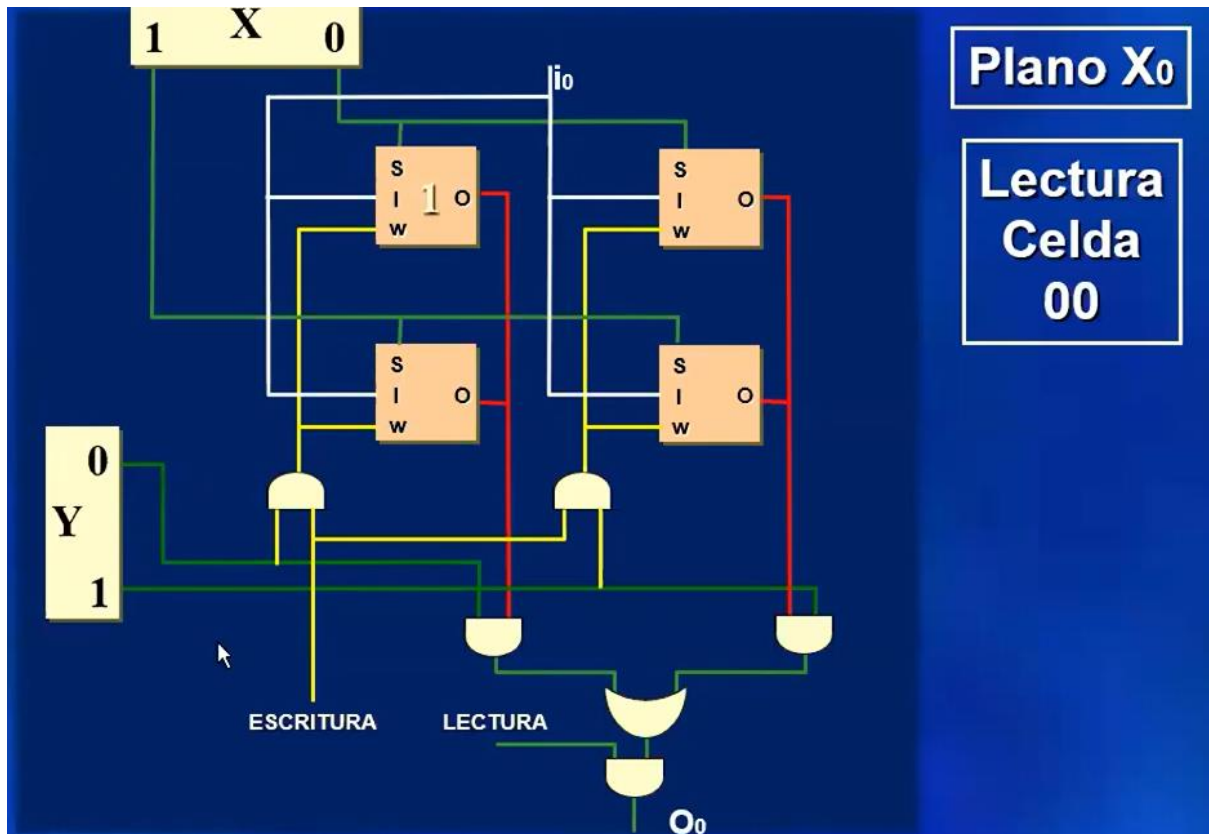
S2: Decodificador de columnas

Desventaja de Organización 3D: El punto de memoria, ya que, al tener 11 entradas, es mucho más cara.

## ORGANIZACIÓN DE MEMORIA DE 2 ½ D

Aprovecha que la 2D tiene un solo selector, lo cual es económico, y utiliza la lectura de múltiples niveles o planos de la 3D.

Representación de 4 palabras de 1 bit



Se selecciona la columna donde se desea realizar las operaciones, luego se envían a todos los componentes las ordenes de lectura/escritura, de esta manera solo se ejecutan las operaciones en la palabra seleccionada (que se encuentra en el espacio).

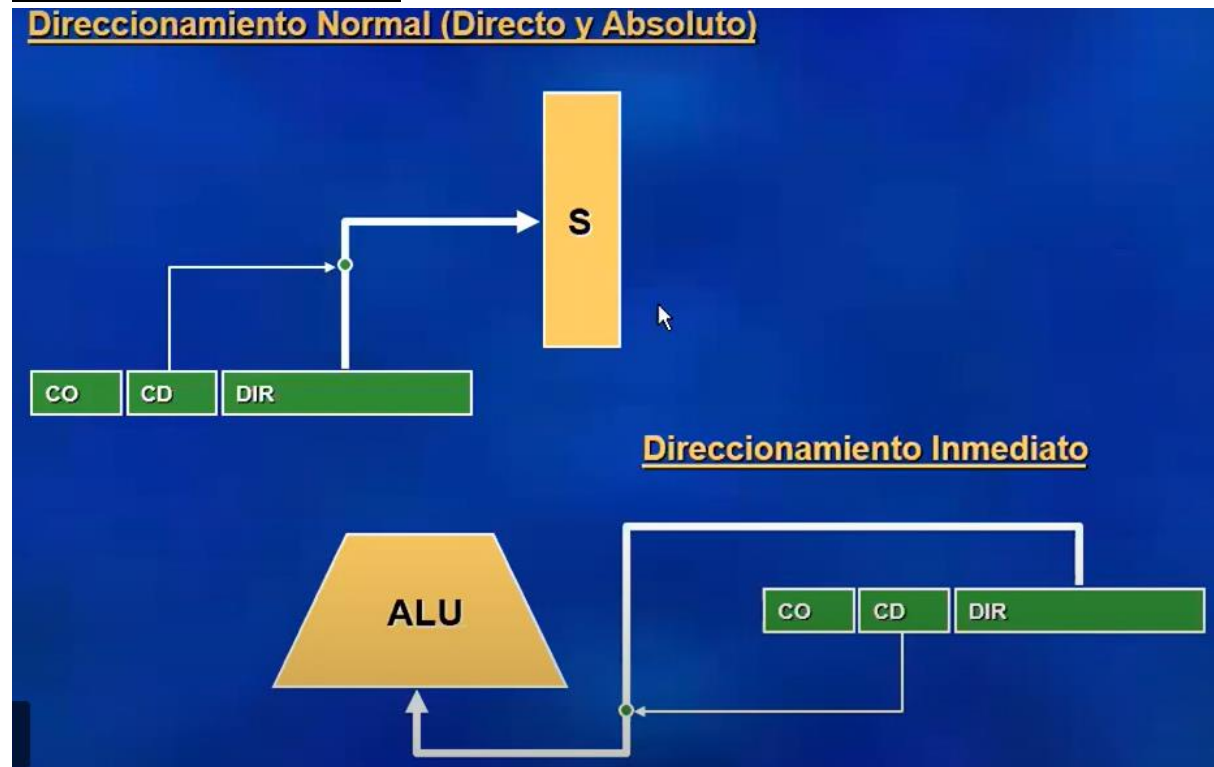
Una pequeña desventaja de este modelo es que aumenta un poco su costo la necesidad de manejar una periferia combinacional. Sin embargo, sigue siendo mucho más rentable que las demás.

## METODOS DE DIRECCIONAMIENTO

El direccionamiento de memoria es una técnica que permite que la arquitectura pueda acceder a una palabra de la memoria central a través de una estrategia específica que permita hacerlo de una manera específica.

Esto se encuentra en la lógica digital, es decir, si queremos darle algún valor agregado a la arquitectura se tendrá que "soldar a mano" el cableado.

## Direccionamiento Directo



La dirección contiene la posición física de la palabra.

Es la técnica más sencilla. Es la más usada.

En cuanto a ventajas y desventajas, no es muy sencillo, ya que son estrategias distintas. Si bien si puede ser uno más eficiente que otro en determinada situación, no se puede decir que tienen ventajas o no.

## Direccionamiento Inmediato

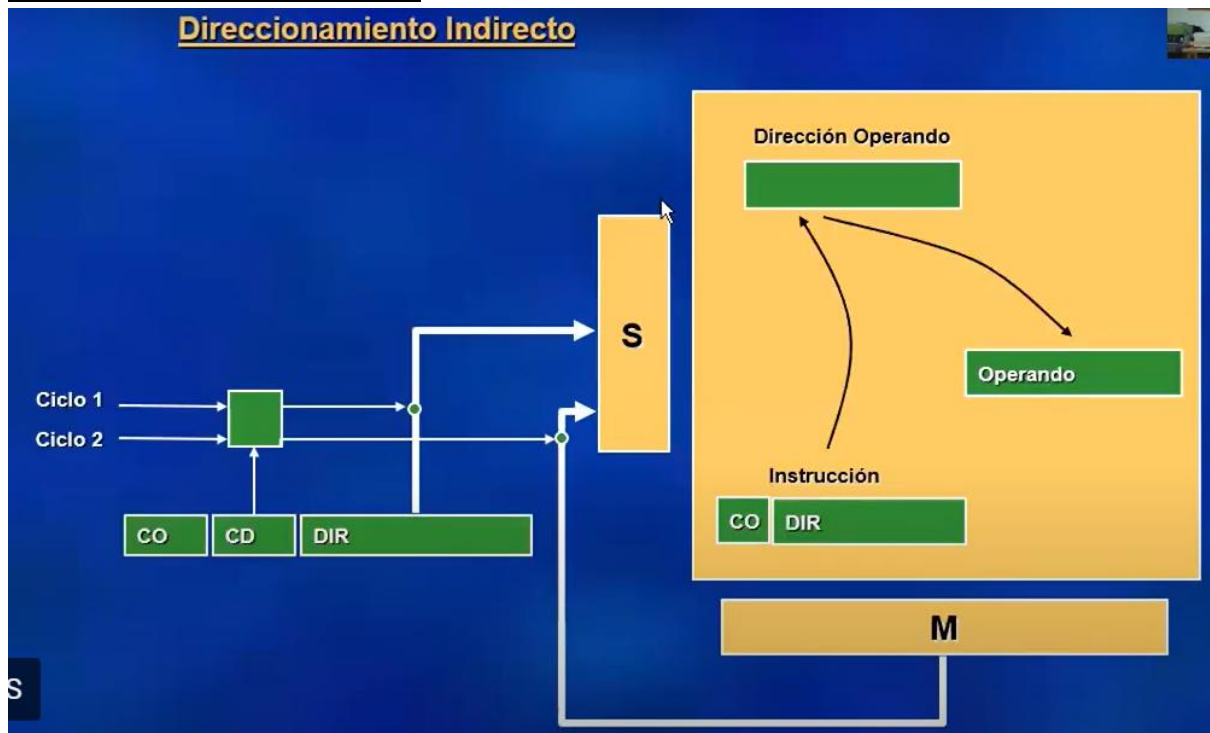
(Imagen arriba)

Dada una condición de direccionamiento, se dice que se hará un direccionamiento inmediato cuando la arquitectura obligue al contenido del campo DIR a no ir a la memoria, sino a la ALU para ser operando, ya que se trata de un operando en si mismo.

Lo que se hace es enviar a DIR directamente a la ALU, porque es un operando.

Esto lo hace un método muy rápido, pero se limita el tamaño del operando a solo el tamaño de DIR.

## Direccionamiento Indirecto



Consiste en acceder a una palabra de memoria de manera indirecta. La dirección de DIR contiene la dirección de memoria que posee la dirección de la palabra que se desea, es algo similar a un puntero. Esto involucra dos ciclos, el 1ro para obtener la dirección de la palabra que se desea, y el segundo para acceder a dicha palabra.

Tiene que haber un Bus extra que conecte el Bus M y el Bus S. Hay una señal llamada TMS de nivel, que permite el paso del Bus M a S (Transferencia de M a S).

Posee una desventaja, ya que consume dos ciclos. Pero tiene grandes ventajas, ya que permite direccionar palabras que no serían direccionables solamente por el campo DIR.

Esta técnica permite direccionar  $2^m$  (siendo m el tamaño del reg M).

## Direccionalamiento por Base y Desplazamiento

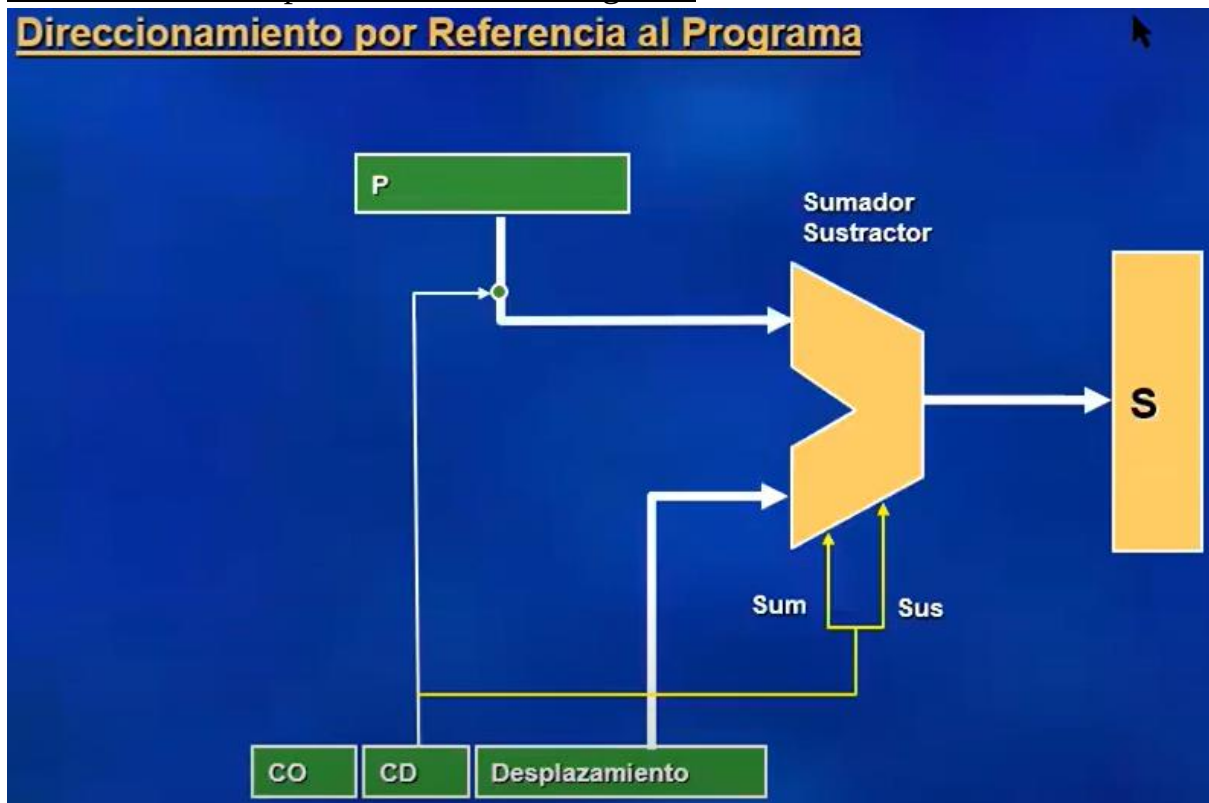
Diagrama de direccionamiento por base y desplazamiento:

- Se muestra un registro **BASE** (verde) y un registro **S** (naranja).
- Se muestran tres registros de entrada: **CO** (verde), **CD** (verde) y **Desplazamiento** (verde).
- El registro **CD** apunta a un punto de suma (verde) que se suma al contenido del registro **BASE**.
- El resultado de la suma se utiliza para direccionar el registro **S**.
- El registro **Desplazamiento** también apunta al registro **S**.

El problema que implica es que se consume tiempo para realizar la operación. Pero otorga una gran ventaja, que es de solo cargar los desplazamientos al campo DIR.

## Direccionamiento por Referencia al Programa

### Direccionamiento por Referencia al Programa



Consiste en que la dirección efectiva de la palabra de memoria está en +/- el contenido del contador del programa.

Nos basamos en el contador del programa, y es este registro quien nos hace recorrer la memoria a medida que aumenta.

Se usa para realizar barridos de palabras que están juntas.

## Direccionamiento por Página

### Direccionamiento por Página ( o por Yuxtaposición)

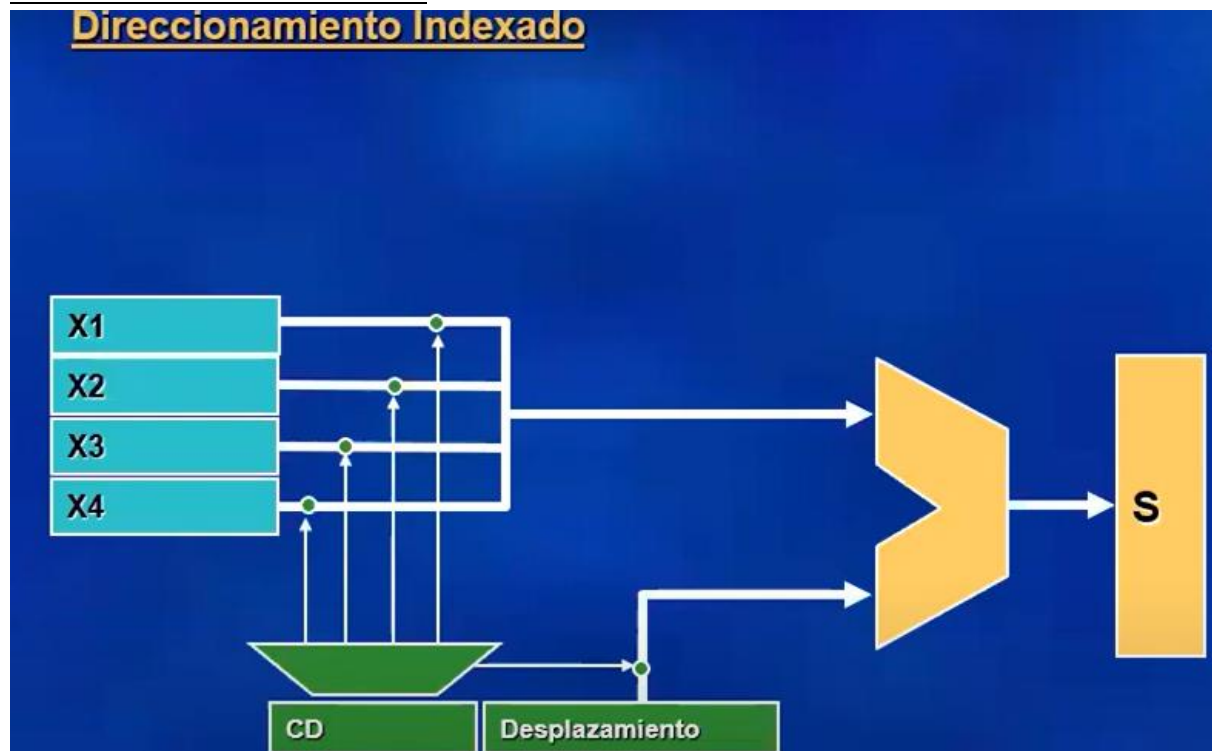


Consiste en concatenar el contenido de los bits de mayor peso del contador del programa con el valor del desplazamiento.

La memoria se organiza en  $2^P$  paginas.

Esto es muy útil para implementar la multiprogramación, ya que se podrían estar ejecutando distintos códigos en distintas páginas a la vez.

## Direcccionamiento Indexado

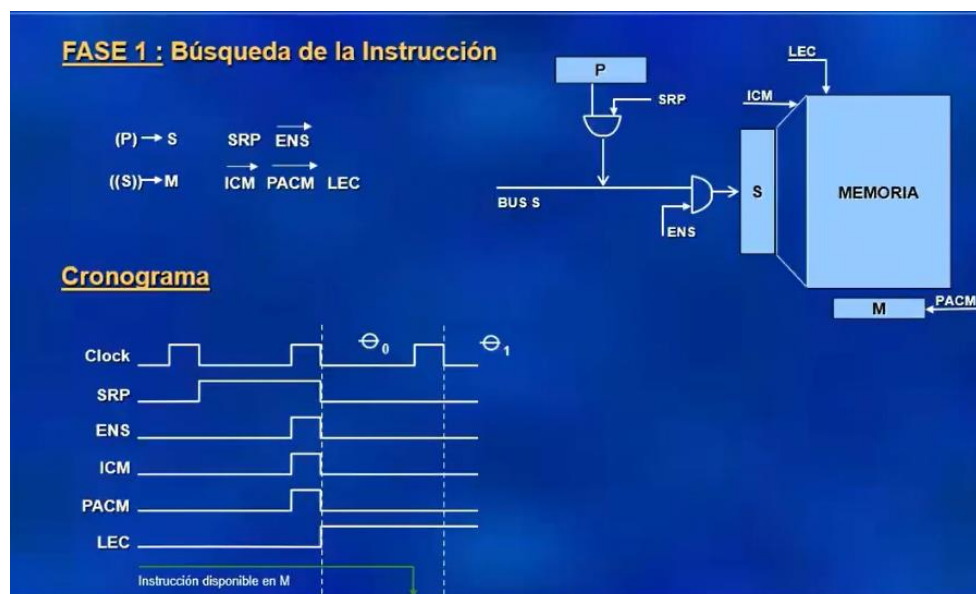


Es uno de los más usados. Se debe indicar el número del índice al que se desea acceder. Se suma el Desplazamiento con el Índice. Hay instrucciones especiales para cargar el valor de los índices, otra para incrementar los índices, y otra para que se pueda decrementar los índices.

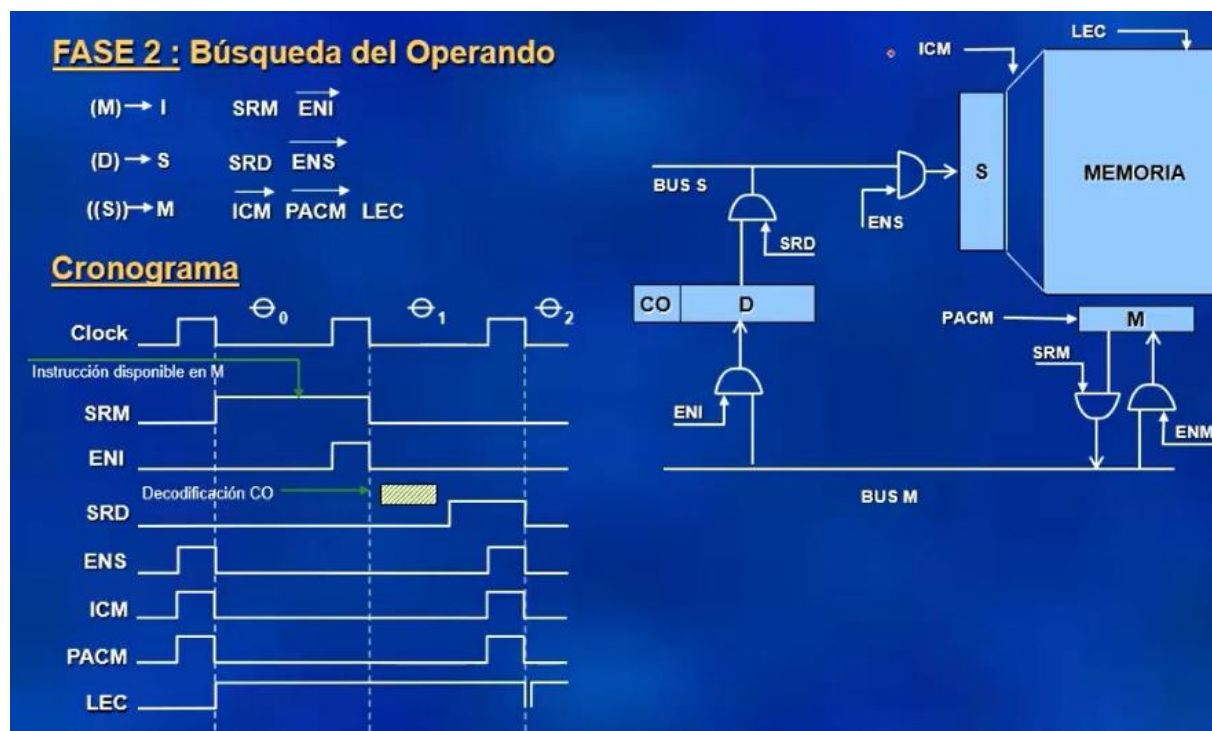
## RUTA DE DATOS

En mayúsculas están las instrucciones de la ALU, en minúsculas van los códigos de operaciones.

SUMA EN ABACUS:



Fase de búsqueda del operando:



Lo primero que se hace en esta etapa es: el contenido del RPM (M) se manda al ámbito de la unidad de control, para esto se debe habilitar SRM (que es de nivel para que se establezca la información en el Bus M). Cuando esto concluye se activa ENI para que pueda ingresar al RI.

En el Cuadrado (Decodificar) la unidad de control debe decodificar el código de operación de la instrucción, ya que si es un salto incondicional, por ejemplo, no hace falta buscar un operando.

La LEC en la suma dura 4 batidos, 2 para leer la instrucción y otros 2 para leer el operando. Esto debido a que la lectura en ABACUS es destructiva, ya que se usan núcleos magnéticos.

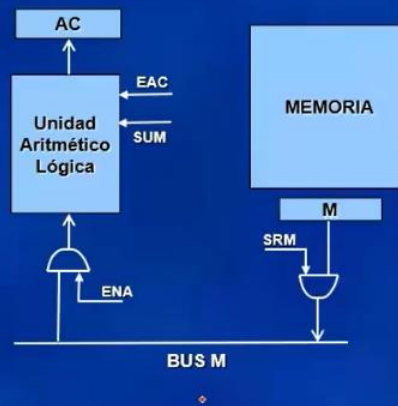
Ahora, como pasamos a la lectura y sabemos que el operando está en el RPM, lo que tenemos que hacer es enviar dicho operando a la ALU para que se realice la SUM contra el acumulador.

Fase 3:

### FASE 3 : Ejecución de la Suma

$(MC) + (AC) \rightarrow AC$     SRM   ENA   SUM   EAC

#### Cronograma

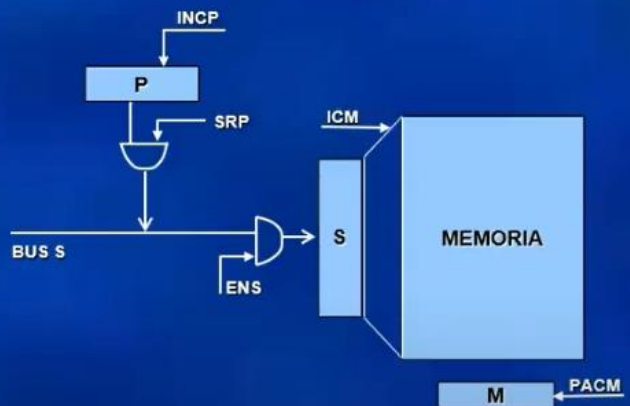


Aclaración: SRM y SUM duran 2 batidos ya que no alcanza solo con 1. Se requiere un batido y algo más, al estar asociados a un clock, se va directamente a 2 batidos del mismo.

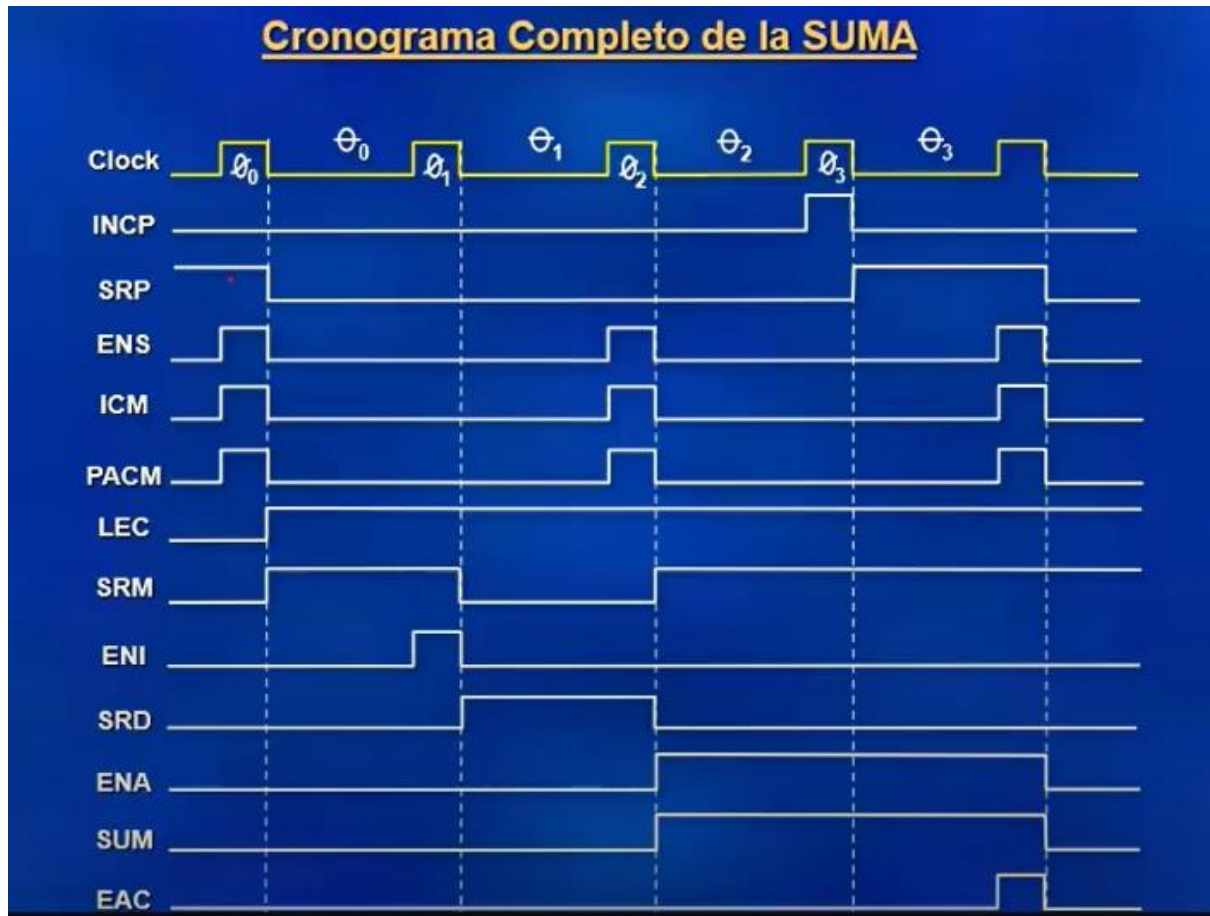
### FASE 4 : Preparación Próxima Instrucción

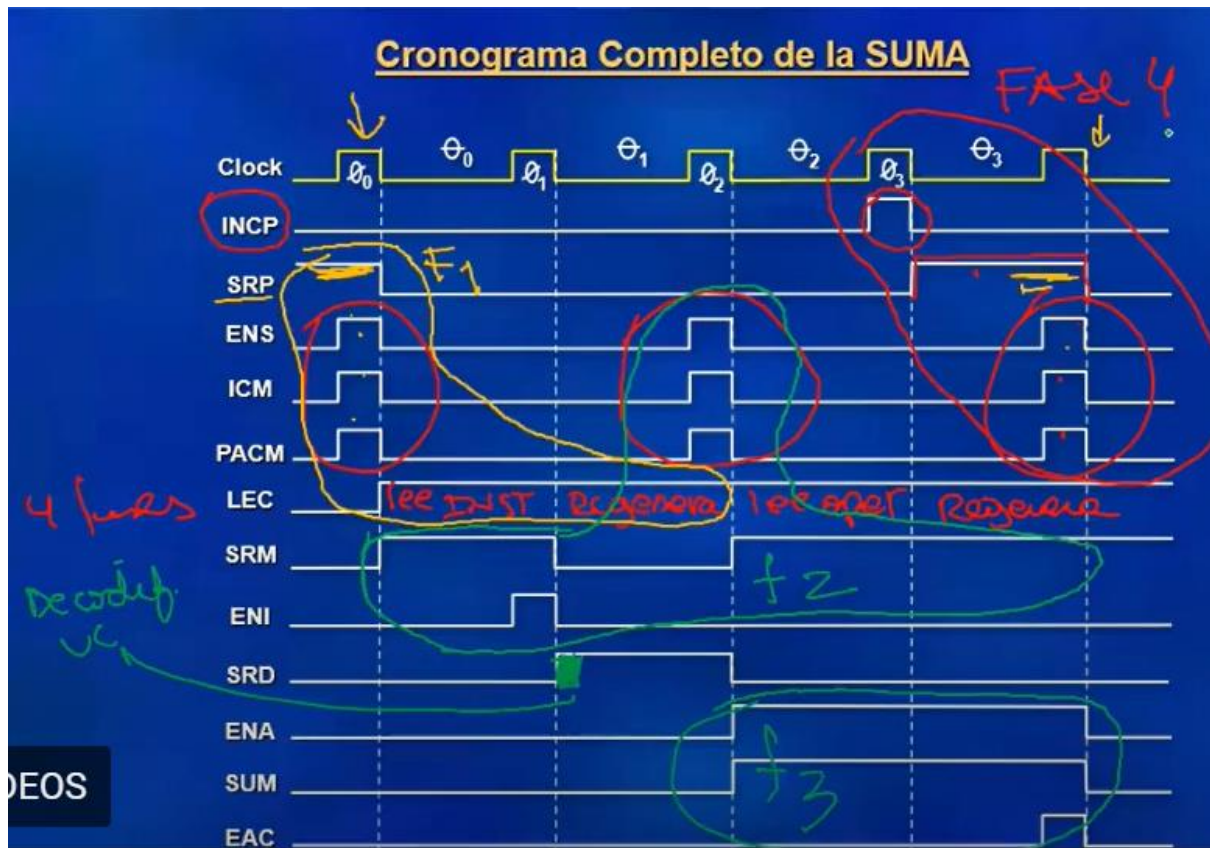
$P + 1 \rightarrow P$     INCP  
 $(P) \rightarrow S$     SRP    ENS

#### Cronograma



## Cronograma de la Suma





## DEFINICIÓN

### RUTA DE DATOS

Conjunto de Órganos para Memorizar, Transferir, o Procesar información proveniente de la memoria central en forma de instrucciones, direcciones, y operandos.

Cada Órgano cuenta con Registros de transferencia que le permiten comunicarse con el resto, a través de los Buses definidos en la Arquitectura.

La generación de las señales de gobierno que coordinen el movimiento de instrucciones, direcciones, y operandos, esta a cargo del un Organo Central conocido como Unidad de Control.

Es el todo de una arquitectura.

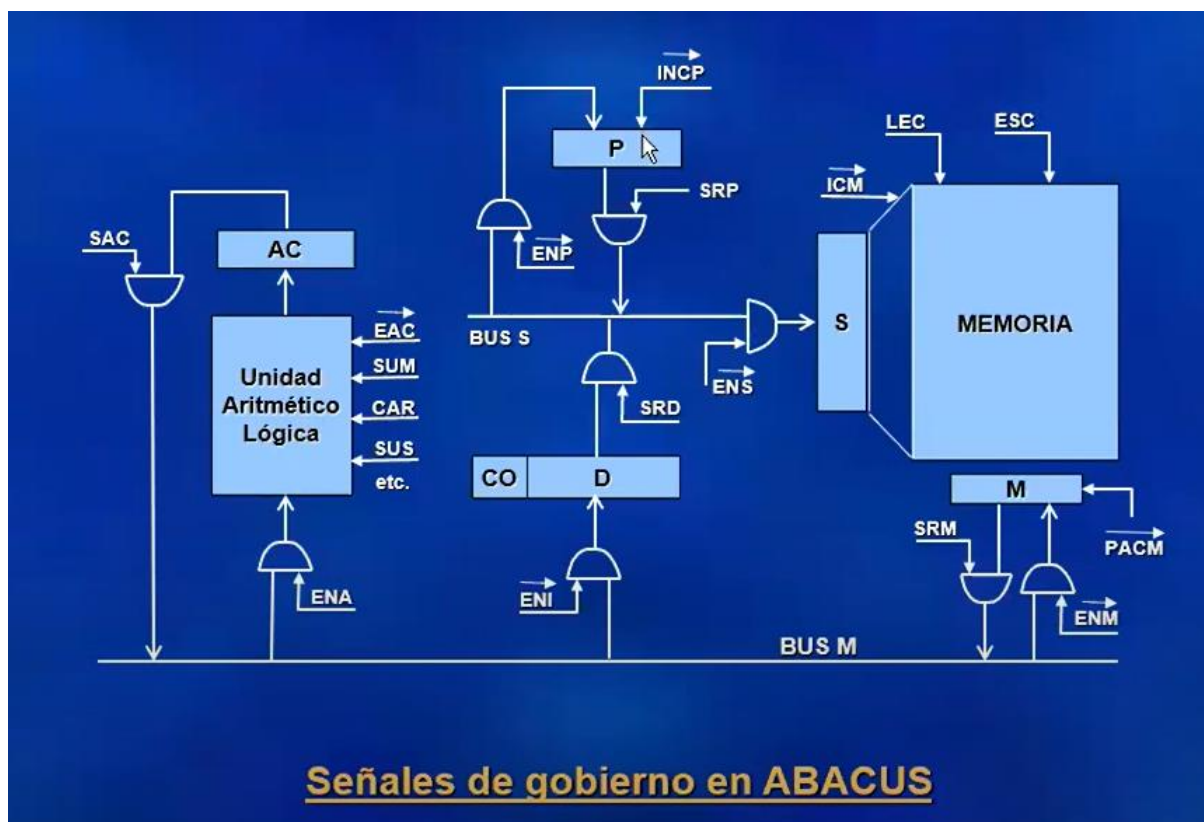
La información proviene de la memoria central (RAM). Viene en 3 maneras, con formato de instrucción, dirección de palabra de memoria o dirección de un operando.

Se compone de buses, registros, etc.

## DIMENSION DE LA RUTA DE DATOS

Es el número de dígitos transmitidos en PARALELO por la ruta de datos. Esta puede variar a lo largo de una misma ruta. Puede ser fija en lo que refiere al tamaño del registro de Instrucción y de Operando y un número más restringido para las direcciones de los operandos. En Arquitecturas más complejas la dimensión puede ser variable, de uno a más octetos para determinados órganos que requieren procesar información aritmética de simple/doble precisión por ejemplo.

Es la cantidad de dígitos que pueden traficarse en una ruta de datos. Puede ser de tamaño fijo, como ABACUS, o en otras arquitecturas más complejas, ser de tamaño variable.



## Formato de las Instrucciones

Sumar el contenido de la palabra de memoria designada por la dirección, con el contenido del registro Acumulador. Es la operación típica del grupo de instrucciones aritmético / lógicas correspondientes a la familia de instrucciones IBO (Instrucciones con Búsqueda de Operando).

Requieren para su ejecución el desarrollo de cuatro fases.

## Fases de la Suma

Requieren para su ejecución el desarrollo de cuatro fases:

- Búsqueda de la Instrucción
- Búsqueda del Operando
- Ejecución de la Suma
- Preparación de la siguiente Instrucción

|     |           |
|-----|-----------|
| SUM | Dirección |
|-----|-----------|

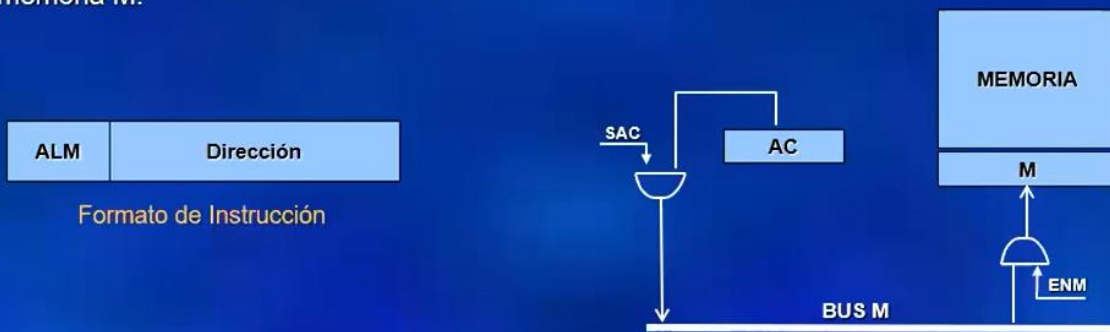
ICM es el tiempo entre la lectura (destrucción) y regeneración que se necesita para la palabra.

## Almacenamiento en ABACUS

Almacena el contenido del registro Acumulador en la palabra de memoria designada por la dirección del registro de instrucción.

El primer ciclo es idéntico al de la suma (Búsqueda de la Instrucción) mientras que el segundo ciclo se dedica a seleccionar la palabra designada por la dirección de la Instrucción, para luego escribir en ella el valor de AC.

En tal sentido el segundo ciclo dedica su tarea a estabilizar la información contenida en AC sobre el bus de datos M (señal de gobierno de nivel SAC), para que luego sea almacenado en la palabra de memoria seleccionada por el campo Dirección, a través del registro palabra de memoria M.

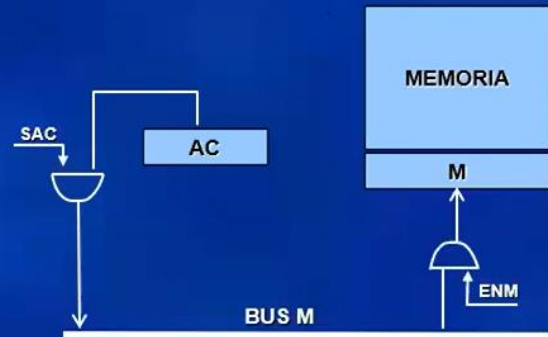
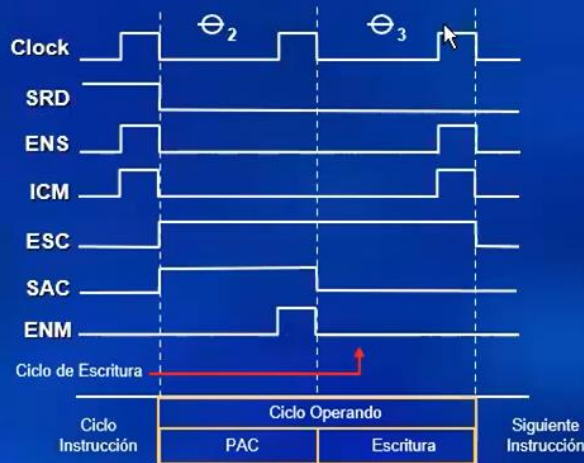


Se guarda lo que está en el acumulador en una palabra de memoria designada por la dirección del campo de DIR.

## Señales de Gobierno

$(D) \rightarrow S$      $\overrightarrow{\text{SRD}}$      $\overrightarrow{\text{ENS}}$   
 $(AC) \rightarrow M$      $\overrightarrow{\text{SAC}}$      $\overrightarrow{\text{ENM}}$   
 $(M) \rightarrow S$      $\overrightarrow{\text{ICM}}$      $\overrightarrow{\text{ESC}}$

## Cronograma

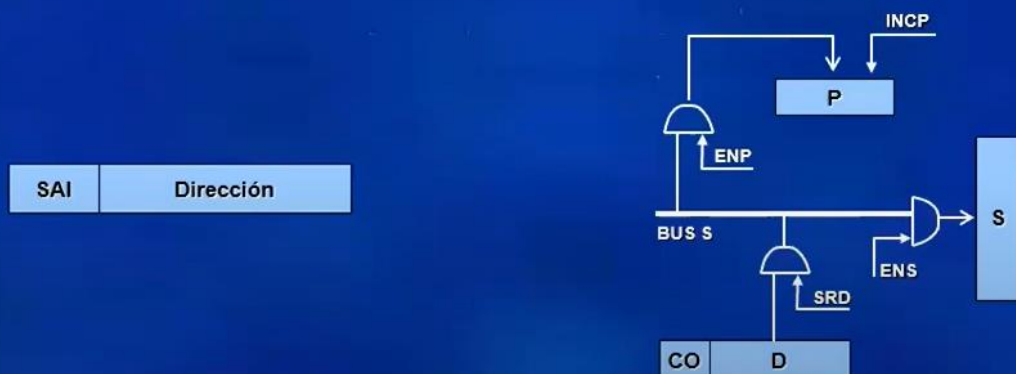


La 1er parte de la escritura corresponde a la puesta a cero (por eso la flecha roja).

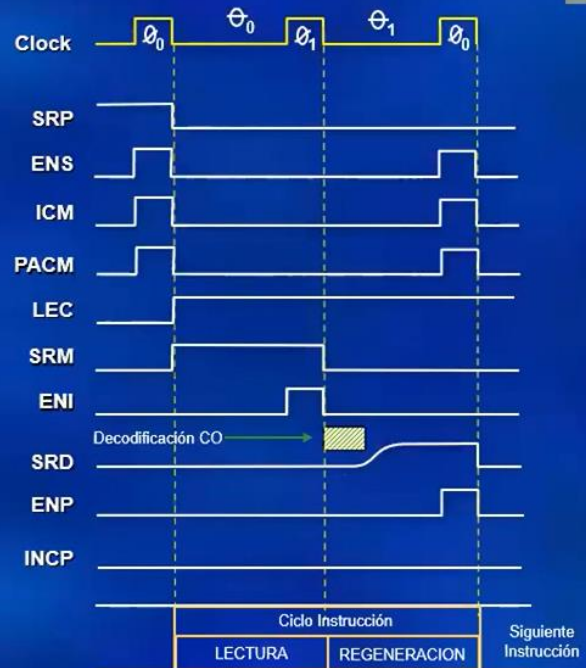
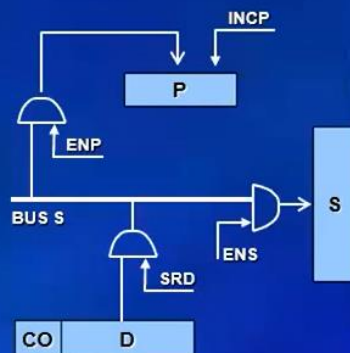
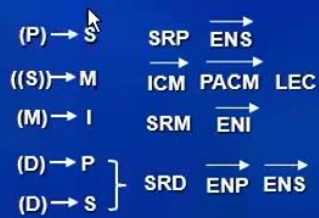
La señal ESC se lanza antes de que ingrese la palabra al registro M ya que se debe poner a 0 el lugar donde se va a insertar la información. Esto debido a las características de las memorias de puntos magnéticos.

## Salto Incondicional en ABACUS

La información de la parte de la Dirección del registro de Instrucción, sobrescribe el valor que lleva el registro Contador de Programa, de manera tal que la siguiente instrucción sea la indicada por la instrucción en curso.



## Salto Incondicional en ABACUS



## Organización de los Registros de la Ruta de Datos

Plano Funcional

### ORGANIZACIÓN DE LOS REGISTROS DE LA RUTA DE DATOS

#### Desde el Plano Funcional

**Registros no Direccionables:** Función bien definida y exclusiva. Registro Inst., Acumulador, y Multiplicador cociente, Contador Programa, Registro M, Reg. Contador de desplazamientos. La mayoría transparentes al programador

**Registros Direccionables:** Usables por el programador. Liberan a la memoria central de esta tarea. Se los conoce como Registros Banalizados ya que algunas arquitecturas usan indistintamente los definidos para almacenar valores aritméticos de los de Direcccionamiento.

Plano Funcional se refiere a como los registros dan soporte a la arquitectura desde el punto de vista de su funcionamiento.

## ORGANIZACIÓN DE LOS REGISTROS DE LA RUTA DE DATOS

### Desde el Plano Estructural

**Montados como Registros Independientes:** Su dirección debe ser decodificada para seleccionarlos. La ventaja es la posibilidad de operar con ellos simultáneamente, pero el costo y cantidad de puntos de entrada y salida encarecen su costo de implementación.

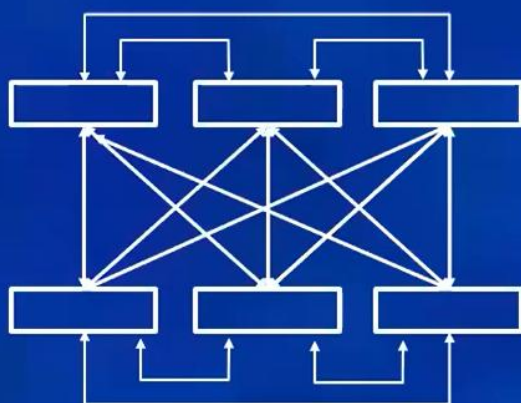
**Montados como Memoria:** Memorias locales dedicadas a decodificar y operar sus palabras como registros independientes. Como ventaja menos puntos de entrada y salida, mas económicas, un solo circuito integrado, pero la desventaja radica en la imposibilidad de operar con varios simultáneamente.

La diferencia entre Reg Independientes y Memorias Dedicadas es que al 1ero podemos acceder de manera directa y alterar su contenido o leerlo. Se puede operar en paralelo, pero son muy costoso al momento de implementación por los puntos de entrada y salida.

Los Montados como Memoria son una pequeña memoria de alta velocidad y baja capacidad. Operan como si fueran registros, no hace falta una gran cantidad de cables, pero no se pueden acceder de manera simultánea a más de uno.

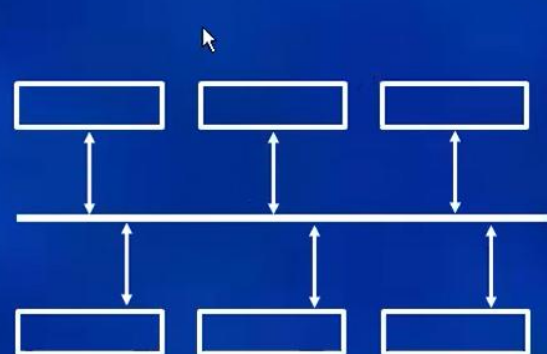
## ORGANIZACIÓN DE LOS REGISTROS DE LA RUTA DE DATOS

### Tipos de Interconexiones



Interconexión directa de 6 registros  
2 a 2

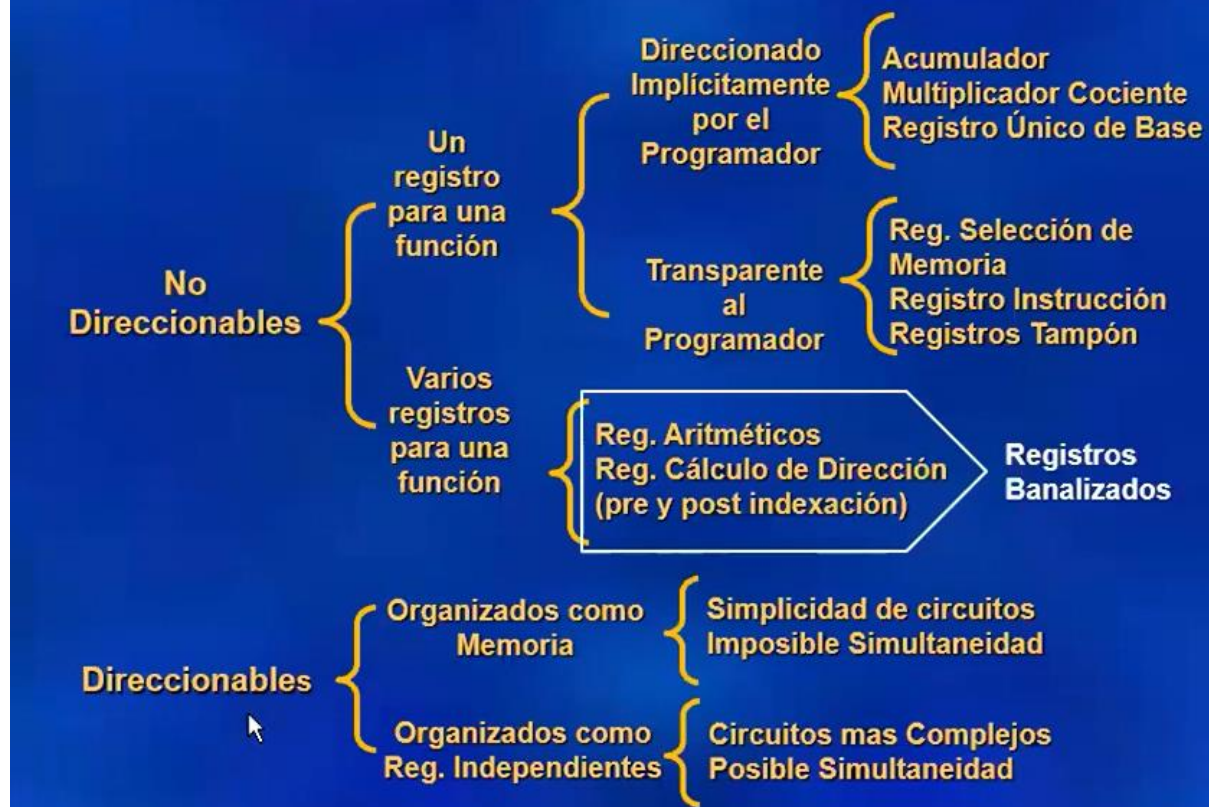
Complejas y caras pero muy  
eficientes y veloces



Direccionamiento por Bus de 6  
Registros

Arquitectura económica pero no  
permite transferencias simultaneas

## LOS REGISTROS DE LA RUTA DE DATOS



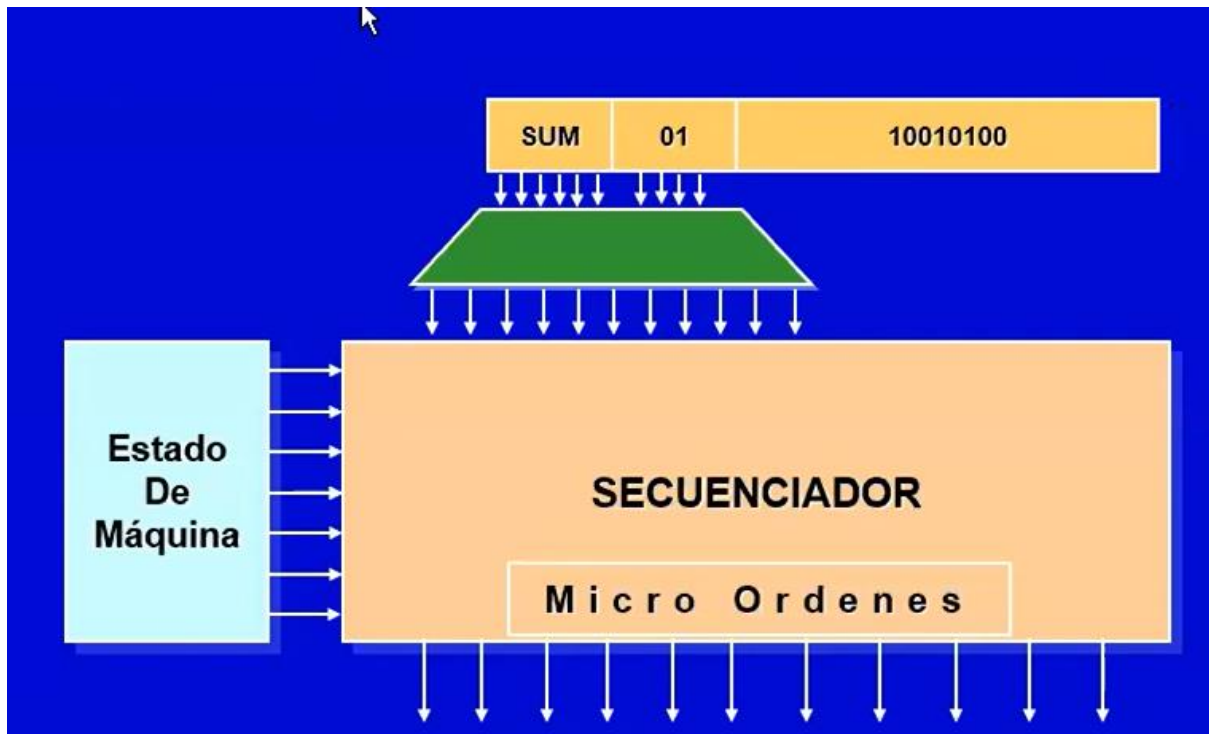
## SECUANCIADOR

### El secuenciador Central:

Es el Organo de un Computador encargado de generar las microordenes necesarias para ejecutar una Instrucción de lenguaje de máquina, distribuyéndolas de manera racional y eficiente a lo largo de la ruta de datos de dicha Arquitectura.

Racional: El exceso de microordenes genera alto consumo de ancho de banda de los buses

Eficiente: El exceso de microordenes para una misma acción consume mayor cantidad de ciclos de máquina de forma innecesaria

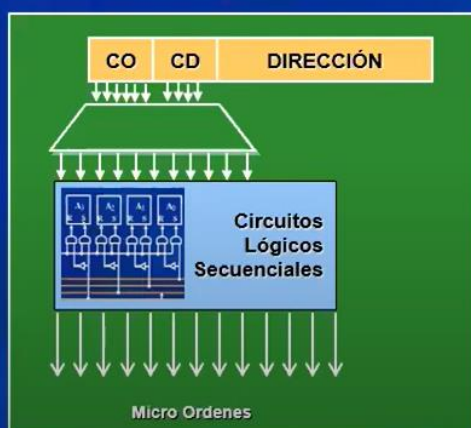


## Modelo Cableado y Microprogramado

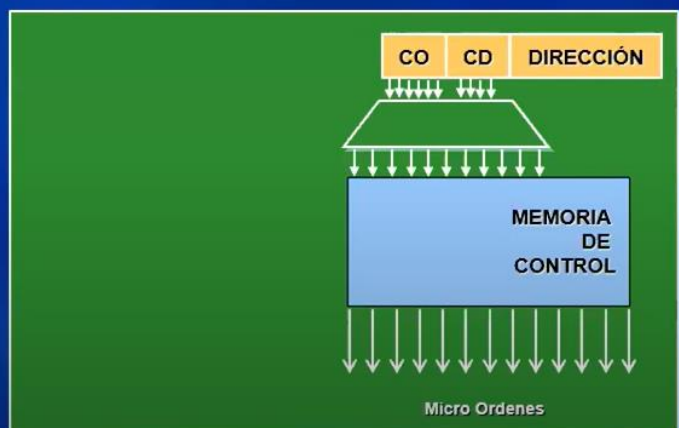
### Modelo Cableado y Microprogramado

Ambos modelos tienen por objeto generar el conjunto de microordenes (señales impulsionales y de nivel) y distribuir las a lo largo de la ruta de datos, con el fin de gobernar los distintos órganos y componentes y realizar la instrucción en curso, basándose en el conjunto de señales de gobierno diseñadas oportunamente en el cronograma de cada operación.

La diferencia se basa en que el modelo cableado posee un conjunto de circuitos lógicos encargados de generar dichas microordenes por cada operación, mientras que el modelo micro programado posee una memoria especial de control que almacena para cada operación, el conjunto de microordenes de gobiernos en el formato ceros y unos.



Modelo Cableado



Modelo Microprogramado

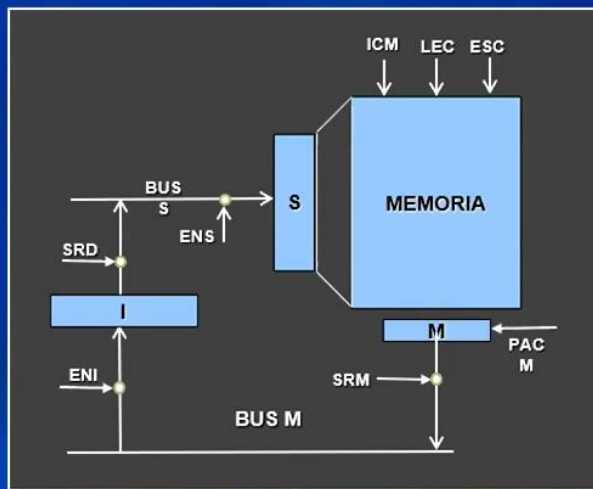
El modelo Cableado permite mayor velocidad, pero es muy costoso debido a la cantidad de circuitos que involucra, y es muy complejo solucionar una falla (esto implica conocer bien la arquitectura del ordenador).

El modelo Microprogramado es más económico, pero es más lento. Sin embargo, es mas sencillo corregir errores, y no hace falta conocer la arquitectura por completo.

## Secuenciador Cableado Sincrónico

### Secuenciador Cableado (Síncrono)

No es posible en el marco de esta materia profundizar sobre la problemática y los aspectos detallados de este modelo. Por tal motivo nos limitaremos a un acercamiento intuitivo al problema tomando de referencia y como ejemplo el Secuenciamiento en la arquitectura ABACUS de Meinadier.



#### Principio del Secuenciamiento:

El objetivo del secuenciador es generar Microordenes distanciadas en el tiempo por retardos apropiados. Una cadena de retardos permite, en teoria realizar esta función.

Tomemos como ejemplo el inicio de la instrucción ALM en Abacus de Meinadier

Se generan instrucciones en el tiempo, se utilizan retardos para determinar el tiempo de vida o tiempo en el cual la microorden esta activa.

### Secuenciador Cableado (Sincrónico)

Realizados en forma de Circuito Secuencial electrónico

Su objetivo es generar microordenes distanciadas en el tiempo por retardos adecuados. Una cadena de retardos permite en teoría, realizar esta función. **Las señales de NIVEL permanecen activas hasta su anulación.**

Por ejemplo, para la Instrucción ALM, la cadena de retardos SINCRONO sería:

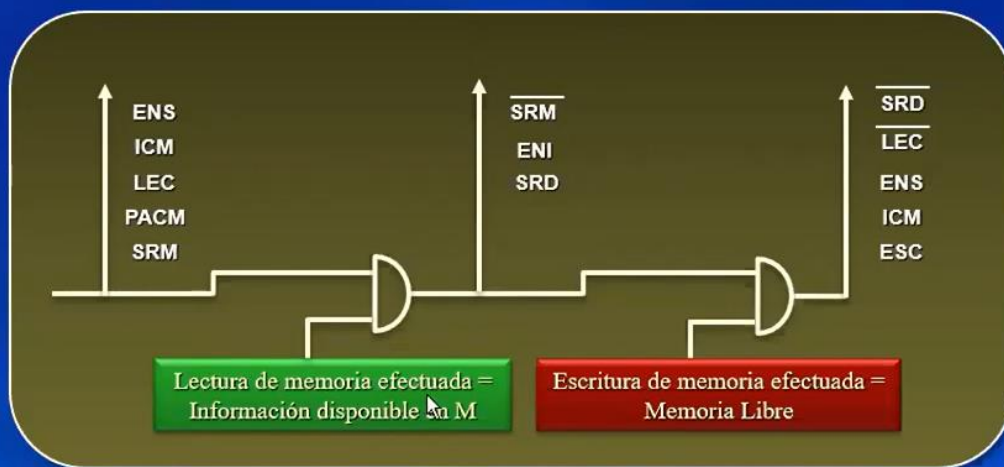


La negación de una instrucción es justamente “apagar” la señal.

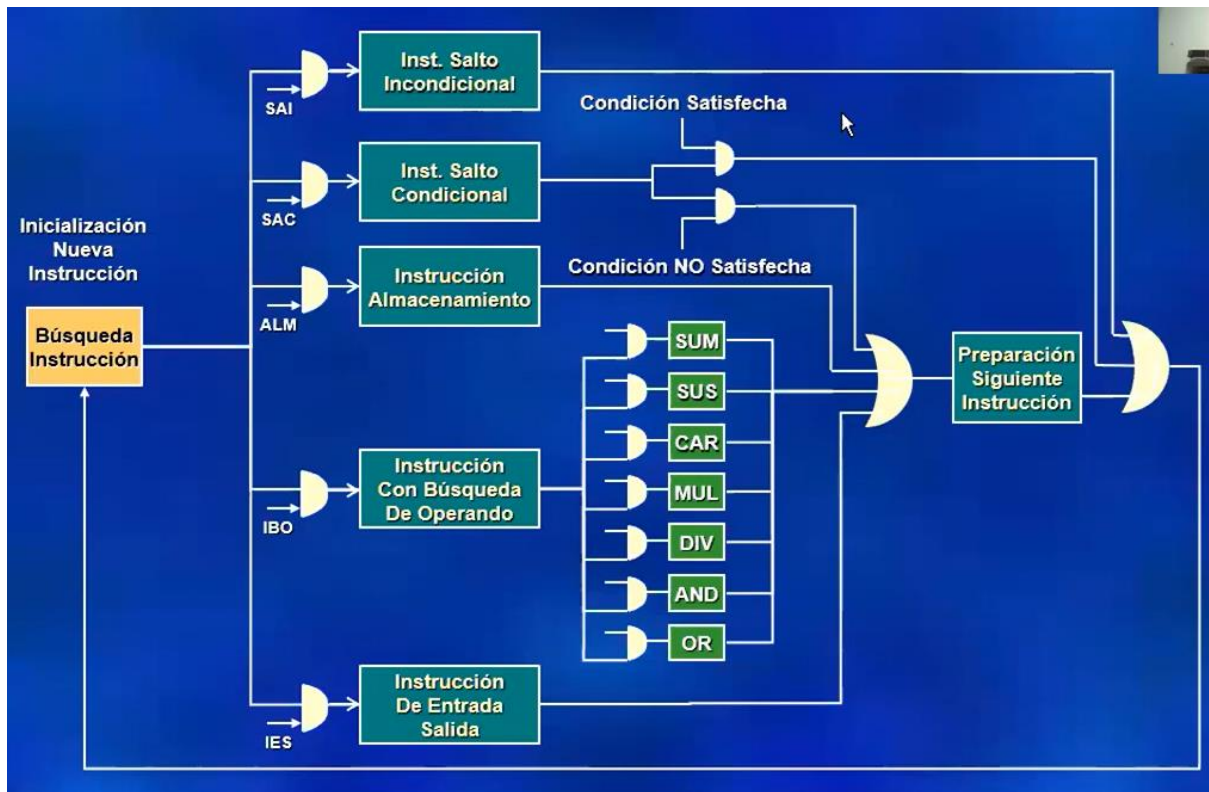
### Secuenciador Cableado Asincrónico

## Secuenciador Cableado (Asincrono)

Si la Memoria posee un funcionamiento Asincrónico respecto al Ordenador, tendremos el esquema de la figura donde las compuertas and simbólicamente representan que solo se pasan a la siguientes microoperaciones siempre que el conjunto de las anteriores hayan finalizado

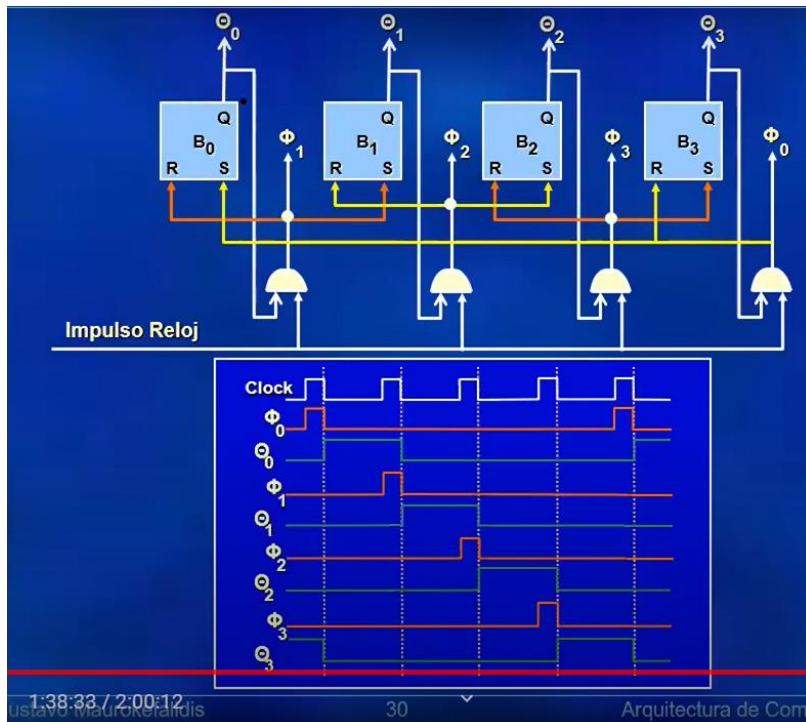


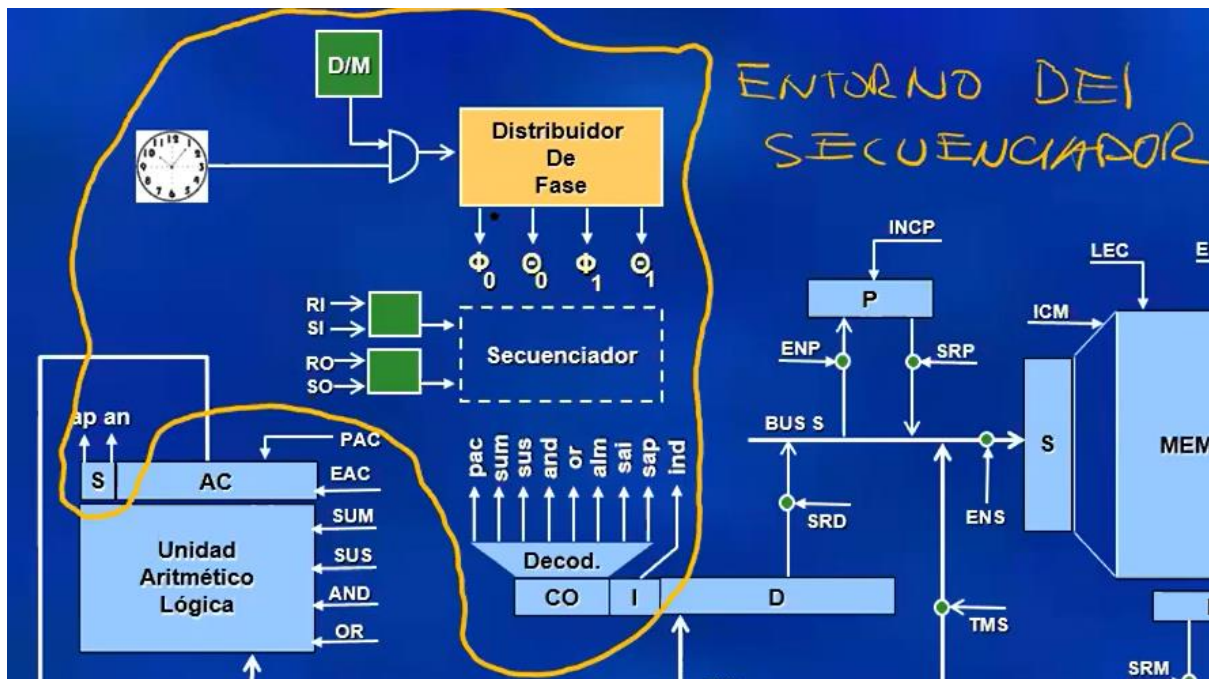
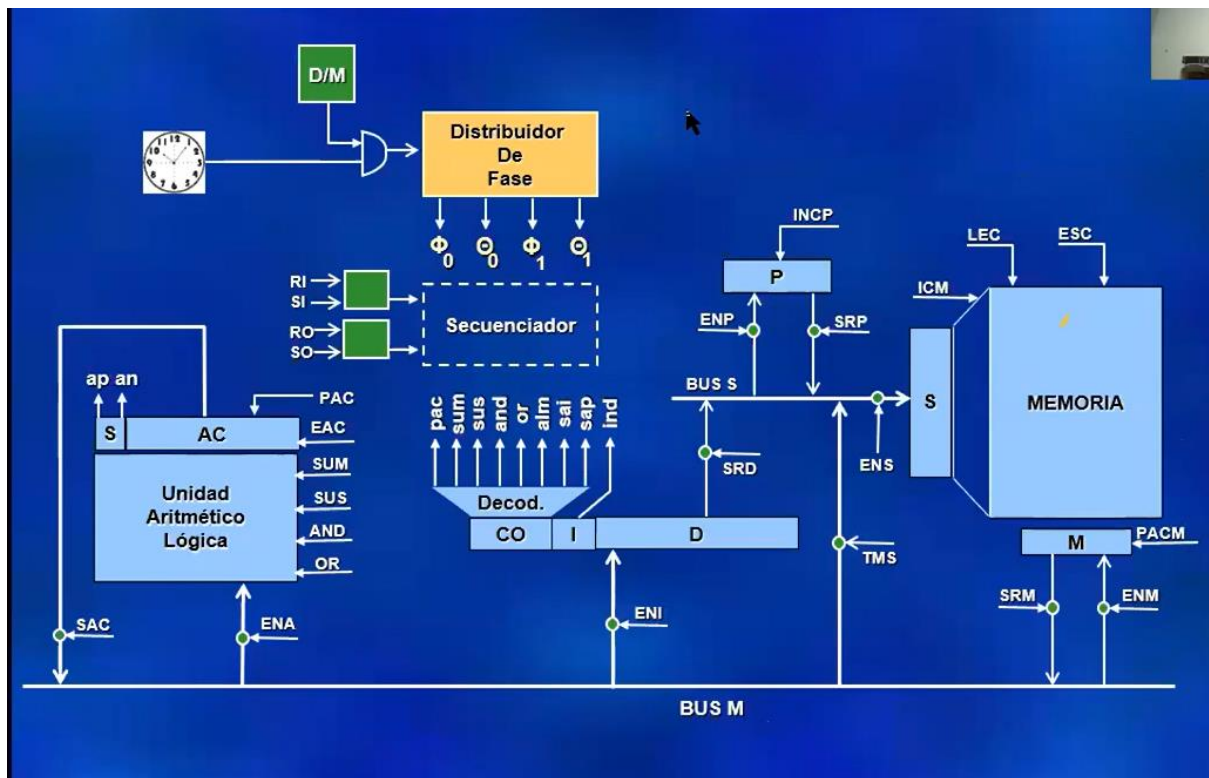
En este modelo se utiliza lógica combinacional para poder pasar a una siguiente etapa.



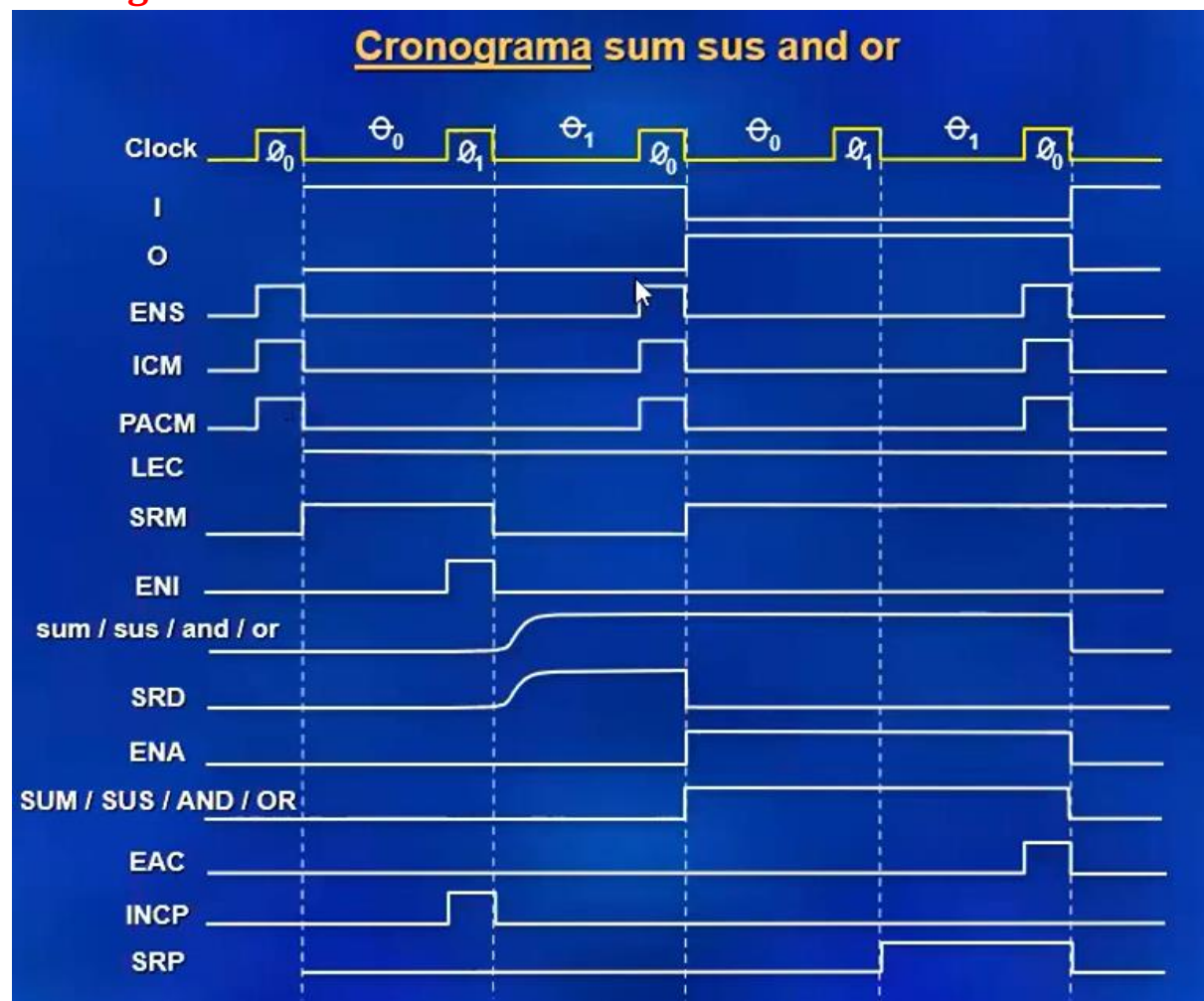
LO DE ARRIBA PUEDE ENTRAR EN FINAL

## Distribuidor de Fase

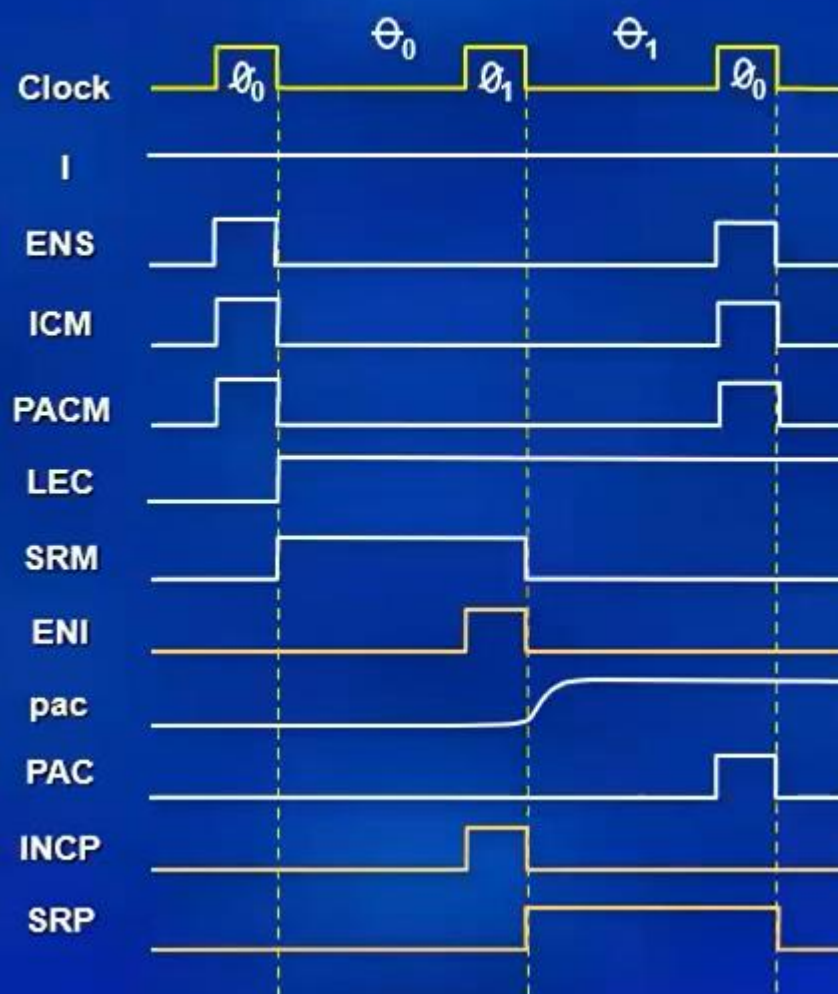




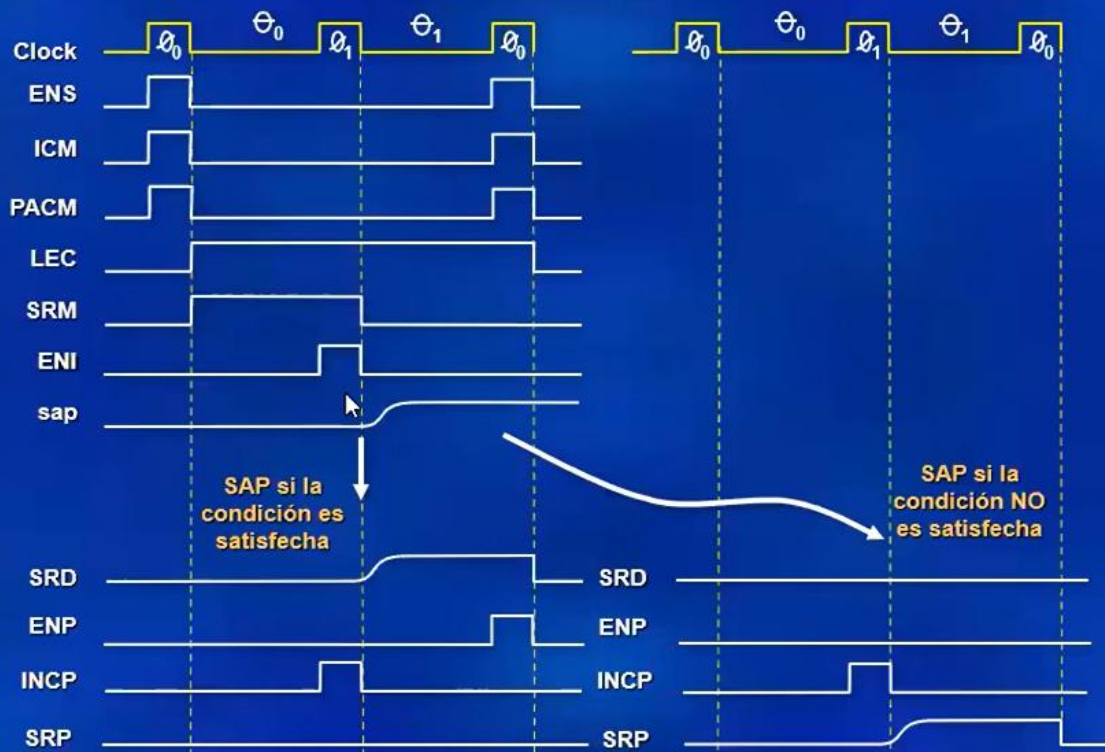
## Cronogramas



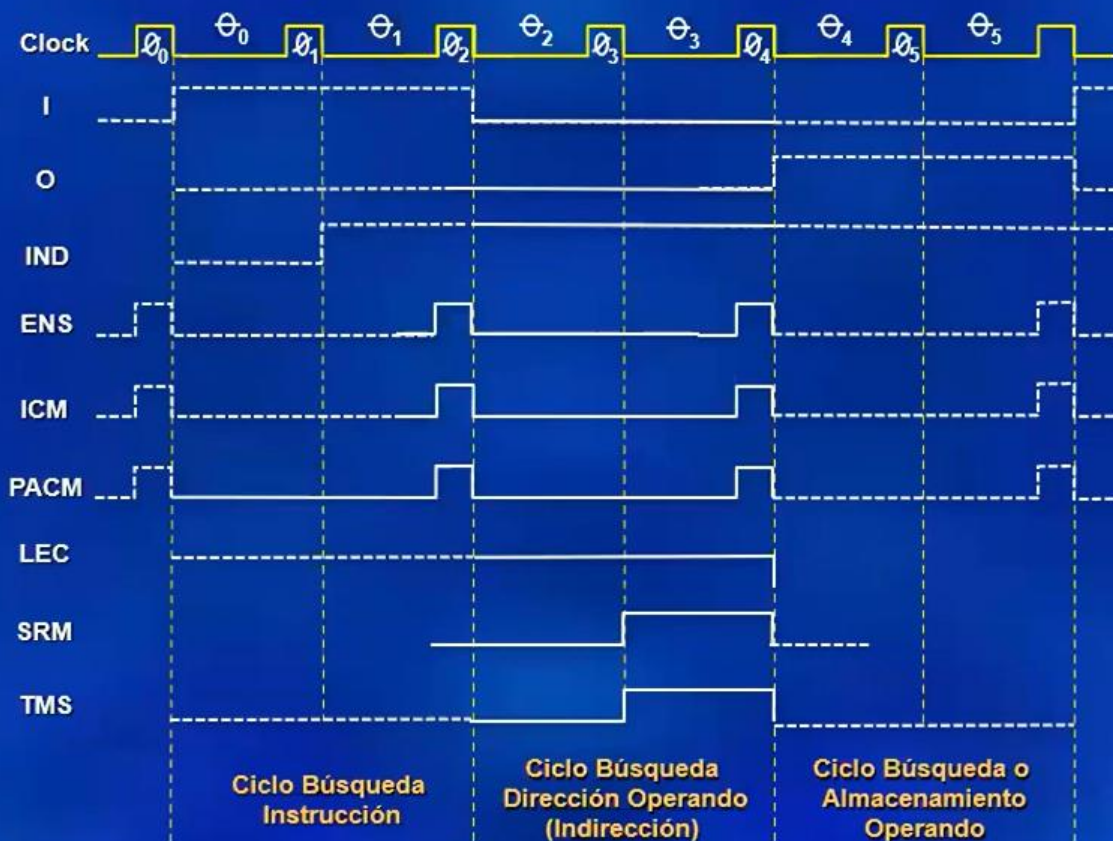
## Cronograma Puesta a Cero del Acumulador



### Cronograma Salto Incondicional y Condicional si Signo AC es +



### Cronograma Instrucción con ciclo de búsqueda del operando (direccionamiento Indirecto)



## Indirección

Consiste en activar TMS, para evitar que se siga con el “camino normal” luego de la lectura del operando. Lo que hace es derivar esa dirección de memoria al Bus S, realizando la indirección, para buscar el operando en la dirección que se obtuvo.

## Operaciones Lógicas

Esto solo es aplicable a arquitecturas RISC, ya que en las CISC sería en extremo complejo realizar las ecuaciones lógicas de cada señal.

$$ENS = \phi_0 \quad ICM = \phi_0 \quad PACM = \phi_0$$

Deben activarse siempre en  $\phi_0$ ,  $\phi_2$ , y  $\phi_4$ , pero debido a que ABACUS trabaja con un Distribuidor de Fase de dos Fases, se deduce que deben estar activos en el instante  $\phi_0$  de TODOS los cronogramas, tanto para la fase de Instrucción como de Operando

$$ENI = \phi_1 \cdot I \quad INCP = \phi_1 \cdot I$$

Son señales de tipo impulsional que solo deben generarse en el ciclo de Instrucción, para la etapa  $\theta_1$

$$SUM = sum \cdot O \quad SUS = sus \cdot O \quad AND = and \cdot O \quad OR = or \cdot O$$

Son señales de Nivel que deben sostenerse a lo largo de  $\theta_0 + \theta_1$  y para cada operación en particular, dentro del ciclo del Operando. La expresión  $\theta_0 + \theta_1$  se elimina ya que la operación dura las dos fases completas

$$LEC = I + O \cdot \overline{alm} + (\bar{O} \cdot \bar{I})$$

La señal de lectura es de nivel y debe durar todo el espacio de tiempo entre ciclo de Instrucción y de Operando (menos para almacenamiento) o bien fuera de los ciclos de Operando e Instrucción

$$SRM = \theta_0 \cdot I + O \cdot \overline{alm} + IND \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O})$$

La salida del Registro M de Nivel, debe ser activada :

- 1) En la búsqueda de instrucción para  $\theta_0$ , es decir  $(\theta_0 \cdot I)$
- 2) En el ciclo de operando para  $sum$ ,  $sus$ ,  $or$ ,  $and$  y durante el tiempo que le lleve a la ALU realizarlo  $O \cdot (\theta_0 + \theta_1) \cdot (sum + sus + or + and)$ . Debido a que  $(\theta_0 + \theta_1)$  cubre todo el espacio de tiempo que dura  $O$ , es inútil expresarlo. A demás debido a que  $sum$ ,  $sus$ ,  $or$ , y  $and$  requieren de un ciclo completo de Operando (con excepción de  $alm$ ), podemos expresarlo de manera más simple como  $O \cdot \overline{alm}$
- 3) En el caso del Direccionamiento Indirecto, en el ciclo de búsqueda de la DIRECCIÓN y durante todo el ciclo  $\theta_1$
- 4) Para Direccionamiento Indirecto, es necesario contemplar la habilitación de la señal  $IND$  en  $\theta_1$  dentro del “Ciclo De Almacenamiento Del Operando (fuera del ciclo de instrucción y el de operando)”

Son señales de tipo impulsional que solo deben generarse en el ciclo de Instrucción, para la etapa  $\theta_1$

$$SI = \phi_0 \cdot O$$

El propio secuenciador deberá tener la tarea de de posicionar a los Biestables I y O , por tal motivo I debe ser puesto a Uno al final del ciclo de operando

$$RI = \phi_0 \cdot I \cdot (sum + sus + or + and + alm)$$

I debe ser puesto a Cero al final del ciclo de aquellas instrucciones que requieren ciclo de operando, es decir para ciclo de instrucción dentro del espacio  $\phi_0$ , para la suma, resta, and, or y alm, el biestable I deberá ser Reseteado ya que se encuentra en la fase de resolución del Operando

$$SO = \overline{IND} \cdot \phi_0 \cdot I \cdot (sum + sus + and + or + alm) + IND \cdot \phi_0 \cdot \bar{I} \cdot \bar{O}$$

O debe ser activado ya sea despues d eun ciclo de instrucción, cuando es con operando y sin direccionamiento Indirecto, o bien al final del ciclo de búsqueda de la dirección efectiva, en el caso de direccionamiento Indirecto y por fuera del del instrucción y operando.

$$RO = \phi_0 \cdot O$$

O debe ser puesto a cero al final del Ciclo de Operando

$$ENS = \phi_0 \quad ICM = \phi_0 \quad PACM = \phi_0$$

$$LEC = I + O \cdot \overline{alm} + (\bar{O} \cdot \bar{I})$$

$$ESC = O \cdot alm$$

$$SRM = \theta_0 \cdot I + O \cdot \overline{alm} + IND \cdot \theta_1 \cdot (\bar{I} \cdot \bar{O})$$

$$TMS = IND \cdot \bar{I} \cdot \bar{O} \cdot \theta_1$$

$$ENI = \phi_1 \cdot I$$

$$INCP = I \cdot \phi_1$$

$$SRD = I \cdot \theta_1 \cdot (sum + sus + and + or + alm + sap \cdot AP + sai)$$

$$SRP = \theta_1 \cdot I \cdot (pac + sac \cdot AN) + \theta_1 \cdot O$$

$$ENP = \phi_0 \cdot (sai + sap \cdot AP)$$

$$SUM = sum \cdot O \quad SUS = sus \cdot O \quad AND = and \cdot O \quad OR = or \cdot O$$

$$ENA = O \cdot \overline{alm}$$

$$EAC = O \cdot \phi_0 \cdot \overline{alm}$$

$$PAC = pac \cdot \phi_0$$

$$SAC = \theta_0 \cdot O \cdot alm$$

$$ENM = \phi_1 \cdot O \cdot alm$$

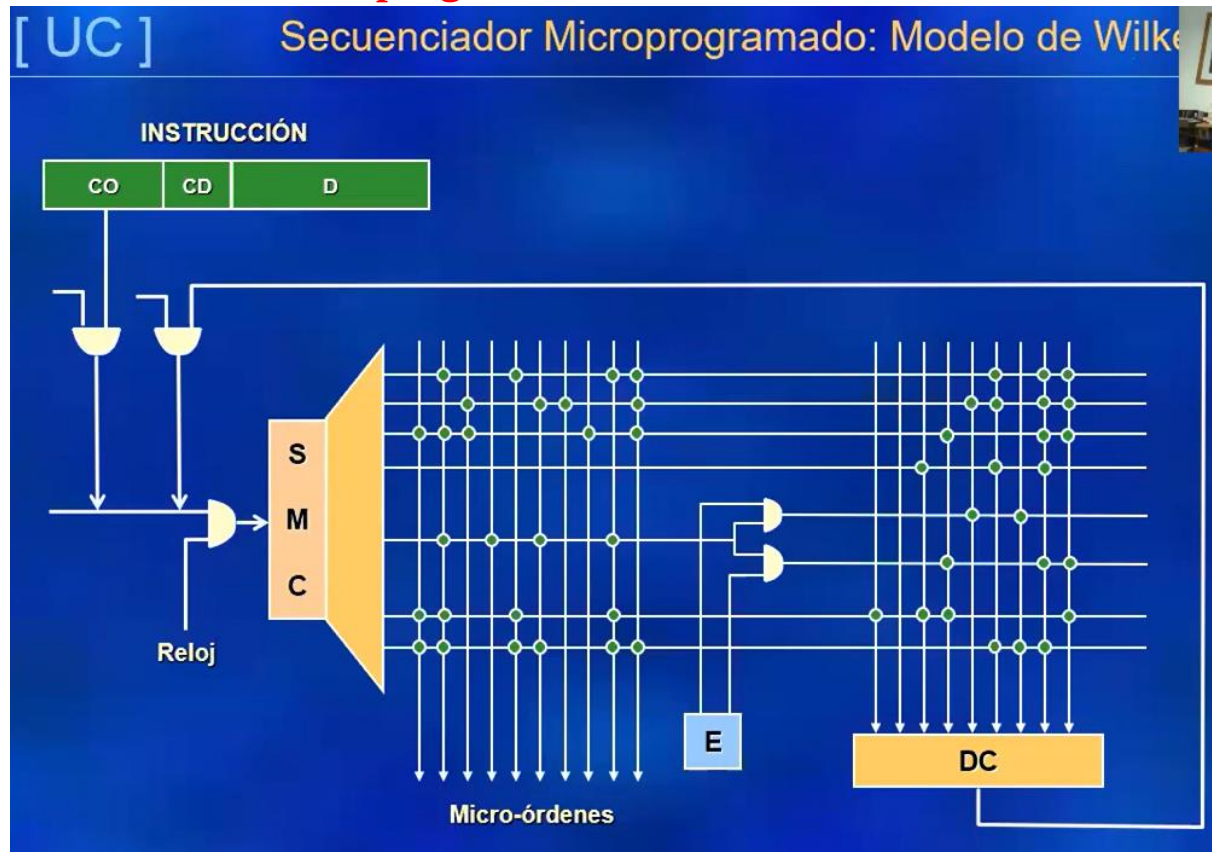
$$SI = \phi_0 \cdot O$$

$$RI = \phi_0 \cdot I \cdot (sum + sus + or + and + alm)$$

$$SO = \overline{IND} \cdot \phi_0 \cdot I \cdot (sum + sus + and + or + alm) + IND \cdot \phi_0 \cdot \bar{I} \cdot \bar{O}$$

$$RO = \phi_0 \cdot O$$

## Secuenciador Microprogramado: WILKES



Aparece el concepto de microprograma, que es un conjunto de microinstrucciones, que son una palabra dentro de la unidad de control.

Tiene una parte cableada, referida a los saltos.

Las ventajas que otorga este modelo son la reducción de componentes en el secuenciador. También la flexibilidad de grabación.

Pero la desventaja es que es bastante más lento. Involucra un acompasamiento. También se incrementa mucho el tamaño de la memoria, al tener que utilizar gran cantidad de bits para representar todas las señales y señales siguientes (es ineficiente).

Es por esto que se plantea codificar parte de las instrucciones en pos de reducir el tamaño de la memoria. (ABAJO)

## Características de la Codificación

- ☐ *En la práctica las Unidades de Control Microprogramadas no se diseñan usando un formato de instrucción puro No Codificado u Horizontal.*
- ☐ *Se hace uso al menos, de algún grado de codificación para reducir el ancho de la memoria de control y simplificar las tareas de la microprogramación.*

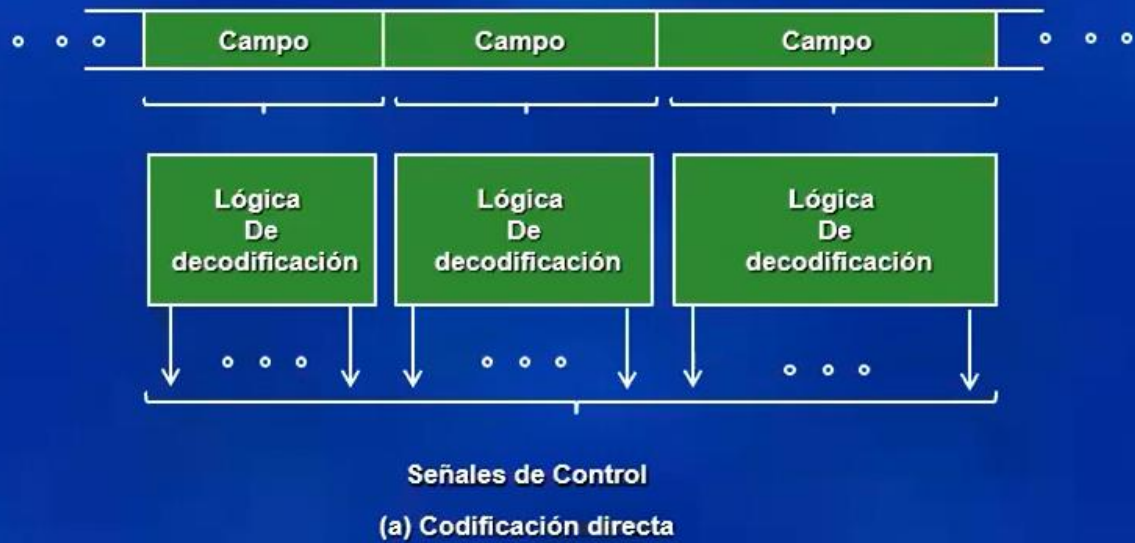
| Características                  |                                 |
|----------------------------------|---------------------------------|
| Microinstrucción no codificada   | Microinstrucción muy codificada |
| Muchos Bits                      | Pocos Bits                      |
| Visión detallada del Hardware    | Visión global del Hardware      |
| Difícil de Programar             | Fácil de Programar              |
| Poca o ninguna lógica de control | Lógica de control compleja      |
| Ejecución Rápida                 | Ejecución Lenta                 |
| Optimización de las Prestaciones | Optimización de la Programación |
| Tecnología                       |                                 |
| Horizontal                       | Vertical                        |
| Desempaquetada                   | Empaquetada                     |
| "Hard"                           | "Soft"                          |

## Codificación por campo

### Codificación Por Campo

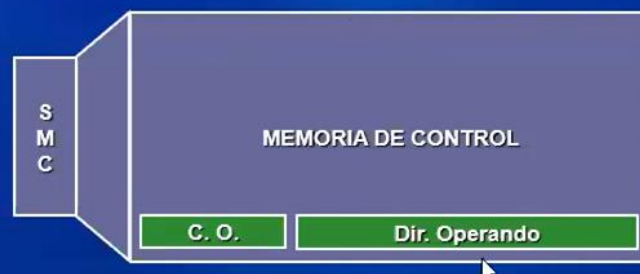
- ☐ *Cuando la MicroInstrucción se ejecuta, cada campo se decodifica y genera señales de control.*
- ☐ *Con N Campos se definen N acciones simultaneas.*
- ☐ *Cada acción se traduce en la activación de una o mas señales de control.*
- ☐ *Se debe cuidar que cada señal de control no pueda ser activada por mas de un campo (no siempre se puede).*
- ☐ *Debe ser posible activar cada señal de control al menos por un campo*
- ☐ *Un campo individual podrá constar de L bits o sea  $2^L$  códigos posibles donde cada uno deba codificar un patrón distinto de señales de control*

## Codificación Por Campo



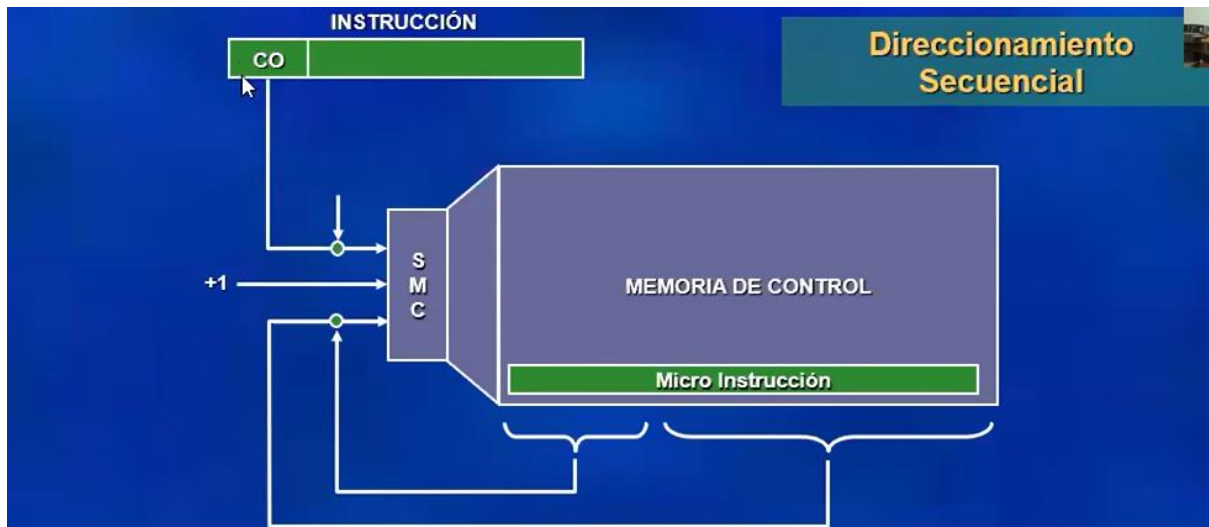
## Codificación por Tipo de Instrucción

### Codificación Tipo de Instrucción



- ☐ *Brinda a la MicroInstrucción un formato semejante al de una instrucción.*
- ☐ *Consta de código de operación y de dirección del operando.*
- ☐ *La decodificación de las MicroInstrucciones para obtener las correspondientes Microordenes es compleja.*
- ☐ *Se compensa la complejidad con la pequeña longitud de las MicroInstrucciones.*
- ☐ *Pone la MicroProgramación al alcance del programador de lenguaje de máquina.*

## Métodos de direccionamiento



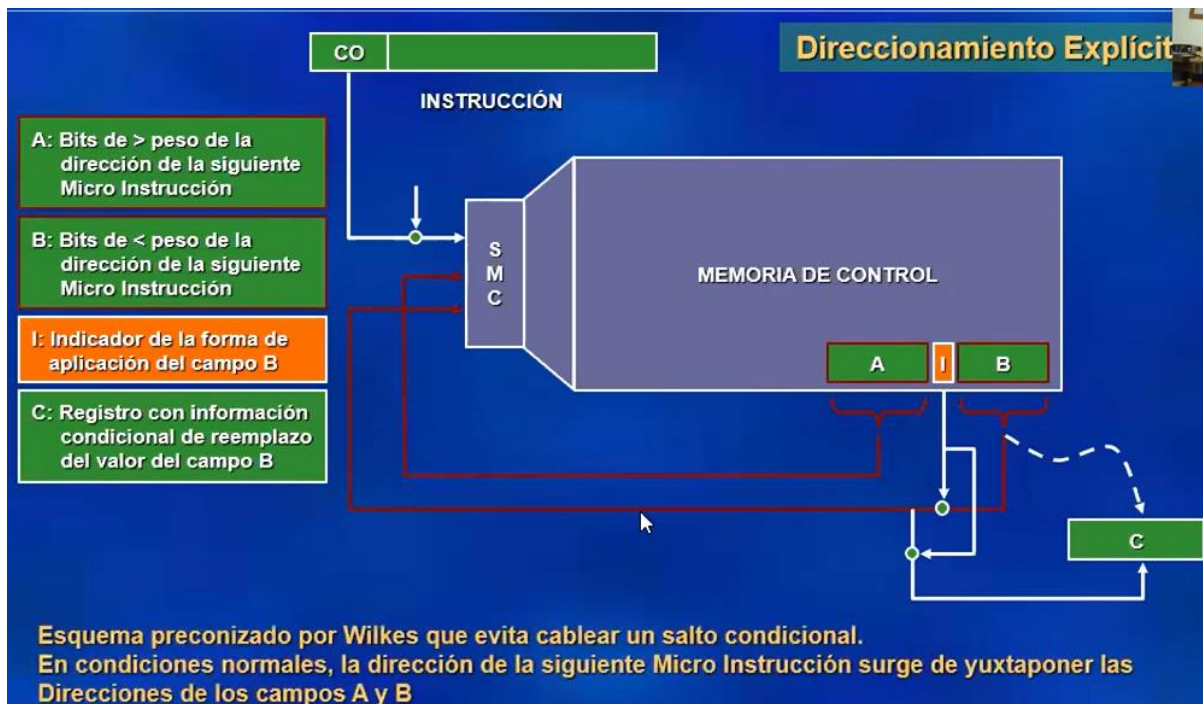
En condiciones normales la dirección de la próxima Micro Instrucción, la calcula el SMC con Capacidad de incrementar la última dirección de micro instrucción y de manera secuencial



Para el caso de un salto, se especializa un bit de la Micro Inst. como indicador V o F dejando El resto de la palabra de control para indicar la dirección de la próxima micro instrucción

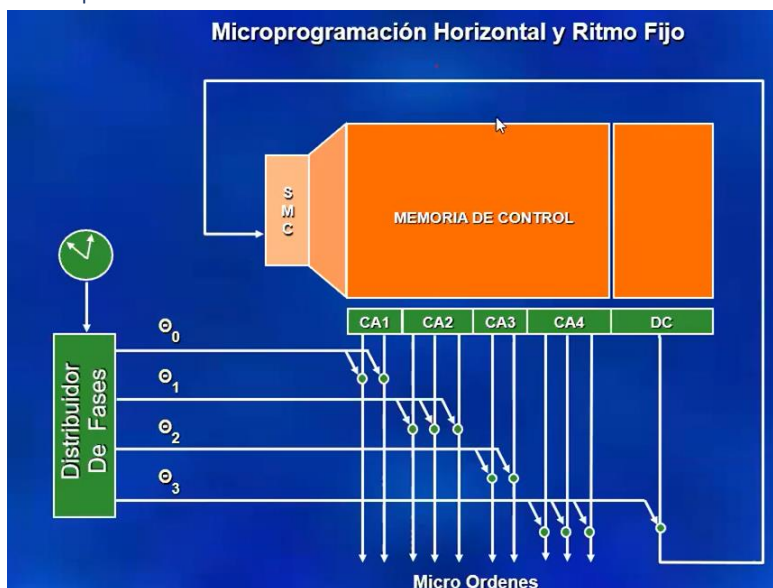
**Ventaja:** Simple de programar y no alarga el tamaño de la microinstrucción

**Desventaja:** Un salto ocupa una micro instrucción por si solo desperdiciando tiempo. Se lo usa cuando la concepción de la micro programación de la máquina no requiera muchos saltos.



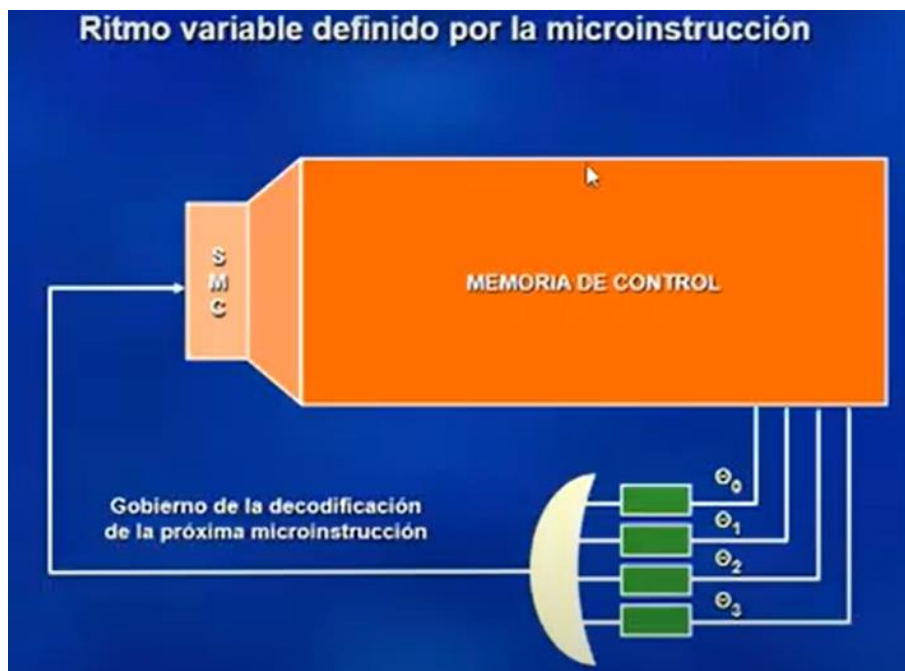
En un salto condicional, Si  $I=0$  el campo B podrá contener los bits de menor peso de la dirección de salto si esta fuese conocida. Si  $I=1$  B contiene la dirección de un registro C que almacena los bits de menor peso de la dirección de salto.

## Acompasamiento



Acompasamiento gobernado por un distribuidor de fase. Permite que salgan los valores de las micro órdenes.

DC es la dirección de la siguiente microinstrucción, y se ejecuta una vez han terminado de salir todas las micro órdenes.



Orientado a codificación por tipo de instrucción.

Utiliza retardos para determinar cuando se procede con la siguiente microinstrucción.

Este modelo depende del tiempo que le lleva a cada instrucción.

## CISC y RISC

| Resumen                      |                                |
|------------------------------|--------------------------------|
| CISC                         | RISC                           |
| Instrucciones más complejas  | Instrucciones más simples      |
| Instrucciones más grandes    | Instrucciones más pequeñas     |
| Ciclos de reloj más extensos | Ciclos de reloj reducidos      |
| Difícil de modificar         | Podrían realizarse cambios     |
| Tiempo de ejecución mayor    | Tiempo de ejecución más rápido |
| Más lento                    | Más rápido                     |

### CISC

#### Computador con Set de Instrucciones Complejas CISC

(Complex instruction set computing)

- ☐ La arquitectura CISC se caracteriza por tener un repertorio de instrucciones complejas
- ☐ Muchas instrucciones potentes para lograr una implementación mas directa con Lenguajes de Alto Nivel.
- ☐ La ejecución de estas instrucciones puede ser bastante complejas
- ☐ Considera que la compilación de programas de alto nivel requiere muchas menos instrucciones de máquina y con tiempos mas cortos.
- ☐ En la práctica lograr elevadas velocidades de ejecución se torna difícil, requiere chips con superficie elevada de silicio, y el repertorio de instrucciones dificultan la generación de códigos óptimos por parte de los Compiladores.

### Criterios para definir una Arquitectura CISC

- ☐ Cantidad de instrucciones en lenguaje máquina (tan grande como sea posible)
- ☐ Cantidad de modos de direccionamiento (tan grande como sea posible).
- ☐ Cantidad de formatos de instrucción (tan grande como sea posible).
- ☐ Muchas instrucciones requieren más de un ciclo para su ejecución.
- ☐ Varios tipos de instrucciones tienen acceso a memoria.
- ☐ Existencia de registros de propósito específico.
- ☐ Control microprogramado.
- ☐ Instrucciones de máquina de un relativo alto-nivel (cercano al alto nivel de las sentencias de los lenguajes de alto nivel).
- ☐ Las instrucciones compuestas son decodificadas internamente y ejecutadas con una serie de microinstrucciones almacenadas en una ROM interna.
- ☐ La microprogramación es una característica importante y esencial de casi todas las arquitecturas CISC. significa que cada instrucción de máquina es interpretada por un microprograma localizado en una memoria.

### RISC

#### Computador con Set de Instrucciones Reducidas RISC

(Reduced instruction set computing)

- ☐ La arquitectura RISC se caracteriza por tener un repertorio de instrucciones simples.
- ☐ Se necesitan muchas mas instrucciones para realizar la misma tarea que una instrucción CISC.
- ☐ A pesar de la cantidad necesaria, son muy convenientes para una ejecución en un cauce Segmentado, logrando velocidades elevadas de ejecución.
- ☐ Su repertorio es utilizado convenientemente por compiladores optimizadores.
- ☐ Ventaja relacionada con la tecnología de fabricación VLSI (Very Long Scale Integration) ya que necesitan menos área para control y procesamiento de secuencia de instrucciones (mayor espacio para registros y cache).
- ☐ Reducción de acceso a datos e instrucciones por fuera del chip logran mejores prestaciones.
- ☐ Unidad de Control Cableada debido a la sencillez de la Arquitectura, e implementación de operaciones por hardware.

Una de las cosas notables de la arquitectura RISC es que permiten el paralelismo, ya que al ser pocas instrucciones y muy reducidas, se use poco tiempo el ciclo del CPU. Esto da lugar a que se puedan realizar otras operaciones mientras tanto. Este paralelismo también se denomina **pipeline**.

### Criterios para definir una Arquitectura RISC

- ☐ Analizar el objeto sobre el cual se desarrollan las aplicaciones a fin de determinar cuáles son las operaciones más frecuentes.
- ☐ Optimizar los caminos que deben recorrer los datos para ejecutar las operaciones o instrucciones (anteriores) tan rápido como sea posible.
- ☐ Incluir otras instrucciones solo si forman parte de los caminos optimizados previamente y si son de relativa frecuencia, y si su inclusión no hace más lenta la ejecución de las instrucciones más frecuentes.
- ☐ Aplicar una estrategia similar a los otros recursos del procesador. Incluir un recurso solo si éste está justificado por su frecuencia de uso, y su inclusión no enlentece a otros recursos más utilizados.
- ☐ Tratar de trasladar lo más que se pueda la complejidad en tiempos de ejecución al momento de compilación, recargando el software de compilación y liberando al hardware de ejecución.

### Características comunes en una Arquitectura RISC

- ☐ Muchas de las instrucciones se ejecutan en un solo ciclo de máquina.
- ☐ Conjunto de instrucciones de cargar/almacenar (load/store). Es decir, se accede a memoria exclusivamente mediante las instrucciones load y store, el resto de las instrucciones realizan sus operaciones entre registros.
- ☐ Decodificación de instrucciones hardwired (en oposición a la técnica de microprograma).
- ☐ Existen relativamente pocas instrucciones y modos de direccionamiento
- ☐ Todas las instrucciones deberían tener la misma longitud para facilitar la tarea de decodificación y homogeneizar los tiempos de carga de las mismas.

Un punto crucial a destacar es que, en la arquitectura RISC, hay gran interconexión de registros para evitar justamente las instrucciones load/store (para acceder a la memoria RAM).

### Características comunes en una Arquitectura RISC

- ☐ La complejidad se ha desplazado hacia los compiladores optimizados.
- ☐ Existe un alto grado de pipeline en los caminos de los datos para obtener mucha concurrencia.
- ☐ Gran cantidad de registros (windowed o no-windowed).
- ☐ Muchos niveles de jerarquías de memoria.
- ☐ Conjunto de instrucciones diseñado para determinada clase de aplicaciones.
- ☐ Soporte de lenguajes de alto nivel (Esto último debido a que como hay pocas instrucciones sobraría lugar en el chip para colocar elementos que ayuden a lenguajes de alto nivel, como manejo de listas, stacks, etc.).

### Comparación

| <b>CISC</b>                                                 | <b>RISC</b>                                                   |
|-------------------------------------------------------------|---------------------------------------------------------------|
| Énfasis en el Hardware (Velocidad)                          | Énfasis en el Software (Sencillez y rapidez en el Pipeline)   |
| Incluye instrucciones complejas multi-reloj                 | Instrucciones con reloj simple                                |
| Instrucciones complejas (Más de 1000)                       | Instrucciones sencillas (Menos de 100)                        |
| Instrucciones que trabajan de memoria a memoria             | Instrucciones que trabajan de registro a registro             |
| Carga y almacenamiento incorporados en la misma instrucción | La carga y el almacenamiento son instrucciones independientes |
| El tamaño del código es pequeño                             | El tamaño del código es muy grande                            |
| Muchos ciclos de reloj por segundo                          | Pocos ciclos de reloj por segundo                             |
| Pocos registros para almacenar los resultados               | Muchos registros para almacenar resultados                    |
| Alto costo de producción                                    | Bajo costo de producción                                      |

## LENGUAJE MAQUINA

### Concepto

# Lenguaje de Máquina

### Concepto:

Lenguaje de máquina es el sistema de códigos directamente interpretable por un circuito microprogramado, como el microprocesador de un ordenador o el microcontrolador de un autómeta.

Este lenguaje está compuesto por un conjunto de instrucciones que determinan acciones a ser tomadas por la máquina.

Un programa en Lenguaje de Máquina consiste en una cadena de estas instrucciones (mas datos y operandos).

El lenguaje de máquina es específico de cada máquina o arquitectura, aunque el conjunto de instrucciones disponibles pueda ser similar entre ellas.

El lenguaje de maquina es un sistema de códigos definidos para una arquitectura (interpretable por ella) que permite a través de una instrucción de maquina debidamente codificada, solicitar a los circuitos lógicos intervinientes el desarrollo de un microprograma, entendiendo a este último como el conjunto de señales de gobierno impulsionales y de nivel, necesarias para ejecutar a la instrucción en curso.

## Forma General de las Instrucciones Aritméticas

### Elementos informativos de una Instrucción

- A. *La naturaleza de la Operación por realizar*
- B. *Datos y Operandos implicados en la Instrucción*
- C. *Lugar donde deba almacenarse el resultado*
- D. *La dirección de la Próxima Instrucción por ejecutar*

### Clasificación de Arquitecturas

#### Arquitectura de cuatro direcciones :

|    |    |    |    |     |
|----|----|----|----|-----|
| CO | D1 | D2 | D3 | DPI |
|----|----|----|----|-----|

#### Arquitectura de tres direcciones :

|    |    |    |    |
|----|----|----|----|
| CO | D1 | D2 | DR |
|----|----|----|----|

#### Arquitectura de dos direcciones :

|    |    |    |
|----|----|----|
| CO | D1 | D2 |
|----|----|----|

SUM D1 D2 (D1) + (D2) -> D1

#### Arquitectura de una direccion :

|    |    |
|----|----|
| CO | D2 |
|----|----|

SUM D2 (D2) + (AC') -> AC

# Tipos de Instrucciones

## Instrucciones de Transferencia

|                |      |       |       |                                                          |
|----------------|------|-------|-------|----------------------------------------------------------|
| Carga          | CAR  | R     | M     | $(M) \rightarrow R$                                      |
| Carga Complem. | CARC | R     | M     | $\text{Comp}(M) \rightarrow R$                           |
| Almacenamiento | ALM  | R     | M     | $(R) \rightarrow M$                                      |
| Transferencia  | TRA  | $R_i$ | $R_j$ | $(R_i) \rightarrow R_j$                                  |
| Intercambio    | INT  | $R_i$ | $R_j$ | $(R_i) \rightarrow R_j \text{ y } (R_j) \rightarrow R_i$ |

## Longitud Simple

### Instrucciones Aritméticas en Punto Fijo Binaria

|                |     |   |   |                                     |
|----------------|-----|---|---|-------------------------------------|
| Suma           | SUM | R | M | $(R) + (M) \rightarrow R$           |
| Sustracción    | SUS | R | M | $(R) - (M) \rightarrow R$           |
| Multiplicación | MUL | R | M | $(R) \times (M) \rightarrow R, R+1$ |
| División       | DIV | R | M | $(R, R+1) / (M) \rightarrow R$      |

## Longitud Doble

### Instrucciones Aritméticas en Punto Fijo Binaria

|             |      |   |   |                                          |
|-------------|------|---|---|------------------------------------------|
| Suma        | SUM2 | R | M | $(R, R+1) + (M, M+1) \rightarrow R, R+1$ |
| Sustracción | SUS2 | R | M | $(R, R+1) - (M, M+1) \rightarrow R, R+1$ |

## Longitud Simple

### Instrucciones Aritméticas en Punto Flotante

|               |       |   |   |                            |
|---------------|-------|---|---|----------------------------|
| Suma          | SUMF1 | R | M | $(R) + (M) \rightarrow R$  |
| Normalización | NORM1 | R |   | Normaliza contenido Reg. R |

## Instrucciones Lógicas

|                 |     |   |   |                                      |
|-----------------|-----|---|---|--------------------------------------|
| Complementación | COM | R |   | $\text{Comp}(R) \rightarrow R$       |
| Producto Lógico | AND | R | M | $(R) \text{ AND } (M) \rightarrow R$ |
| Suma Lógica     | OR  | R | M | $(R) \text{ OR } (M) \rightarrow R$  |

### Instrucciones de Desplazamiento

|      |   |   |                                             |
|------|---|---|---------------------------------------------|
| DESD | R | n | Desplazamiento Derecha de R, n Posiciones   |
| DESI | R | n | Desplazamiento Izquierda de R, n Posiciones |

### Instrucciones Aritméticas Decimales Fijas

|                 |      |    |    |                                                                                        |
|-----------------|------|----|----|----------------------------------------------------------------------------------------|
| Suma (paralela) | DSUM | R  | M  | R : N° de caracteres o palabras 2° Operando<br>M : Dirección último carácter o palabra |
| Suma (serie)    | DSUM | M1 | M2 | M1 y M2 : Direcciones de Fin de Operando                                               |

### Instrucciones de Conversión

Permiten convertir valores numéricos de un formato a otro, por ejemplo de Punto Fijo Binario a Decimal

### Instrucciones de Movimiento de Cadenas de Caracteres

Permiten transferir cadenas de caracteres de longitud variable de una a otras posiciones de memoria, comparar dos cadenas de caracteres, etc.

### Instrucciones de Ruptura de Secuencia

|               |     |      |   |                                    |
|---------------|-----|------|---|------------------------------------|
| Incondicional | SAI |      | M | M : Dirección de Salto             |
| Condicional   | SA* | COND | M | M : Dirección de Salto si COND = V |

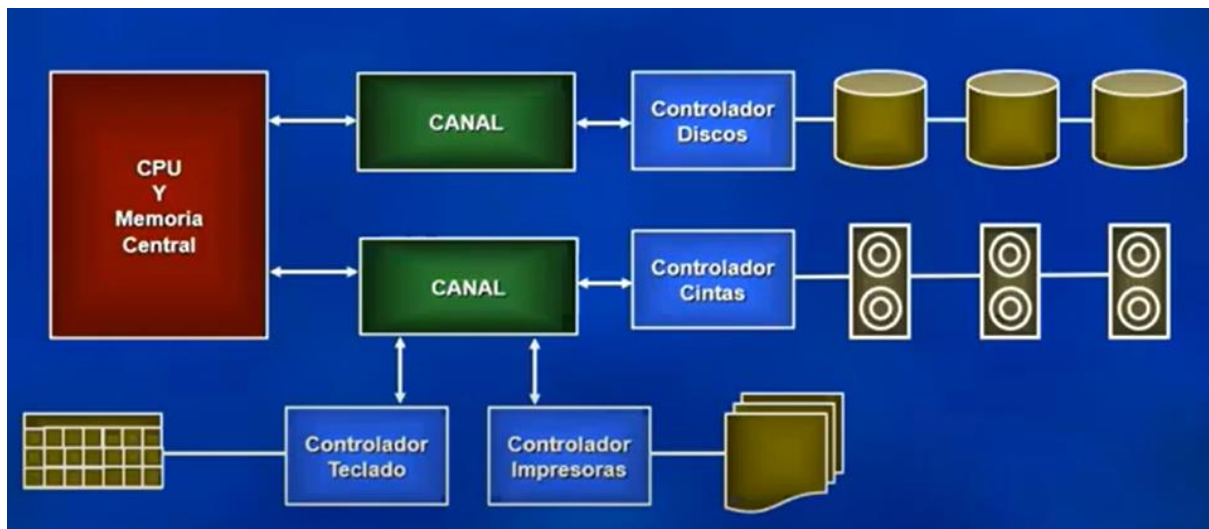
### Instrucciones sobre estructuras Elaboradas de Información

Inserción, Extracción, Concatenación en una Lista o Pila

### Instrucciones de Gobierno y Estado

Instrucciones para el gobierno de Órganos de Entrada / Salida  
Instrucciones para Gobierno del Vector de Interrupciones  
Etc.

## CANALES



### Métodos de Transferencia

#### Modo Bloqueado

##### 1-Modo Bloqueado

**El computador ejecuta el programa de entrada – salida esperando por cada nueva instrucción de transferencia que el elemento periférico esté disponible**

Esto malgasta segundos mientras se realizan las operaciones mecánicas en el periférico. (Se bloquea el procesador)

#### Modo por Prueba de Estado

##### 2-Modo Por Prueba de Estado

**Inclusión de puntos de prueba en el programa con el objeto De consultar el estado del periférico antes de realizar la Transferencia elemental**

Pregunta si el periférico está libre, en caso de ser verdadero, salta a realizar la instrucción correspondiente. En caso de negativo, sigue con el curso normal.

Esto sigue quedando a cargo del programador para analizar cuando era conveniente realizar las consultas. Aunque evita el modo bloqueado.

Pero se generaba tiempo ocioso tanto en el CPU como en el periférico.

Modo por Interrupción de Programa

### **3-Modo Por Interrupción de Programa**

**Son los periféricos los que interrumpen para avisar al computador que se encuentran disponibles.**

**La interrupción detiene momentáneamente el programa en curso  
Para permitir que el programa de transferencia sea ejecutado con prioridad**

Es el periférico quien interrumpe al computador. Esto es mucho mejor que los modos anteriores ya que solucionan gran cantidad de incomodidades y problemáticas.

Modo Canal

### **4-Modo Automático, por Suspensión de Programa o MODO CANAL**

**Son unidades capaces de tomar a su cargo la tarea de realizar  
La transferencia de todo un bloque de información**

**Si el canal necesita realizar una transferencia, ya no genera  
Una interrupción, sinó una solicitud de servicio para que se le  
Conceda un ciclo de memoria**

**La instrucción provee al canal de la siguiente info:**

**A-Número y emplazamiento de información a transferir**

**B-dirección del periférico**

**C-condición de finalización de la transferencia**

**La transferencia se realiza al ritmo del periférico**

**El canal roba ciclos al procesador**

**El canal avisa la finalización**

La instrucción tiene el formato:

|         |                         |                            |                         |                  |
|---------|-------------------------|----------------------------|-------------------------|------------------|
| Sentido | Dirección<br>Periférico | Dirección de la 1° palabra | Cantidad de<br>palabras | Condición de Fin |
|---------|-------------------------|----------------------------|-------------------------|------------------|

Es el canal quien maneja las transferencias en este caso.

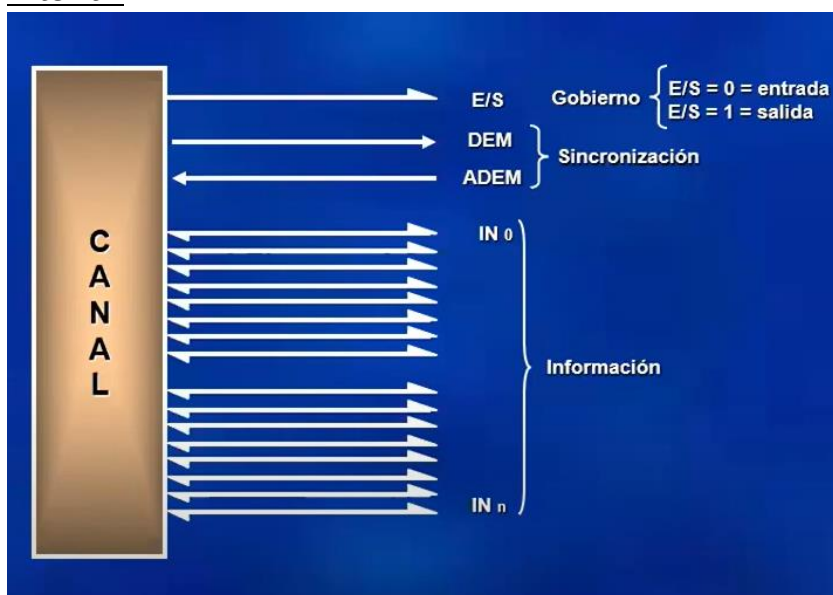
Esto es útil siempre y cuando la información esté contigua.

## 5-Encadenamiento Automático de las Transferencias

**La CPU genera en memoria un programa CANAL, permitiendo Encadenar varias transferencias sin la intervención de la CPU**

En este caso, la información no debe estar necesariamente continua. Las palabras mismas tienen, al final, la dirección de donde se encuentra la siguiente palabra, y si es la última o no.

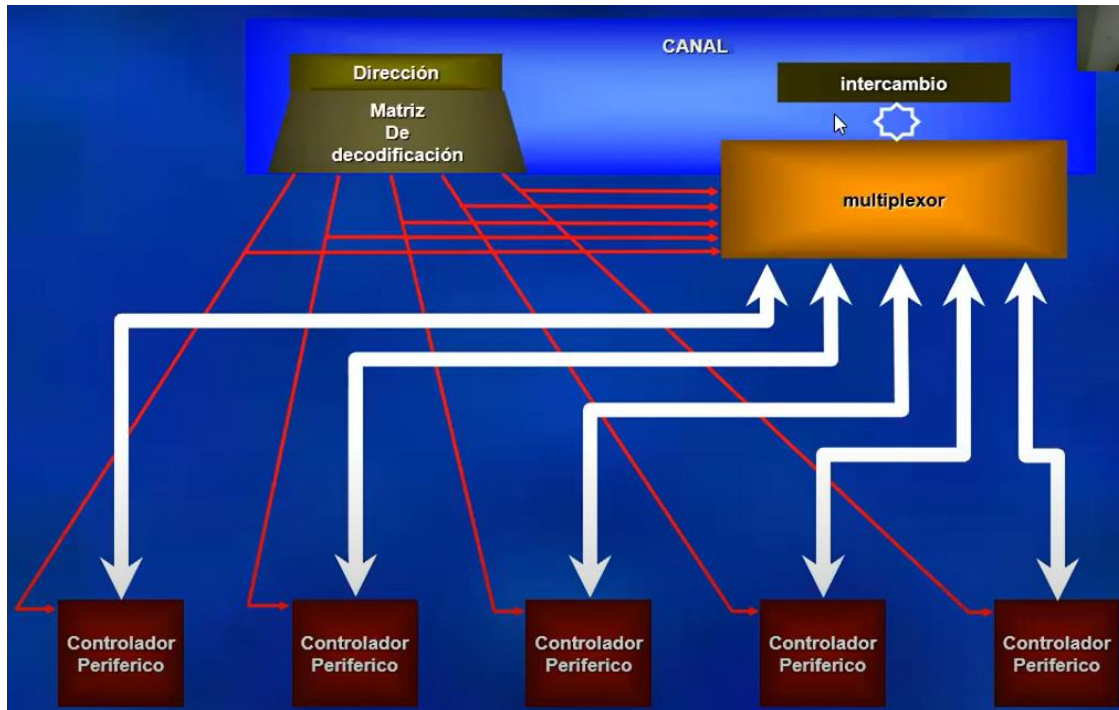
### Interfaz



Establece un protocolo de comunicación entre el controlador y el periférico.

## Esquemas de conexión de Canales con Controladores

### Concentración por Multiplaje

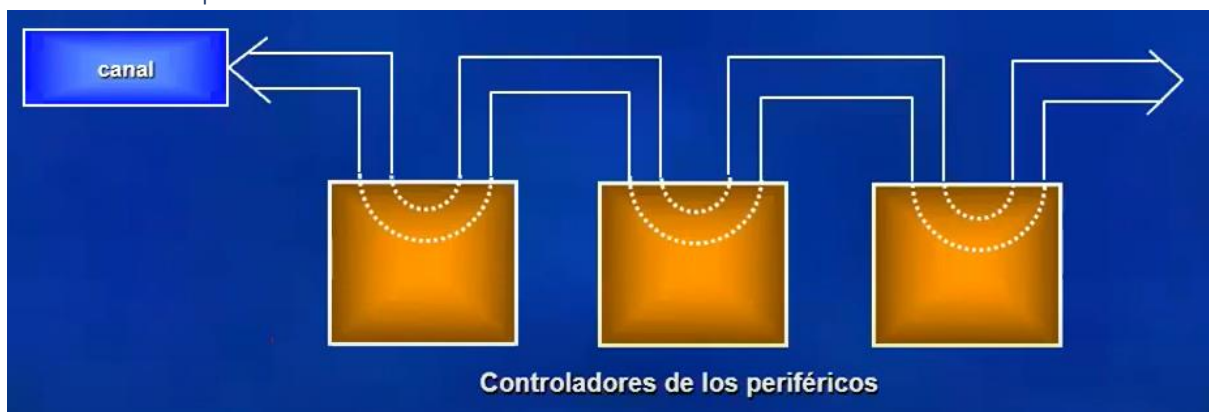


Decodifica la dirección del periférico a utilizar, luego se establece a través de un multiplexor (que es como un conmutador) con cual se establecerá la comunicación.

Ventaja: El canal puede comunicarse de manera simultanea con muchos controladores diferentes. Es muy flexible para agregar periféricos.

Desventaja: Utiliza una gran cantidad de cables y conexiones.

### Concentración por Línea ómnibus



Este esquema establece que la información posee un número/código de controlador. Pasa a través de los mismos y cada uno compara si dicha información es para ellos o no.

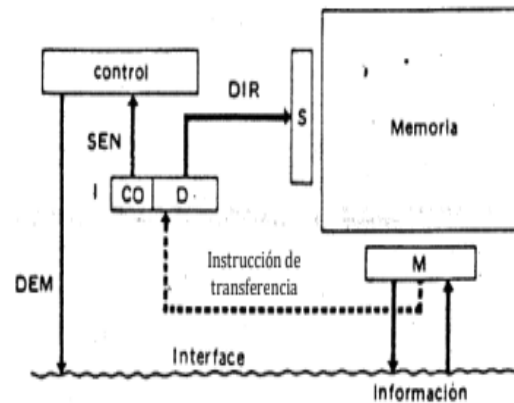
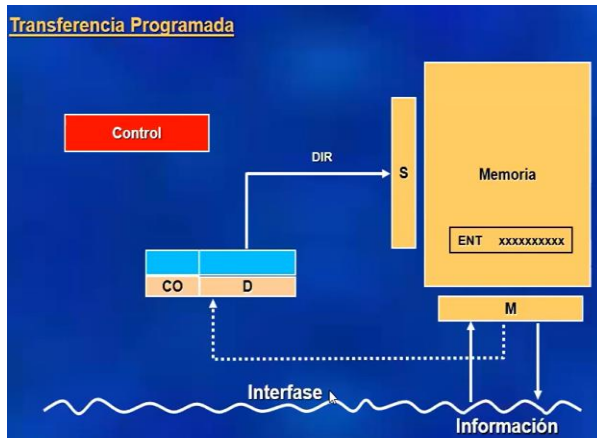
Ventajas: Se reducen las líneas de buses, es más económico.

Desventaja: La información viaja mucho más lento.

## Técnicas de Transferencias

### Transferencia Programada

#### Transferencia Programada



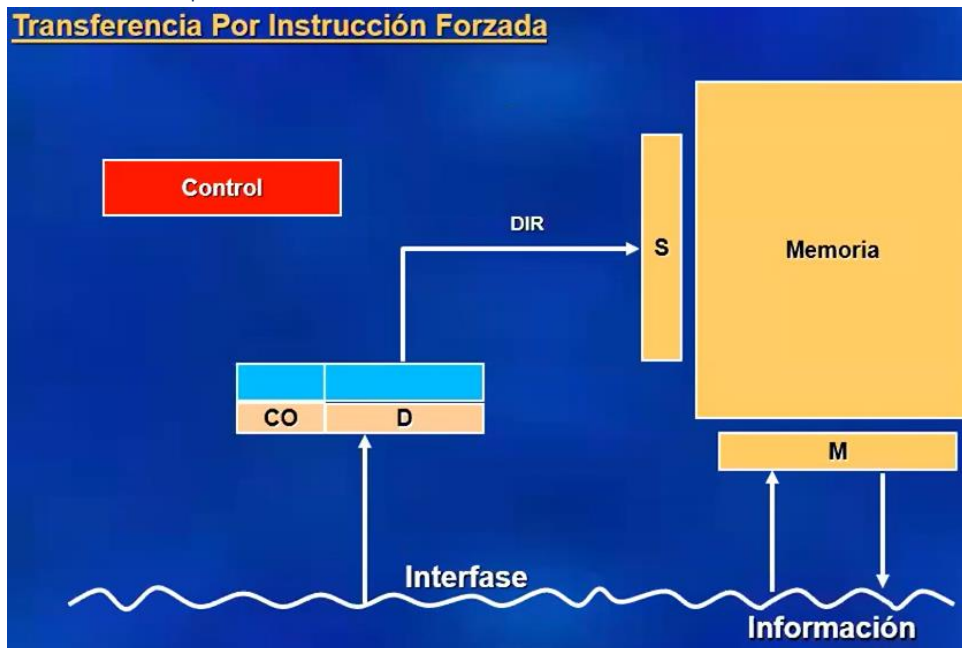
Se ejecuta cuando el programa llega a una instrucción de transferencia.

- El computador inicia la demanda de transferencia (DEM)
- La información complementaria la brinda la instrucción según la forma:
  - ENT DIR: Toma información del exterior y almacena en DIR
  - SAL DIR: Lee información de memoria de posición DIR y la envía al exterior

Se utiliza en arquitecturas donde no había un Canal como tal, sino una interfaz.

### Transferencia por Instrucción Forzada

#### Transferencia Por Instrucción Forzada

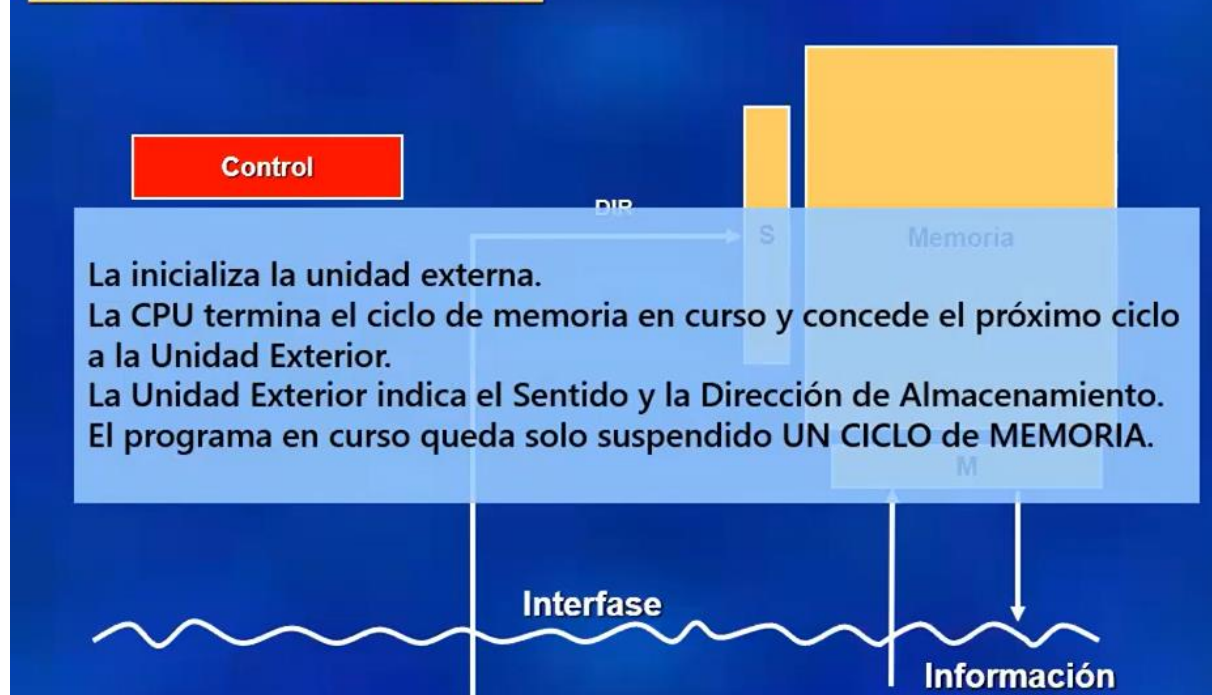


La instrucción se “fuerza”, dejando el contador de programa en espera.

Es similar a la Transferencia Programada, nada mas que en este caso es el periférico quien genera la instrucción.

Transferencia por Robo de Ciclo

### Transferencia Por Robo de Ciclo

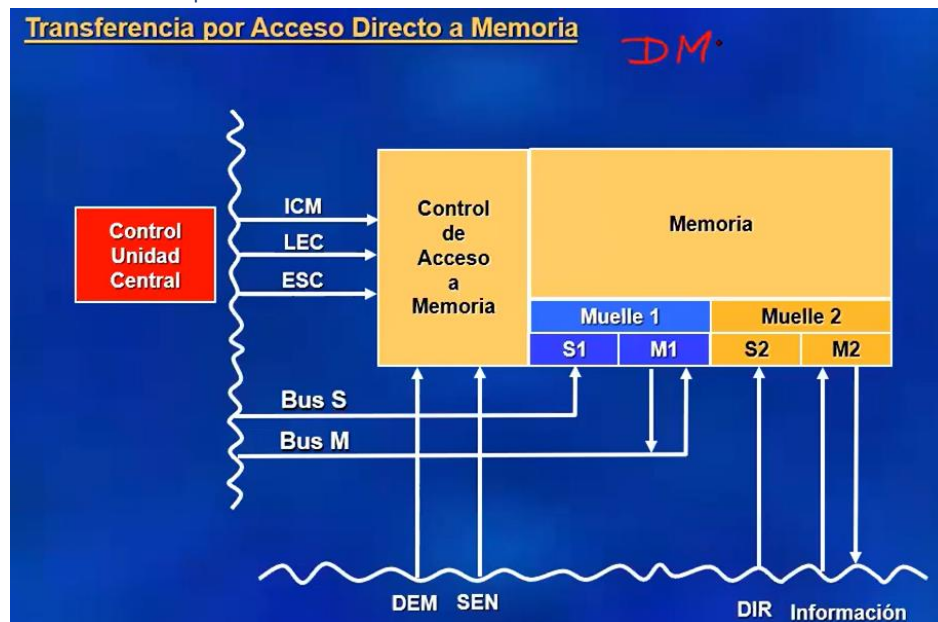


En este caso, se pausa la instrucción actual para realizar la transferencia.

Va directo a la memoria, no pasa por el registro Instrucción.

Transferencia por Acceso Directo a Memoria

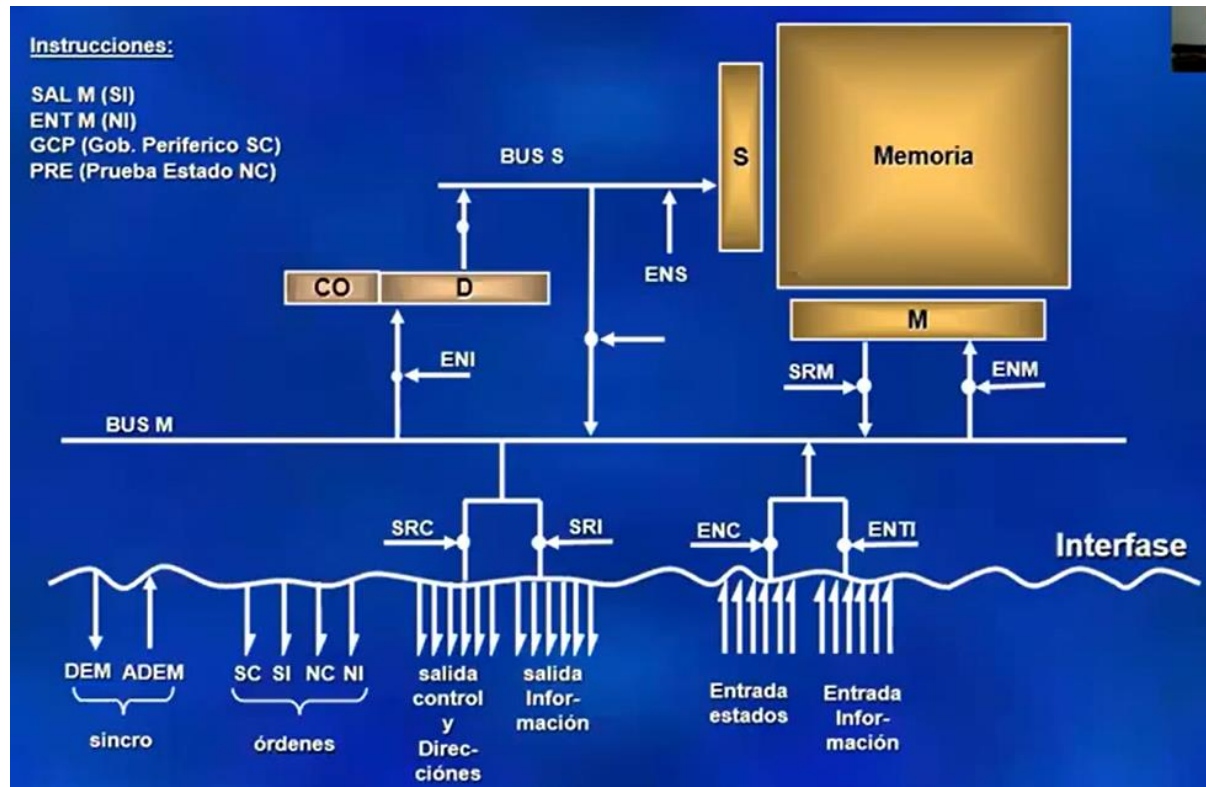
### Transferencia por Acceso Directo a Memoria



También se puede llegar a pausar la instrucción en curso si es que se solicita un ciclo al mismo tiempo, se da prioridad a la transferencia.

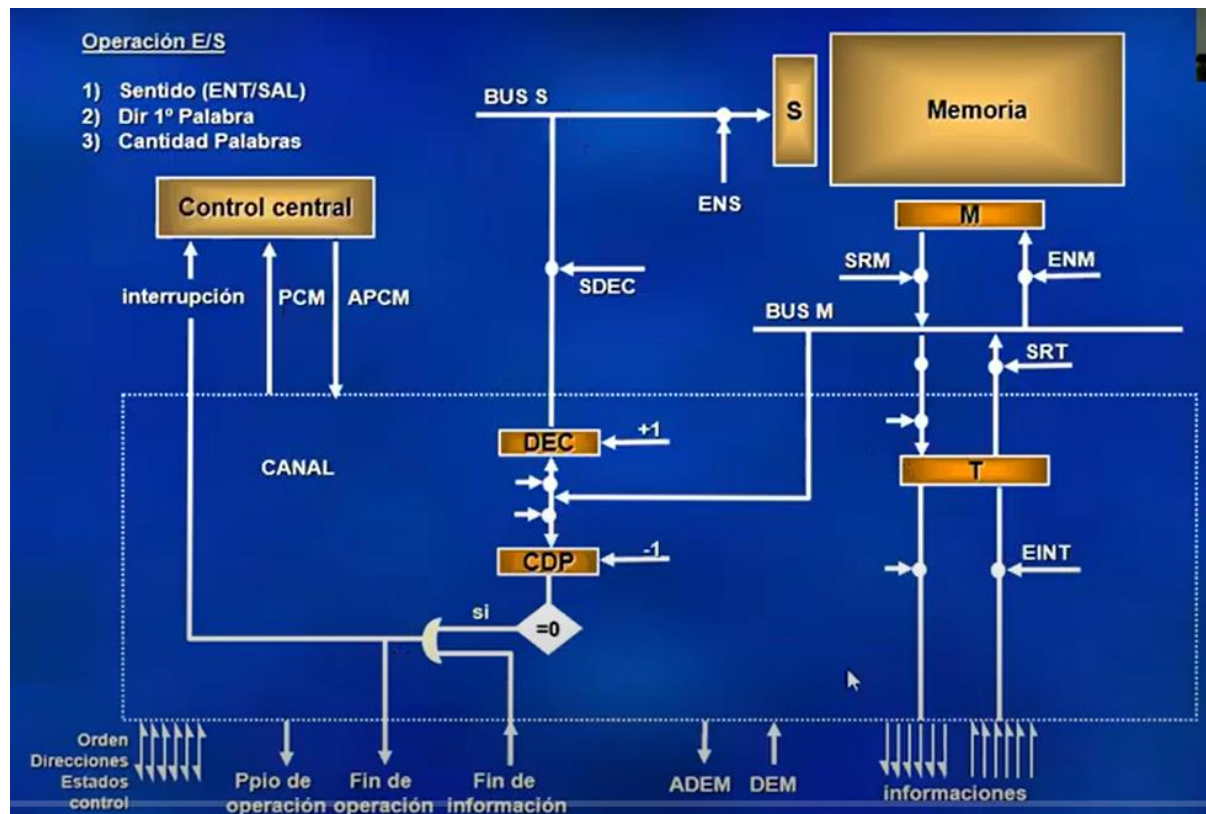
La ventaja de este modelo es que reduce los tiempos muertos de la memoria, permitiendo realizar varias operaciones.

## Enlace Programado



En este caso es el programador quien debe realizar las preguntas para efectuar las transferencias.

## Canal Automático



Esto es ya más parecido a un canal. Más evolucionada, agregando registros. Involucra robo de ciclos, pero gracias al registro T es que se minimiza la pausa del contador de programa.

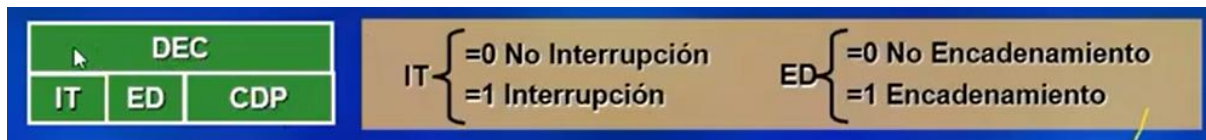
**Registro T**: libera a la memoria de almacenar una palabra, o la retiene para cuando la memoria este disponible para realizar una transferencia.

**Registro DEC** (Dirección en Curso): lleva la dirección de las palabras que deben ser transferidas desde/hacia el periférico.

**Registro CDP** (Cantidad de Palabras): Registro descontador que descuenta a medida que cada transferencia es exitosa.

Una desventaja es que la información debe estar contigua en memoria.

Encadenamiento de Datos: Esta técnica sacrifica la última palabra del grupo de palabras para indicar si la palabra anterior era la última o no, y donde se encuentra la siguiente en caso de no haber terminado.



Esto genera una gran ventaja en el performance, permitiendo la encadenación de palabras.

## Programa Canal

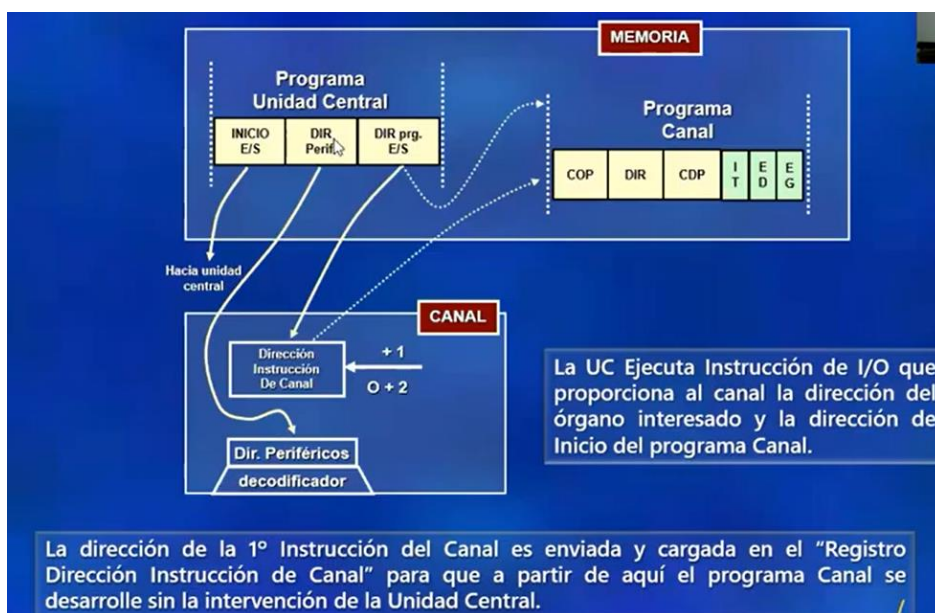
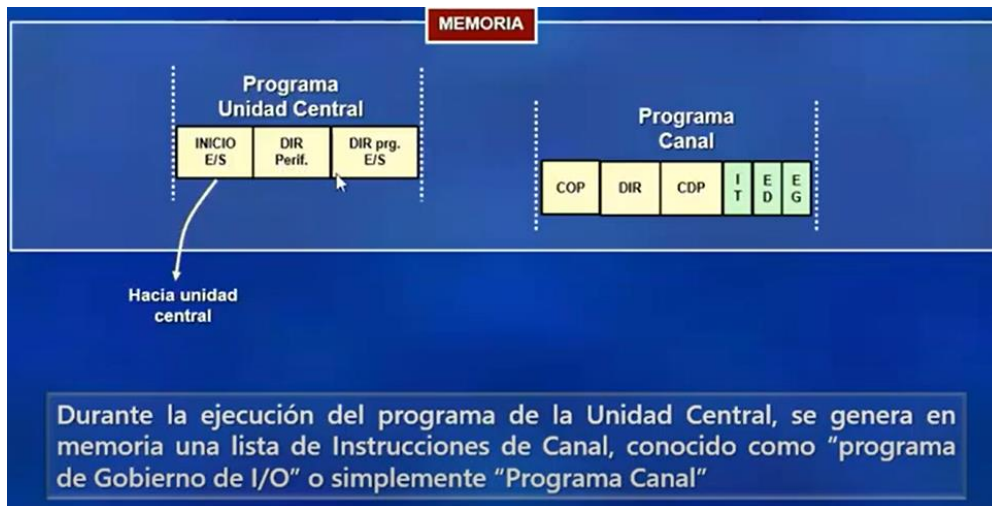
Es una unidad programable capaz de leer, codificar, y ejecutar un programa de gobierno de I/O registrado en memoria central.

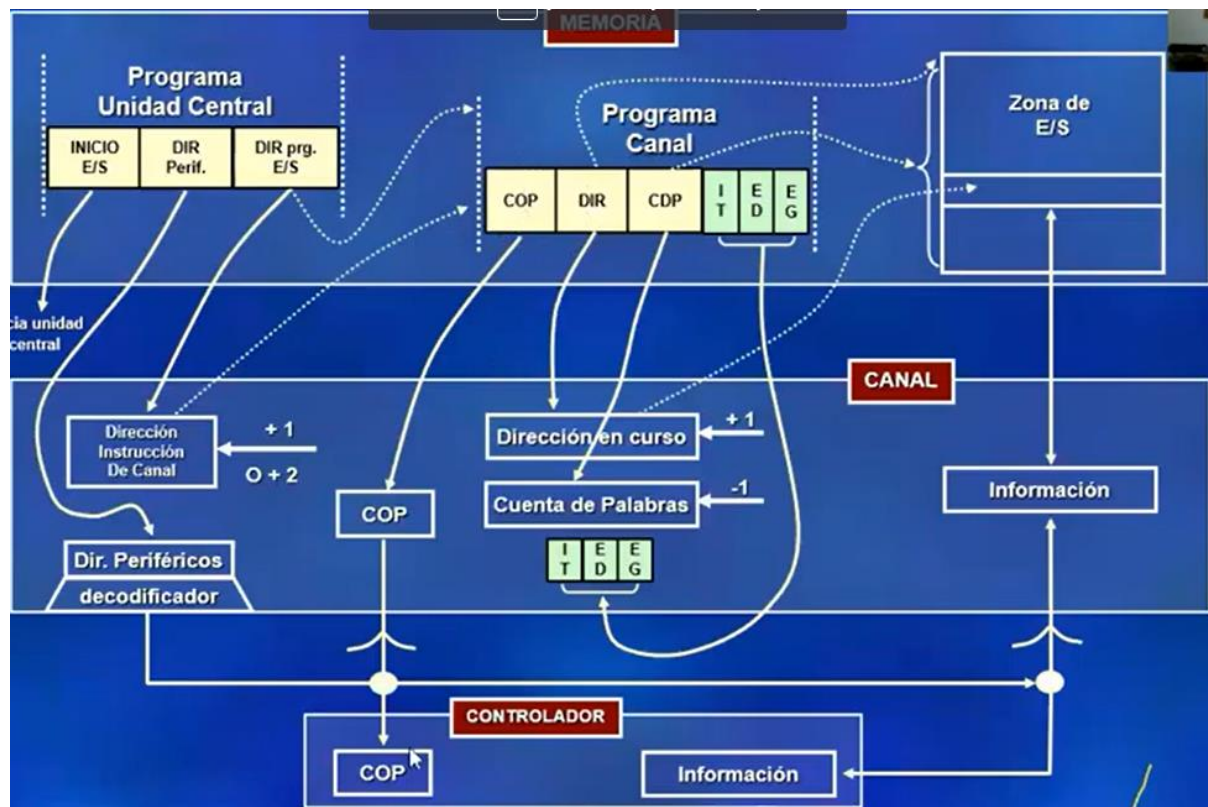
El programa de Gobierno se ejecutará con los mismos derechos con los que la Unidad Central lee, decodifica, y ejecuta un programa de procesamiento.

### Convenciones:

- La Unidad Central lee, decodifica, y ejecuta Instrucciones
- El Canal lee, decodifica, y ejecuta instrucciones de Canal
- El controlador de Periférico recibe, decodifica, y ejecuta códigos de operación de periféricos

El canal tiene su propio programa. Esto para poder darle al mismo la información del encadenamiento de datos y demás tareas complejas.





La transferencia termina cuando cuenta de palabras=0 o por señal "fin de transferencia" del controlador de periférico.

El canal controla entonces los tres indicadores:

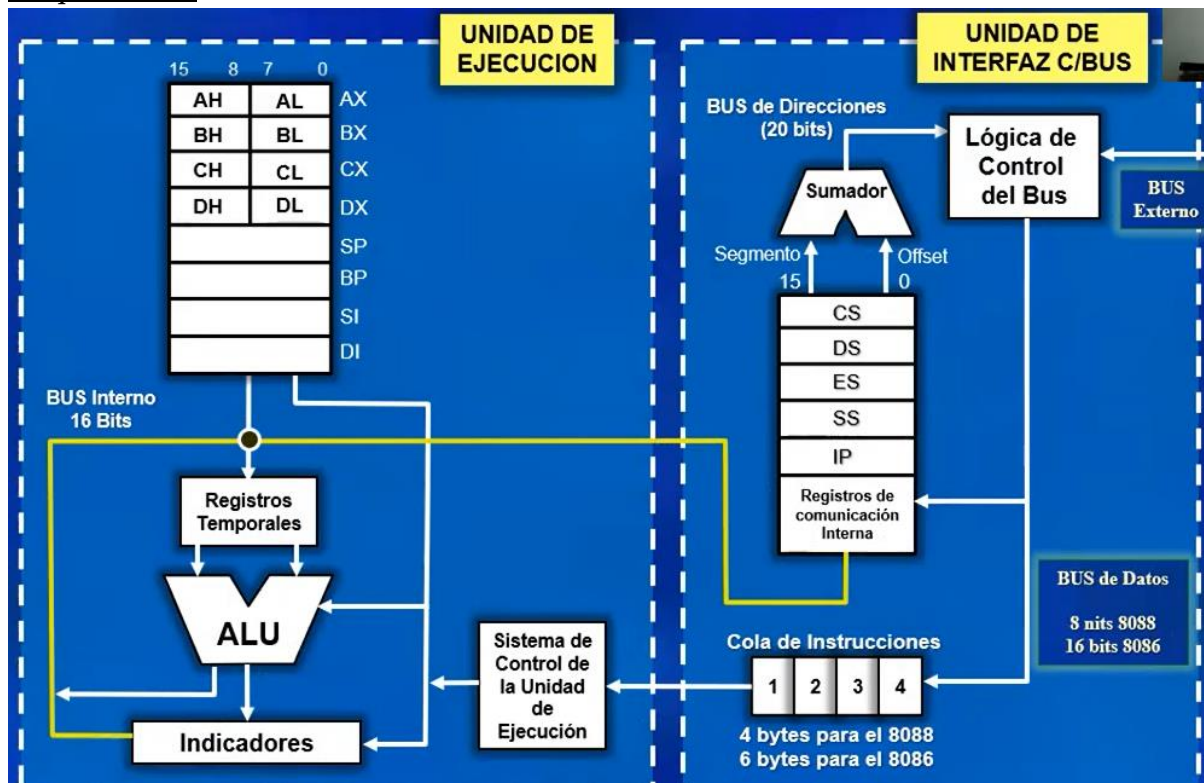
Si ED=0 canal pasa a siguiente instrucción de canal ignorando COP (Cód.Op. Periférico)

Si EG=0 canal pasa a siguiente instrucción de canal usando el nuevo COP

Si IT= V Se genera interrupción al final de una transferencia

# Arquitectura 8086

## Arquitectura



Unidad de Interfaz con el Bus: unidad dedicada a la cola de instrucciones, la comunicación con la MC en cuanto a llevar y traer instrucciones y operandos.

## Registros de Direcciones de Segmentos

La unidad de interfaz de bus (BIU) contiene cuatro registros de propósito especial de 16 bits denominados registros de segmento:

**Registro de segmento de código (CS):** se utiliza para direccionar la ubicación de la memoria en el segmento de código de la memoria, donde se almacena el programa ejecutable.

**Registro de segmento de datos (DS):** apunta al segmento de datos de la memoria donde se almacenan los datos.

**Registro de segmento Extra (ES):** también se refiere a un segmento en la memoria de la zona alta disponible para almacenar información.

**Registro de segmento de pila (SS):** se utiliza para direccionar el segmento de pila de la memoria. El segmento de pila es el segmento de memoria que se utiliza para almacenar datos de pila.

## Registros de Propósito General

**Registro AX** es el acumulador principal, utilizado para operaciones que implican entrada/salida y la mayor parte de la aritmética. Por ejemplo, las instrucciones para multiplicar, dividir y traducir suponen el uso del AX. Algunas operaciones generan código más eficiente si se refieren al AX en lugar de a los otros registros.



**Registro BX** conocido como el registro base ya que es el único de propósito general que puede ser índice para direccionamiento indexado y emplearse para cálculos. Se utiliza para almacenar el offset de una estructura de datos (normalmente indica el número de posiciones de memoria sumadas a una dirección base para conseguir una dirección absoluta específica)



**Registro CX** conocido como el registro contador. Usado para cálculos, pero su función principal es la de controlar la cantidad de ciclos de una estructura loop.



**Registro DX** conocido como el registro de datos. Algunas operaciones de entrada/salida requieren su uso, y las operaciones de multiplicación y división con cifras grandes suponen al DX y al AX trabajando juntos.

## Comentarios:

- Recomendable usar solo BX para lo que son índices/bases/desplazamientos, ya que es justamente el que se piensa para dicho objetivo.
- DX es el registro al cual van las llamadas de las interrupciones.
- Si se quisiese trabajar con palabras de 32 bits, se deberían usar 2 registros. Por ejemplo AX y DX, ya que se requiere una doble precisión.

## Los Registros de Direcciones de Datos (Apuntadores)

\*Los que están debajo de los registros de propósito general.

**Registro SP (Puntero de Pila):** Aunque es un registro de uso general, debe utilizarse sólo como puntero de pila, la cual sirve para almacenar las direcciones de retorno de subrutinas y los datos temporarios (mediante las instrucciones PUSH y POP). Al introducir (push) un valor en la pila a este registro se le resta dos, mientras que al extraer (pop) un valor de la pila este a registro se le suma dos.



**Registro BP (Puntero de Base):** Generalmente se utiliza para realizar direccionamiento indirecto dentro de la pila



**Registro SI o Índice Fuente:** Sirve como puntero fuente para las operaciones con cadenas. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde comienza los datos que se quieran leer dentro de un segmento de datos.



**Registro DI o Índice Destino:** Sirve como puntero destino para las operaciones con cadenas. También sirve para realizar direccionamiento indirecto. Lleva la dirección donde terminan los datos que se quieran leer dentro de un segmento de datos.

## Registros de Banderas

Maneja información de estado y de control de las operaciones del microprocesador.

**CF - (Carry Flag)** Bandera de acarreo. Indica acarreo en las instrucciones aritméticas.

**OF - (Overflow Flag)** Bandera de desbordamiento aritmético.

**ZF - (Zero Flag)** Bandera de resultado cero o comparación igual.

**SF - (Sign Flag)** Bandera de resultado o comparación negativa.

**PF - (Parity Flag)** Bandera de paridad, utilizada en la verificación de la transferencia de bytes entre dos componentes de la computadora.

**AF - (Auxiliar Flag)** Bandera auxiliar. Indica si hay necesidad de ajuste en las operaciones aritméticas con números BCD.

Tres banderas de control que registran el modo de funcionamiento de la computadora.

**DF (Direction Flag)** Bandera de dirección. Controla la dirección (hacia adelante o hacia atrás) en las operaciones con cadenas de caracteres incrementando o decrementando automáticamente los registros índices (SI y DI).

**IF (Interrupt Flag)** Bandera de interrupciones. Indica si están disponibles o no las interrupciones de los dispositivos externos.

**TF (Trap Flag)** Bandera de atrape. Controla la operación modo paso a paso (usada por el programa DEBUG).

## Modos de Direcccionamiento

- 1-Direcccionamiento de registro
- 2-Direcccionamiento inmediato
- 3-Direcccionamiento directo
- 4-Direcccionamiento indirecto mediante registro
- 5-Direcccionamiento indirecto por registro base
- 6-Direcccionamiento indexado
- 7-Direcccionamiento indexado respecto a una base

## Datos Extras

En la segmentación de memoria (ES, CS, SS, DS), se usan solo 4 bits, ya que se sabe que el de menor peso es un 0. Es por esto por lo que se debe multiplicar por 10 (por la base) cuando se quiere pasar cualquier valor a la Lógica del Bus.